



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

Trabajo fin de Grado

Grado en Ingeniería Informática (Ingeniería del Software)

BarGAIN: Sistema inteligente de optimización de la cesta de la compra

**Realizado por
Nicolás Parrilla Geniz**

**Dirigido por
Juan Vicente Gutiérrez Santacreu**

**Departamento
Matemática Aplicada I**

Sevilla, Junio 2026

Resumen

Hacer la compra cada semana plantea una decisión que casi nadie tiene tiempo de resolver bien: en qué tiendas comprar para gastar menos sin recorrer media ciudad. Los precios varían notablemente de un supermercado a otro, pero compararlos a mano resulta tedioso y, cuando se hace, rara vez se tiene en cuenta lo que cuesta desplazarse hasta el establecimiento más barato.

BarGAIN es una aplicación móvil y web pensada para resolver precisamente eso. A partir de la lista de la compra del usuario y de su ubicación, calcula la combinación de tiendas más conveniente atendiendo a la vez a tres factores: cuánto cuesta la cesta, cuánto hay que desplazarse y cuánto tiempo lleva. Cada persona decide a qué da más importancia —ahorrar, ir cerca o tardar poco— y la aplicación ajusta su recomendación en consecuencia.

Además de comparar precios entre supermercados, la aplicación incorpora tres capacidades que la distinguen de los comparadores habituales: propone la ruta de compra más ventajosa repartida entre varias tiendas; permite crear la lista a partir de una foto de un ticket o de una nota escrita a mano, sin teclear; y ofrece un asistente conversacional al que se puede preguntar en lenguaje corriente por precios, ofertas o por la mejor manera de organizar la compra. Un portal web complementario da cabida al pequeño comercio local, que puede publicar sus precios y promociones para competir en igualdad de condiciones con las grandes cadenas.

El proyecto se ha llevado a cabo recorriendo el ciclo completo de la ingeniería del software —análisis de requisitos, diseño, implementación, pruebas y despliegue—. El resultado es un prototipo plenamente funcional, accesible de forma pública y verificado mediante pruebas automatizadas y ensayos en un dispositivo real. La memoria recoge también las decisiones de diseño adoptadas, las limitaciones de esta primera versión y las líneas de mejora previstas para convertir BarGAIN en un producto a mayor escala.

Abstract

Doing the weekly grocery shopping involves a decision few people have time to get right: which shops to visit in order to spend less without driving halfway across town. Prices vary widely from one supermarket to another, but comparing them by hand is tedious and, even when done, it rarely accounts for the cost of travelling to the cheapest store.

BarGAIN is a mobile and web application designed to solve exactly that. Starting from the user's shopping list and location, it works out the most convenient combination of shops by weighing three factors at once: how much the basket costs, how far one has to travel, and how long it takes. Each person chooses what matters most—saving money, staying nearby, or spending little time—and the application adapts its recommendation accordingly.

Beyond comparing supermarket prices, the application offers three capabilities that set it apart from conventional comparators: it suggests the most advantageous shopping route across several shops; it lets users build their list from a photo of a receipt or a handwritten note, with no typing required; and it provides a conversational assistant that answers everyday questions about prices, offers, or how best to organise the shopping. A complementary web portal welcomes small local businesses, which can publish their prices and promotions to compete on equal terms with the large chains.

The project was carried out following the full software engineering lifecycle—requirements analysis, design, implementation, testing, and deployment—. The outcome is a fully functional, publicly accessible prototype, validated through both automated tests and trials on a real device. The report also documents the design decisions made, the limitations of this first version, and the planned improvements toward turning BarGAIN into a larger-scale product.

Agradecimientos

A mi tutor, Juan Vicente Gutiérrez Santacreu, por su orientación constante, su criterio técnico y su apoyo durante todo el desarrollo del TFG.

A mi familia y personas cercanas, por su paciencia y motivación en las fases de mayor carga de trabajo.

A la comunidad de software libre y a la documentación técnica de las herramientas utilizadas, que han sido clave para avanzar con rigor en el diseño, la implementación y la validación de BarGAIN.

Índice general

Índice general	IV
Índice de cuadros	VIII
Índice de figuras	IX
Acrónimos	XI
1 Introducción y objetivos del proyecto	1
1.1 Contexto y motivación	1
1.2 Problema abordado	1
1.3 Objetivo principal	1
1.4 Objetivos específicos	1
1.5 Alcance del proyecto	2
1.6 Tecnologías clave	2
1.7 Estructura de la memoria	3
2 Análisis de antecedentes y aportación realizada	4
2.1 Contexto del sector	4
2.2 Estado del arte en comparación de precios	4
2.3 Tecnologías relevantes para BarGAIN	4
2.3.1 Geoespacial y rutas	4
2.3.2 Ingesta de datos de precio	5
2.3.3 OCR y asistente	5
2.4 Aportación diferencial de BarGAIN	5
3 Comparación con otras alternativas	6
3.1 Competidores considerados	6
3.2 Matriz funcional resumida	6
3.3 Brechas estratégicas detectadas	7
3.4 Posicionamiento de BarGAIN	7
3.5 Riesgos competitivos	7
4 Análisis temporal y costes de desarrollo	8
4.1 Planificación temporal	8
4.2 Metodología de trabajo y trazabilidad del proceso	8
4.2.1 El framework GSD: flujo de trabajo y comandos	9
4.2.2 Planificación viva: el árbol .planning/	11
4.2.3 Trazabilidad de tareas y evidencias	11
4.2.4 Agentes de IA con contrato de contexto explícito	11
4.2.5 Registro de errores y reglas derivadas	11

4.2.6	Valoración del enfoque	12
4.3	Estimación de costes	12
4.4	Riesgos y mitigaciones	13
4.5	Resultado de planificación	13
5	Análisis de requisitos	14
5.1	Actores del sistema	14
5.2	Requisitos de información	14
5.3	Requisitos funcionales	15
5.4	Criterios de aceptación del producto	17
5.5	Requisitos no funcionales	17
5.6	Reglas de negocio	18
5.7	Diagramas de casos de uso	18
5.8	Trazabilidad	18
6	Diseño e Implementación	22
6.1	Arquitectura del Sistema	22
6.1.1	Visión general	22
6.1.2	Patrón de organización interna del backend	23
6.1.3	Comunicación entre componentes	23
6.2	Modelo de Datos	24
6.2.1	Módulo de usuarios	24
6.2.2	Módulo de productos	24
6.2.3	Módulo de tiendas	24
6.2.4	Módulo de precios	24
6.2.5	Módulo del optimizador	25
6.2.6	Índices clave	25
6.2.7	Modelo entidad-relación	25
6.2.8	Diagrama de clases	25
6.3	Diseño de la API REST	25
6.3.1	Estructura general	25
6.3.2	Endpoints principales	27
6.3.3	Autenticación y autorización	27
6.4	Implementación del Backend	28
6.4.1	Módulo de notificaciones	28
6.4.2	Módulo de portal business	28
6.5	El Algoritmo de Optimización de Rutas	28
6.5.1	Formulación del problema	28
6.5.2	Fase 1 — Resolución de los artículos de la lista	30
6.5.3	Fase 2 — Consolidación de paradas	30
6.5.4	Fase 3 — Cálculo de distancias y tiempos	31
6.5.5	Fase 4 — Ordenación de la ruta	31
6.5.6	Fase 5 — Persistencia y desambiguación interactiva	32
6.5.7	Diagrama de secuencia del optimizador	32
6.5.8	Evolución respecto al diseño inicial	32
6.5.9	Complejidad y salvaguardas de rendimiento	34
6.6	Módulo de Web Scraping	34
6.6.1	Consideraciones legales y éticas del scraping	34
6.7	Módulo OCR	35
6.7.1	Diagrama de secuencia OCR	35
6.8	Integración del Asistente LLM	35
6.9	Infraestructura y Despliegue	37

6.9.1	Contenedores Docker	37
6.9.2	Pipeline CI/CD	37
6.9.3	Gestión de la configuración sensible	38
6.10	Diseño de la Interfaz de Usuario	38
6.10.1	Sistema de diseño	38
6.10.2	Pantallas principales	38
6.10.3	Adaptación a uso web (responsive y conveniencias de escritorio)	39
7	Manual de Usuario	40
7.1	Introducción	40
7.2	Despliegue y acceso al sistema	40
7.2.1	Acceso al portal para empresas (GitHub Pages)	40
7.2.2	Ejecución de la aplicación de usuario en local (Expo web)	41
7.2.3	Despliegue del frontend en Docker (la forma más simple)	41
7.2.4	Instalación de la aplicación en iPhone	42
7.3	Registro y acceso al sistema	42
7.4	Pantalla principal	43
7.5	Gestión de listas de la compra	43
7.6	Catálogo y comparativa de precios	46
7.7	Mapa de tiendas	48
7.8	Alertas de precio y notificaciones	50
7.9	Perfil y preferencias del usuario	50
7.10	Ruta de compra optimizada	53
7.11	Reconocimiento de tickets y listas (OCR)	54
7.12	Asistente conversacional	54
7.13	Lista de comprobación en pantalla de bloqueo (iOS)	55
7.14	La aplicación en el navegador de escritorio	56
7.15	Portal Business para PYMEs	57
7.16	Resolución de incidencias comunes	61
8	Pruebas y Resultados	62
8.1	Objetivo	62
8.2	Estrategia de testing	62
8.3	Entorno de ejecución	62
8.4	Suite de tests backend	63
8.4.1	Tests de integración	63
8.4.2	Cobertura	63
8.5	Tests E2E con Playwright	64
8.5.1	Configuración Playwright	64
8.6	UAT manual en dispositivo móvil	64
8.7	Validación de Requisitos No Funcionales	64
8.7.1	RNF-002: Disponibilidad del entorno de staging	64
8.7.2	RNF-004: Usabilidad y flujo máximo de 3 interacciones	65
8.7.3	RNF-005: Escalabilidad para 10.000 usuarios	65
8.8	Resumen de resultados	66
8.9	Conclusión	66
9	Conclusiones	67
9.1	Grado de cumplimiento	67
9.2	Aportaciones principales del trabajo	67
9.3	Limitaciones	68
9.4	Lecciones aprendidas	69

9.5	Trabajo futuro	69
9.6	Cierre	70
A	Relación de endpoints de la API	71
B	Capturas de la versión de escritorio	73
	Referencias	75

Índice de cuadros

3.1	Comparativa de capacidades clave (elaboración propia, marzo de 2026) . . .	6
4.1	Resumen por fases del proyecto	8
4.2	Coste estimado del desarrollo	12
4.3	Estimación del coste operativo mensual (escenario inicial de producción) . .	13
5.1	Agrupación de requisitos funcionales	15
5.2	Catálogo de requisitos funcionales	15
5.3	Trazabilidad de dominios funcionales a módulos y suites de prueba	21
6.1	Índices principales de la base de datos	25
6.2	Endpoints de autenticación y usuarios	27
6.3	Endpoints del optimizador, OCR y asistente	27
6.4	Salvaguardas de rendimiento del optimizador	34
8.1	Suite de tests de integración backend	63
8.2	Suite de tests E2E con Playwright	64
8.3	Resultados UAT en dispositivo móvil	64
8.4	Resumen de resultados de la estrategia de testing	66
A.1	Endpoints de la API por módulo	71

Índice de figuras

5.1	Casos de uso del Consumidor	19
5.2	Casos de uso del Comercio y Administrador	20
6.1	Arquitectura por capas de BarGAIN	23
6.2	Diagrama entidad-relación de BarGAIN	26
6.3	Diagrama de clases del dominio	26
6.4	Diagrama de secuencia del proceso de optimización de ruta	33
6.5	Diagrama de secuencia del flujo OCR	36
7.1	Acceso al sistema: inicio de sesión (izquierda) y registro de un nuevo usuario (derecha)	43
7.2	Pantalla principal de la aplicación móvil, con notificaciones, listas activas, tiendas cercanas y alertas de precio	44
7.3	Gestión de listas: colección de listas del usuario (izquierda) y detalle de una lista con controles de cantidad y verificación (derecha)	45
7.4	Plantillas de lista: instanciación de compras recurrentes a partir de una plantilla guardada	45
7.5	Catálogo de productos con filtros y añadido rápido (izquierda) y comparativa de precios por tienda de un producto (derecha)	46
7.6	Propuesta de alta de un nuevo producto en el catálogo, sujeta a aprobación por el administrador	47
7.7	Mapa de tiendas con marcadores por cadena y panel de establecimientos (izquierda) y perfil de tienda con sus productos y precios (derecha)	48
7.8	Tiendas favoritas del usuario, con acceso directo a su perfil	49
7.9	Bandeja de notificaciones con avisos de precio, promociones y listas compartidas (izquierda) y gestión de alertas de precio con su objetivo (derecha)	50
7.10	Perfil del usuario (izquierda) y configuración del optimizador con radio, paradas y pesos del <i>scoring</i> multicriterio (derecha)	51
7.11	Operaciones sobre la cuenta: edición del perfil (izquierda) y cambio de contraseña (derecha)	52
7.12	Ruta de compra optimizada: parámetros, resultado agregado, desglose por parada y resolución de ambigüedades semánticas con recálculo	53
7.13	Asistente OCR: captura del ticket o lista (izquierda) y revisión de los ítems reconocidos con su nivel de confianza (derecha)	54
7.14	Asistente conversacional con sugerencias de inicio y campo de consulta; las respuestas con productos incluyen fichas de añadido directo a la lista	55
7.15	Versión de escritorio: pantalla principal con accesos horizontales (izquierda) y catálogo en rejilla multicolumna con soporte de teclado (derecha)	56
7.16	Página de presentación del portal Business, con la propuesta de valor para el comercio local y el proceso de alta en tres pasos	57

7.17 Acceso al portal Business: inicio de sesión (izquierda) y registro de una cuenta de comercio (derecha)	57
7.18 <i>Onboarding</i> del comercio: alta de los datos legales y comerciales que un administrador verificará antes de activar la cuenta	58
7.19 Centro de operaciones del comercio: indicadores de actividad, salud de promociones y ajustes recientes de precio	59
7.20 Gestión del surtido: resumen de productos del comercio (izquierda) y carga masiva de catálogo mediante CSV con validación (derecha)	59
7.21 Gestión de precios: tabla editable con panel lateral de edición de precio base, unitario y oferta con caducidad	60
7.22 Promociones del comercio con analítica básica (izquierda) y perfil del negocio con umbral de alerta competitiva (derecha)	60
7.23 Panel de administración: verificación y aprobación de perfiles PYME y de propuestas de producto	61
B.1 Comparativa de precios con cabecera fija y exportación CSV (izquierda) y detalle de lista con reordenado por arrastre y exportación (derecha)	73
B.2 Colecciones en rejilla responsiva: listas de la compra (izquierda) y plantillas (derecha)	73
B.3 Mapa con panel lateral de tiendas (izquierda) y perfil de tienda a dos columnas (derecha)	74
B.4 Tiendas favoritas en rejilla (izquierda) y asistente conversacional centrado con copia por mensaje (derecha)	74
B.5 Ruta optimizada con exportación del itinerario (izquierda) y módulo OCR adaptado al escritorio (derecha)	74
B.6 Bandeja de notificaciones (izquierda) y alertas de precio (derecha) en la versión de escritorio	74

Acrónimos

- ADR** *Architecture Decision Record*: registro de decisión arquitectónica.
- API** *Application Programming Interface*: interfaz de programación de aplicaciones.
- CI/CD** *Continuous Integration / Continuous Delivery*: integración y entrega continuas.
- CRUD** *Create, Read, Update, Delete*: operaciones básicas de persistencia.
- CSV** *Comma-Separated Values*: formato tabular de texto plano.
- DRF** *Django REST Framework*: framework para construir APIs REST sobre Django.
- E2E** *End-to-End*: pruebas de extremo a extremo del sistema integrado.
- EAN** *European Article Number*: código de barras normalizado de producto.
- GIN** *Generalized Inverted Index*: índice invertido generalizado de PostgreSQL.
- GiST** *Generalized Search Tree*: familia de índices de PostgreSQL usada por PostGIS.
- HMR** *Hot Module Replacement*: recarga en caliente de módulos en desarrollo.
- HU** Historia de Usuario.
- IA** Inteligencia Artificial.
- JSON** *JavaScript Object Notation*: formato de intercambio de datos.
- JWT** *JSON Web Token*: estándar de testigos de autenticación firmados.
- LLM** *Large Language Model*: modelo de lenguaje de gran tamaño.
- LSSI-CE** Ley de Servicios de la Sociedad de la Información y de Comercio Electrónico.
- OCR** *Optical Character Recognition*: reconocimiento óptico de caracteres.
- ORS** OpenRouteService: servicio de cálculo de rutas sobre OpenStreetMap.
- PYME** Pequeña y Mediana Empresa.
- REST** *Representational State Transfer*: estilo arquitectónico de APIs web.
- RF** Requisito Funcional.
- RGPD** Reglamento General de Protección de Datos (Reglamento (UE) 2016/679).
- RI** Requisito de Información.
- RN** Regla de Negocio.

RNF Requisito No Funcional.

SRID *Spatial Reference System Identifier*: identificador de sistema de referencia espacial.

TFG Trabajo Fin de Grado.

TSP *Travelling Salesman Problem*: problema del viajante de comercio.

UAT *User Acceptance Testing*: pruebas de aceptación de usuario.

UI *User Interface*: interfaz de usuario.

URL *Uniform Resource Locator*: localizador uniforme de recursos.

VRP *Vehicle Routing Problem*: problema de enrutamiento de vehículos.

WCAG *Web Content Accessibility Guidelines*: pautas de accesibilidad para contenido web.

Introducción y objetivos del proyecto

1.1— Contexto y motivación

El consumidor español afronta la compra cotidiana en un contexto de fuerte variabilidad de precios entre cadenas, ausencia de una fuente unificada y alto coste temporal para comparar ofertas manualmente. Aunque el ahorro potencial entre supermercados es significativo, en la práctica ese ahorro se pierde cuando no se considera el coste de desplazamiento ni el tiempo de ruta.

BarGAIN nace para resolver ese problema como una plataforma de compra inteligente que integra precio, distancia y tiempo en una única decisión. El sistema combina una app móvil para consumidores, un portal web para pequeñas y medianas empresas (PYME), y un backend modular con capacidades geoespaciales, reconocimiento óptico de caracteres (OCR) y asistencia conversacional.

1.2— Problema abordado

Las soluciones actuales de comparación de supermercados suelen responder a la pregunta “qué tienda tiene el precio más bajo”, pero no a la pregunta completa “qué compra me conviene realmente según mi ubicación y mis preferencias”.

El problema de BarGAIN se formula como una optimización multicriterio en la que intervienen:

- coste total de la cesta,
- distancia adicional de desplazamiento,
- tiempo total estimado de compra y ruta.

1.3— Objetivo principal

Desarrollar una aplicación móvil y web que optimice la cesta de la compra del usuario, calculando la combinación de supermercados y comercios locales que ofrece la mejor relación precio–distancia–tiempo según preferencias configurables.

1.4— Objetivos específicos

1. Ofrecer un servicio centralizado que reúna productos, tiendas, precios y listas de la compra, accesible tanto desde la aplicación de consumo como desde el portal de comercios.

2. Localizar las tiendas próximas al usuario y preparar el cálculo de rutas a partir de su ubicación.
3. Recomendar la mejor combinación de tiendas ponderando precio, distancia y tiempo según las preferencias que cada usuario configure.
4. Permitir crear la lista de la compra a partir de una foto de un ticket o de una nota manuscrita, con revisión y corrección por parte del usuario antes de guardarla.
5. Proporcionar un asistente conversacional que responda en lenguaje natural a consultas relacionadas con la compra, ceñido a ese dominio.
6. Integrar a las grandes cadenas de supermercados en la comparativa manteniendo su catálogo de precios actualizado de forma automática y periódica, de modo que las recomendaciones partan siempre de datos reales y vigentes.
7. Ofrecer al pequeño comercio un portal propio desde el que gestionar sus precios y promociones.
8. Garantizar la calidad mediante pruebas automatizadas a distintos niveles y ensayos manuales en los flujos que dependen del dispositivo móvil.
9. Disponer de un despliegue público y reproducible que permita probar el sistema en condiciones reales.

1.5— Alcance del proyecto

El alcance del Trabajo Fin de Grado (TFG) cubre el ciclo completo de ingeniería del software:

- análisis y requisitos,
- diseño arquitectónico y de datos,
- implementación backend y frontend,
- pruebas funcionales y no funcionales,
- despliegue en entorno staging,
- documentación técnica y memoria final.

Quedan fuera del alcance de esta versión: despliegue productivo global, acuerdos formales con todas las cadenas para interfaces de programación de aplicaciones (API) oficiales, y la versión final de Live Activities nativas en iOS (registro de decisión arquitectónica, ADR-010).

1.6— Tecnologías clave

El sistema se apoya en un stack full-stack moderno y coherente:

- **Backend:** Django 5 con Django REST Framework (DRF), Celery y Redis.
- **Datos:** PostgreSQL 16 + PostGIS 3.4.
- **Frontend:** React Native con Expo + companion web en React.

- **Inteligencia artificial (IA):** Google Cloud Vision API (OCR) y Gemini vía back-end proxy.
- **Optimización:** OR-Tools + cálculo de distancias (OpenRouteService + fallback haversine).
- **Ops:** Docker, GitHub Actions, Render, Sentry.

1.7— Estructura de la memoria

La memoria se organiza para reflejar el recorrido completo del proyecto: contexto y objetivos, estado del arte, comparativa, planificación, requisitos, diseño e implementación, manual de usuario, pruebas y resultados, y conclusiones. Cierran el documento los apéndices —con la relación de endpoints de la API y las capturas de la versión de escritorio— y las referencias bibliográficas. Al inicio se incluye un glosario con los acrónimos empleados.

Análisis de antecedentes y aportación realizada

2.1— Contexto del sector

El mercado de distribución alimentaria en España presenta una combinación de gran competencia entre cadenas, alta sensibilidad al precio y fuerte fragmentación de oferta. Durante los últimos años, la evolución al alza del índice de precios de la alimentación registrada por el IPC (Instituto Nacional de Estadística, 2026) ha incrementado la necesidad de herramientas que permitan comprar mejor sin aumentar el tiempo invertido.

Aunque existen comparadores de precios consolidados, la mayor parte se centra en mostrar información de coste por tienda, sin integrar de forma nativa el impacto logístico de desplazarse a varios establecimientos.

2.2— Estado del arte en comparación de precios

La evolución observada puede resumirse en tres generaciones:

1. **Directorios de precios:** listados estáticos o semiactualizados, con poca cobertura y baja frecuencia de refresco.
2. **Comparadores multisupermercado:** soluciones como Soysuper (Soysuper S.L., 2026), Findit u OCU Market (Organización de Consumidores y Usuarios (OCU), 2026), con mejor catálogo y funcionalidades de lista.
3. **Personalización con IA:** tendencia emergente en la que se introducen recomendaciones contextuales y asistencia conversacional.

En España, el salto completo hacia la tercera generación todavía es limitado, especialmente en el segmento de cesta alimentaria diaria.

2.3— Tecnologías relevantes para BarGAIN

2.3.1. Geoespacial y rutas

PostGIS (PostGIS Project Steering Committee, 2026; Obe y Hsu, 2021) permite resolver consultas de proximidad con precisión y rendimiento, mientras que OR-Tools (Google OR-Tools Team, 2026) facilita modelar el problema de ordenación de paradas como una variante del problema del viajante de comercio (TSP) y del problema de enrutamiento de vehículos (VRP) (Applegate, Bixby, Chvatal, y Cook, 2006) en instancias pequeñas (máximo 3–4 paradas en nuestro caso). Para el cálculo de distancias reales por carretera

existen servicios de enrutamiento sobre datos de OpenStreetMap (Luxen y Vetter, 2011), de los que el proyecto adopta OpenRouteService (Heidelberg Institute for Geoinformation Technology (HeiGIT), 2026).

2.3.2. Ingesta de datos de precio

Para que el sistema sea útil, necesita conocer los precios reales de muchos productos en muchas tiendas y mantenerlos al día. Reunir esa información es uno de los retos principales del proyecto, porque cada cadena publica sus precios de una forma distinta y los cambia con frecuencia. El planteamiento que adopta BarGAIN consiste en recopilar de manera automática y periódica los precios que los supermercados muestran en sus propias webs, transformándolos a un formato común que permita compararlos entre sí.

Por robustez, el sistema no depende de un único origen de datos, sino que combina dos vías complementarias: la recogida automática y programada de los precios publicados por las grandes cadenas, y la actualización directa que los propios comercios verificados realizan desde el portal de empresas. De este modo, si una fuente falla o queda desactualizada, la otra mantiene la información viva, y los datos aportados de primera mano por los comercios gozan de prioridad por su mayor fiabilidad. Las herramientas concretas que sustentan esta recogida automatizada se detallan en el capítulo de diseño e implementación.

2.3.3. OCR y asistente

El reconocimiento óptico de caracteres (OCR, del inglés *Optical Character Recognition*) es la tecnología que convierte el texto contenido en una imagen —una fotografía, un ticket o una nota manuscrita— en texto digital editable y procesable. En BarGAIN constituye la vía para que el usuario transforme un ticket o una lista escrita a mano en ítems estructurados de su lista de la compra. Para esta tarea se adopta Google Cloud Vision API (Google Cloud, 2026) por su mejor rendimiento práctico en tickets y fotos reales. Para el asistente se integra Gemini (Google, 2026) mediante backend proxy, con guardrails para limitar respuestas al dominio de la compra. La viabilidad de este tipo de asistentes se apoya en la capacidad de los grandes modelos de lenguaje para resolver tareas con pocos ejemplos (Brown y cols., 2020) y en técnicas de generación aumentada con datos externos (Lewis y cols., 2020), que en BarGAIN se concreta en inyectar el contexto real de precios y tiendas del usuario en cada consulta.

2.4— Aportación diferencial de BarGAIN

La aportación del proyecto se concreta en cuatro ejes:

- **Optimización multicriterio real** (precio + distancia + tiempo) en lugar de comparación aislada de precios.
- **Inclusión de comercio local** mediante portal business y actualización de precios/promociones.
- **Digitalización de lista** por OCR con validación de usuario.
- **Interacción conversacional** para consultas complejas de compra.

Estas capacidades, integradas en una sola plataforma, posicionan BarGAIN en un espacio de mercado poco cubierto por herramientas actuales.

Comparación con otras alternativas

3.1— Competidores considerados

Para el análisis competitivo se han considerado cinco soluciones de referencia en el mercado español de comparación de compra:

- Soysuper (Soysuper S.L., 2026)
- Findit
- OCU Market (Organizacion de Consumidores y Usuarios (OCU), 2026)
- PreciRadar
- RadarPrice

Adicionalmente, se han valorado como competencia indirecta las apps nativas de cadenas y como sustituto no tecnológico la comparación manual con folletos. La revisión de capacidades se realizó sobre las versiones públicas de cada herramienta durante la fase de análisis del proyecto (marzo de 2026), por lo que la matriz refleja el estado de las soluciones en esa fecha.

3.2— Matriz funcional resumida

El Cuadro 3.1 resume la comparativa de capacidades clave; el análisis extendido, característica a característica, se mantiene en la documentación técnica versionada del repositorio del proyecto.

Cuadro 3.1: Comparativa de capacidades clave (elaboración propia, marzo de 2026)

Capacidad	BarGAIN	Soysuper	OCU Market	Resto
Comparación multi-tienda de cesta	Completa	Completa	Parcial	Parcial
Optimización de ruta (precio, distancia y tiempo)	Completa	No	No	No
Integración de PYMEs / comercio local	Completa	No	No	No
OCR de lista / ticket	Completa	No	No	No
Asistente conversacional IA	Completa	No	No	No
Histórico avanzado de precios	Parcial	Limitado	Limitado	Variable

3.3– Brechas estratégicas detectadas

El análisis identifica cuatro brechas de mercado que justifican la propuesta:

1. **Gap precio-ruta:** los comparadores muestran precios, pero no optimizan desplazamiento y tiempo.
2. **Comercio local invisible:** no existe cobertura estable de PYMEs en herramientas generalistas.
3. **Fricción de entrada:** la lista de compra sigue siendo manual en la mayoría de soluciones.
4. **Interacción limitada:** predominan filtros y búsquedas rígidas frente a consultas en lenguaje natural.

3.4– Posicionamiento de BarGAIN

BarGAIN se posiciona como un *optimizador inteligente de compra* y no solo como comparador. Su propuesta de valor combina ahorro económico, eficiencia logística y experiencia de usuario asistida por IA.

3.5– Riesgos competitivos

Los riesgos principales identificados son:

- reacción de competidores consolidados con mayor base de usuarios,
- dependencia de fuentes de datos de terceros para la recolección masiva de datos,
- necesidad de mantener alta calidad de datos para sostener confianza.

Como mitigación, se adopta una estrategia de fuentes híbridas de datos, modularidad arquitectónica y pipeline de

Análisis temporal y costes de desarrollo

4.1— Planificación temporal

La planificación de BarGAIN se definió con una dedicación planificada de 300 horas, distribuidas en fases funcionales y técnicas, que se amplió hasta 325 horas reales al incorporar el cierre documental y la preparación de la defensa. El proyecto se ejecutó de forma iterativa, manteniendo trazabilidad en `TASKS.md` y en el árbol `.planning/`.

Cuadro 4.1: Resumen por fases del proyecto

Fase	Semanas	Horas	Estado
F1 · Análisis y diseño	S1–S3	45	Completada
F2 · Infraestructura base	S3–S5	30	Completada
F3 · Desarrollo core backend	S5–S12	95	Completada
F4 · Desarrollo frontend	S8–S16	70	Completada
F5 · IA, optimizador y scraping	S10–S17	35	Completada
F6 · Portal business y app móvil	S15–S18	25	Completada
F7 · Pruebas, deploy y cierre	S18–S20	25	Completada
Total		325	Cerrado

4.2— Metodología de trabajo y trazabilidad del proceso

La distribución por fases descrita en la sección anterior no es un calendario estático, sino el reflejo de la metodología de trabajo que gobernó el proyecto día a día. Cada una de las fases del Cuadro 4.1 se abordó con el mismo ciclo incremental —definición de alcance y criterios de aceptación, implementación por módulos, validación automatizada (análisis estático y pruebas), verificación funcional y documental, y cierre de fase con artefactos de evidencia—, lo que permitió absorber cambios sin comprometer la estabilidad del sistema. Esta sección explica cómo se ejecutó ese ciclo y por qué las horas anteriores se tradujeron en un proceso trazable.

La inteligencia artificial desempeña en este Trabajo Fin de Grado un doble papel que conviene delimitar desde el principio. Por un lado, es una **herramienta para desarrollar el TFG**: la programación del código se llevó a cabo mediante agentes de IA, mientras que el papel del autor fue el de **gestor y verificador** del proceso —definir el alcance y los criterios de aceptación de cada fase, dirigir a los agentes, y revisar y validar cada entrega antes de aceptarla—. Por otro lado, ese mismo modo de trabajo es también, en sí mismo,

un **objeto de estudio del propio TFG**: el proyecto no solo construye un producto, sino que documenta y evalúa de forma reproducible una metodología de ingeniería del software asistida por agentes. Así, el flujo descrito a continuación debe leerse en esa doble clave —como el instrumento que produjo BarGAIN y como una aportación metodológica observable por el tribunal—.

Todo el ciclo de vida se gobernó mediante el método *GSD* (*Get Stuff Done*) (Christopherson, 2026), un sistema abierto de desarrollo dirigido por especificación para agentes de código, y se versionó íntegramente en el propio repositorio. El resultado es que no solo el producto, sino también el *proceso* de ingeniería que lo produjo, es inspeccionable, auditable y reproducible por terceros.

4.2.1. El framework GSD: flujo de trabajo y comandos

El problema que GSD resuelve es la *degradación del contexto* (*context rot*): a medida que una conversación con un agente se alarga, su ventana de contexto se satura de historial —razonamientos exploratorios, lecturas de ficheros, diferencias de código— y la calidad de sus respuestas decae, llegando a contradecir decisiones anteriores. La respuesta de GSD es el **aislamiento de contexto**: el trabajo se descompone en tareas atómicas y cada una se ejecuta en una instancia nueva del agente, con la ventana limpia, apoyándose en ficheros de memoria persistentes que transportan la información entre sesiones sin recargar ninguna de ellas.

Sobre esa idea, GSD organiza el desarrollo en *hitos* (*milestones*) que se descomponen en fases, y cada fase recorre un ciclo ordenado —**discutir, planificar, ejecutar y verificar**— antes de darse por cerrada, siguiendo el patrón *idea* → *hoja de ruta* → *plan de fase* → *ejecución atómica*. Cada paso produce un artefacto escrito que actúa como fuente de verdad del siguiente, de modo que la especificación, y no el historial de la conversación, es lo que sobrevive a los límites de sesión. Todo el flujo se opera mediante órdenes (*comandos*) de un único espacio de nombres `gsd`, cada una asociada a un momento concreto del proceso. A continuación se describe el flujo completo, agrupado por etapas.

Arranque del proyecto.

/gsd-new-project Inicia el proyecto: a partir de unas preguntas —o de un documento de requisitos existente— genera la definición del producto, los requisitos y la hoja de ruta en el árbol `.planning/`, y deja aprobado el roadmap inicial.

/gsd-map-codebase Para proyectos ya existentes, analiza el código actual con agentes en paralelo y construye un mapa fiel de su estructura, de modo que las fases siguientes partan de lo que realmente hay y no de suposiciones.

Ciclo de cada fase. Cada fase de la hoja de ruta atraviesa, en orden, los siguientes pasos:

/gsd-discuss-phase Abre una conversación previa para fijar las preferencias y decisiones de la fase —el qué y el cómo— antes de planificar nada, dejando por escrito los supuestos acordados.

/gsd-ui-phase Solo en fases con interfaz: establece el *contrato de diseño* (componentes, estados y pautas visuales) que la implementación deberá respetar.

/gsd-plan-phase Investiga y redacta el plan de la fase, descomponiéndola en tareas atómicas con sus criterios de aceptación, y lo verifica antes de tocar código. Es la fase de *planificar*: produce un documento, no código.

/gsd-execute-phase Ejecuta el plan de forma autónoma —a menudo con tareas en paralelo—: escribe el código, confirma los cambios en el control de versiones y registra en un resumen lo realmente hecho y las desviaciones. Es la fase de *ejecutar*, idealmente sobre una copia de trabajo aislada.

/gsd-verify-work Comprueba el resultado mediante una validación funcional manual (UAT) sobre lo construido. Es la fase de *verificar*.

/gsd-code-review Somete los cambios a una revisión de código —seguridad, rendimiento y corrección— con un contexto independiente del que los escribió.

/gsd-validate-phase Validación retroactiva: contrasta de forma sistemática que cada requisito de la fase quedó efectivamente cubierto, cerrando los huecos que la verificación manual pudiera haber pasado por alto.

/gsd-ship Empaqueta el trabajo ya verificado en una solicitud de incorporación (*pull request*).

/gsd-ui-review Solo en fases con interfaz: realiza una auditoría visual del resultado frente al contrato de diseño fijado en **/gsd-ui-phase**.

Cierre de hito. Cuando todas las fases de un hito están completas:

/gsd-audit-milestone Audita el hito comprobando que todos sus requisitos se han entregado realmente; constituye la *verificación a nivel de hito*, complementaria a la verificación por fase.

/gsd-complete-milestone Da el hito por terminado: archiva sus artefactos y etiqueta la versión entregada.

/gsd-new-milestone Abre el siguiente hito y reanuda el ciclo de fases.

Apoyo y continuidad. De forma transversal, el flujo dispone de órdenes auxiliares que sostienen el resto:

/gsd-progress Muestra el estado global del proyecto y, con la opción adecuada, detecta y lanza automáticamente el siguiente paso pendiente.

/gsd-pause-work y /gsd-resume-work Suspenden y retoman el trabajo entre sesiones generando un resumen de contexto, de modo que una interrupción no haga perder el hilo.

/gsd-spike, /gsd-sketch y /gsd-explore Habilitan trabajo exploratorio —prototipos desechables, bocetos de solución o exploración del problema— antes de comprometer una fase, mientras que **/gsd-quick** atiende correcciones pequeñas sin recorrer el ciclo completo.

La separación en contextos distintos es deliberada: quien planifica, quien implementa y quien verifica son instancias independientes del agente, lo que evita que la verificación herede la confianza del código recién escrito y reproduce, con agentes, la sana separación entre desarrollo y revisión propia de los equipos humanos. En BarGAIN, este flujo se aplicó fase a fase (F1–F7), y sus artefactos —planes, resúmenes y registros— son precisamente los que pueblan el árbol **.planning/** que se describe a continuación.

4.2.2. Planificación viva: el árbol `.planning/`

El método GSD sustituye los documentos de planificación estáticos por un árbol de ficheros Markdown versionados que evolucionan con el proyecto. En `.planning/` conviven la definición del producto (`PROJECT.md`), los requisitos operativos (`REQUIREMENTS.md`), la hoja de ruta por fases (`ROADMAP.md`) y una instantánea del estado actual (`STATE.md`, `STATUS.md`) que cualquier agente —o persona— lee antes de actuar. Cada fase se descompone en `.planning/phases/` en pares `PLAN/SUMMARY`: el plan fija el alcance y los criterios de aceptación *antes* de implementar, y el resumen registra lo realmente ejecutado, las desviaciones y las evidencias de verificación al cerrar. El subdirectorio `.planning/debug/` funciona como base de conocimiento de incidencias: cada problema no trivial (un 404 de CORS en GitHub Pages, un archivo iOS que no se generaba, la conectividad móvil contra Render) queda documentado con su diagnóstico y resolución, y se mueve a `resolved/` al cerrarse.

4.2.3. Trazabilidad de tareas y evidencias

La fuente de verdad del seguimiento es `TASKS.md`, un tablero versionado que asigna a cada tarea un identificador estable (`F3-01`, `F7-08...`), su estado, horas estimadas y reales, y el entregable que produce. La convención de *commits* exige referenciar ese identificador (`feat(optimizer): ... (F5-06)`), de modo que cualquier línea de código es trazable hacia atrás —hasta la tarea, la fase y el requisito que la motivaron— y hacia delante —hasta el test y la evidencia que la verifican—. El tablero se sincroniza además con GitHub Issues y con un *backlog* en Notion, manteniendo una única numeración. Esta disciplina, habitual en equipos industriales pero infrecuente en un TFG individual, fue la que permitió absorber 325 horas de trabajo en 20 semanas sin pérdida de control del alcance.

4.2.4. Agentes de IA con contrato de contexto explícito

Los agentes (Claude, GitHub Copilot y Codex) no operan sobre instrucciones improvisadas: el repositorio define un *contrato de contexto* común. El fichero `CLAUDE.md` reúne la arquitectura, las convenciones de código, las reglas de negocio y el protocolo obligatorio que todo agente debe seguir antes, durante y después de cada tarea; los directorios `.github/agents/`, `.github/instructions/` y `.agent/workflows/` versionan agentes especializados (orquestador, ejecutor y revisor de fase), instrucciones por tecnología y flujos de trabajo reutilizables; y `docs/ai-prompts/` contiene las plantillas de inicio, cierre, revisión y sincronización de tareas —el equivalente a los *runbooks* de operaciones—, duplicadas como *prompts* de Copilot para mantener paridad entre herramientas. Las *skills* empleadas durante el desarrollo se versionan igualmente junto al código, de manera que el entorno de trabajo de los agentes forma parte del entregable.

4.2.5. Registro de errores y reglas derivadas

El componente más distintivo del flujo es `docs/ai-mistakes-log.md`: cada error cometido por un agente se registra con un formato fijo —contexto, error, causa raíz, solución aplicada y prevención— y, cuando procede, se destila en una *regla derivada* numerada que todos los agentes releen obligatoriamente al iniciar una tarea. Así, por ejemplo, la regla que prohíbe entregar diagramas de interfaz en ASCII o la que obliga a validar la existencia de rutas de sistema antes de aplicarlas en *settings* compartidos nacieron de errores reales y han evitado su repetición. El registro convierte los fallos de los agentes en conocimiento acumulativo del proyecto: el proceso *aprende* de manera verificable, y el historial de ese aprendizaje queda en el control de versiones.

4.2.6. Valoración del enfoque

Conviene delimitar el reparto de responsabilidades: los agentes ejecutan, pero la definición de fases, los criterios de aceptación, la revisión de cada entrega y la decisión final son del autor. La combinación resultó eficaz por dos razones. Primera, la productividad: tareas mecánicas (generación de tests, capturas, scripts de despliegue) se delegaron sin pérdida de calidad gracias al contrato de contexto. Segunda, la fiabilidad del proceso: la planificación viva y el registro de errores con reglas derivadas impusieron al trabajo con IA la misma disciplina de evidencias que se exigiría a un equipo humano. El enfoque es, además, reproducible: cualquier evaluador puede clonar el repositorio y reconstruir no solo el sistema, sino la historia completa de decisiones que lo produjo.

4.3— Estimación de costes

La estimación económica se realizó mediante coste-hora por rol junior, tomando como referencia el informe de perfiles profesionales TIC de la Junta de Andalucía (Junta de Andalucía, 2018) (columna de media acotada para perfiles con menos de seis años de experiencia). Conviene señalar que se trata de la referencia pública más completa disponible para tarifas por rol en Andalucía pero data de 2018, por lo que las tarifas aplicadas deben entenderse como una cota conservadora: los salarios TIC de mercado en 2026 son superiores, de modo que el coste real de un desarrollo equivalente sería previsiblemente mayor que el estimado.

Cuadro 4.2: Coste estimado del desarrollo

Bloque	Horas	Coste (EUR)
Análisis y diseño	45	1.490,40
Infraestructura	30	861,60
Backend + frontend	165	4.738,80
IA, optimizador y scraping	35	1.005,20
Portal business y app móvil	25	718,00
Pruebas y despliegue	25	594,25
Total	325	9.408,25

Junto al coste de desarrollo, conviene distinguir el **coste operativo**, es decir, el gasto recurrente de mantener el sistema en funcionamiento. Durante el TFG este coste se mantuvo prácticamente nulo gracias al uso deliberado de planes gratuitos: el backend, la base de datos PostgreSQL/PostGIS y la cola Redis se alojan en el plan gratuito de Render; el portal de comercios se publica como sitio estático en GitHub Pages; la integración y entrega continuas y la generación de la versión para iPhone se ejecutan dentro de los minutos gratuitos de GitHub Actions; la monitorización de errores emplea el nivel gratuito de Sentry; y los servicios de IA y de rutas (Google Cloud Vision para el OCR, Gemini para el asistente y OpenRouteService para las distancias) operan dentro de sus respectivas cuotas gratuitas, holgadas para los volúmenes de un entorno de demostración. La contrapartida de esta gratuidad es la limitación de recursos: el servicio gratuito de Render hiberna tras un periodo sin tráfico, lo que introduce un arranque en frío de entre 30 y 60 segundos en la primera petición, asumible en *staging* pero no en producción.

Una explotación real exigiría migrar a planes de pago, cuyo orden de magnitud se estima en el Cuadro 4.3 para un escenario inicial de baja carga. Las cifras son orientativas y se basan en las tarifas públicas de cada proveedor a fecha de redacción; los servicios facturados por uso (OCR, modelo de lenguaje y cálculo de rutas) crecen con el número de usuarios activos, por lo que su coste real dependería directamente de la adopción.

Cuadro 4.3: Estimación del coste operativo mensual (escenario inicial de producción)

Servicio	Función	Coste mensual (EUR)
Render (servicio web + PostgreSQL/Redis)	Backend, base de datos y cola	20–50
Sentry (plan de equipo)	Monitorización de errores	0–25
GitHub Actions / Pages	CI/CD y portal estático	0–10
Google Cloud Vision (OCR)	Reconocimiento de tickets	Por uso (~1,30/1.000 imágenes)
Gemini API (asistente)	Asistente conversacional	Por uso (según consultas)
OpenRouteService	Distancias y tiempos de ruta	0 (cuota) – por uso
Base fija estimada		~20–85

En conjunto, el sistema puede sostenerse con un coste fijo reducido en su arranque y escalar el gasto variable de forma proporcional al uso, lo que resulta coherente con el carácter de prueba de concepto del TFG y con una eventual puesta en marcha gradual.

4.4– Riesgos y mitigaciones

- Los riesgos operativos principales fueron:
- **Cambios en fuentes de scraping:** mitigado con recolectores modulares e independientes por cadena y con la actualización directa de los comercios verificados.
 - **Dependencia de APIs externas:** mitigado con caché y degradación controlada.
 - **Carga de trabajo en fases finales:** mitigado con cierre incremental de entregables y automatización de validación.

4.5– Resultado de planificación

La planificación permitió completar el alcance funcional priorizado del TFG, incluyendo arquitectura, implementación, pruebas y despliegue de staging, manteniendo coherencia entre roadmap, requisitos y evidencias de verificación.

Análisis de requisitos

5.1— Actores del sistema

El sistema define cuatro actores principales:

- **Consumidor:** crea listas, compara precios, optimiza rutas, usa OCR y consulta al asistente.
- **Comercio/PYME:** gestiona precios y promociones desde el portal business.
- **Administrador:** verifica perfiles, modera datos y supervisa la operación.
- **Sistema automatizado:** tareas de scraping y mantenimiento en segundo plano.

5.2— Requisitos de información

El modelo de información se articula en doce requisitos de información (RI), identificados como RI-001 a RI-012, que definen los datos que el sistema debe almacenar y gestionar:

RI-001 Usuarios. Identidad (nombre de usuario único, email único, contraseña *hasheada*), rol (consumidor, comercio, administrador), ubicación por defecto (punto geográfico con identificador de sistema de referencia espacial, SRID, 4326), radio de búsqueda, máximo de paradas, pesos de optimización y preferencias de notificación.

RI-002 Productos. Catálogo normalizado: nombre, nombre normalizado, número de artículo europeo (EAN-13) opcional, categoría, marca, unidad, imagen y estado.

RI-003 Categorías. Jerarquía navegable con categoría padre y orden de visualización.

RI-004 Marcas. Nombre, logotipo y vínculo opcional con cadena (marca blanca).

RI-005 Cadenas. Nombre, logotipo, web, disponibilidad de API oficial y color corporativo.

RI-006 Tiendas. Establecimiento físico geolocalizado (punto con índice espacial), cadena opcional, horario en formato *JavaScript Object Notation* (JSON), indicador de comercio local y nivel de suscripción.

RI-007 Precios. Precio actual e histórico por par producto–tienda: precio, precio unitario, oferta y caducidad, fuente (API oficial, recogida automática o comercio verificado), verificación y bandera de caducidad.

RI-008 Listas. Listas por usuario con ítems de texto libre normalizado, cantidad y estado de comprado; la vinculación a productos se resuelve de forma diferida.

RI-009 Optimizaciones. Resultado persistente: ubicación, radio, pesos, totales, ruta (paradas ordenadas e ítems por parada) y preferencias semánticas confirmadas.

RI-010 Perfiles de negocio. Datos fiscales (NIF/CIF único), estado de verificación, umbral de alerta competitiva y promociones asociadas.

RI-011 Sesiones OCR. Imagen procesada, texto extraído, candidatos reconocidos con confianza y estado de la sesión.

RI-012 Conversaciones. Historial de mensajes usuario/asistente con fechas y estado.

5.3– Requisitos funcionales

Se definieron 34 requisitos funcionales (RF), identificados como RF-001 a RF-034 y agrupados por dominios, tal y como resume el Cuadro 5.1. El enunciado completo de cada requisito se detalla a continuación en el Cuadro 5.2. La especificación extendida, con criterios de aceptación e historias de usuario (HU-001 a HU-030), se mantiene versionada en el repositorio (`docs/memoria/07-requisitos.md`).

Cuadro 5.1: Agrupación de requisitos funcionales

Dominio	IDs	Capacidad principal
Autenticación y perfil	RF-001..RF-005	Registro, login JWT, preferencias
Catálogo de productos	RF-006..RF-010	Búsqueda, detalle, propuesta de productos
Tiendas y mapa	RF-011..RF-014	Proximidad, detalle, favoritos
Precios	RF-015..RF-018	Comparación, histórico, alertas
Listas de compra	RF-019..RF-022	CRUD, compartir, plantillas
Optimizador	RF-023..RF-026	Ruta multicriterio y recálculo
OCR	RF-027..RF-028	Captura y matching fuzzy
Asistente LLM	RF-029..RF-030	Consultas naturales y sugerencias
Portal business	RF-031..RF-033	Onboarding, precios, promociones
Notificaciones	RF-034	Push / email por eventos

Cuadro 5.2: Catálogo de requisitos funcionales

ID	Nombre	Síntesis
RF-001	Registro de usuario	Alta con nombre de usuario, email único y contraseña validada.
RF-002	Inicio de sesión	Autenticación con nombre de usuario y contraseña; par de tokens JWT con rotación; tiempos de vida configurables por entorno (60 min/7 días en la configuración de referencia).
RF-003	Gestión de perfil	Consulta y modificación de datos personales y preferencias.
RF-004	Preferencias de optimización	Pesos relativos de precio/distancia/tiempo, radio (1–25 km) y paradas (1–4).

ID	Nombre	Síntesis
RF-005	Recuperación de contraseña	Restablecimiento por enlace de correo con caducidad.
RF-006	Consulta de productos	Búsqueda con filtros, paginación y ordenación.
RF-007	Detalle de producto	Ficha completa con rango de precios en tiendas cercanas.
RF-008	Autocompletado	Sugerencias en tiempo real con coincidencia aproximada sobre nombre normalizado.
RF-009	Categorías	Jerarquía de categorías navegable.
RF-010	Propuesta de producto	Alta de un nuevo producto propuesta por el usuario, sujeta a aprobación del administrador.
RF-011	Tiendas por proximidad	Listado por radio configurable ordenado por distancia.
RF-012	Detalle de tienda	Información completa y precios de la lista activa del usuario.
RF-013	Mapa	Mapa interactivo con marcadores diferenciados por cadena/comercio local.
RF-014	Tiendas favoritas	Marcado de favoritas con acceso rápido y notificaciones.
RF-015	Comparación por producto	Precios en todas las tiendas del radio con fuente y antigüedad.
RF-016	Comparación de cesta	Coste total de la lista en cada tienda individual.
RF-017	Histórico de precios	Evolución temporal del precio con gráfico.
RF-018	Alerta de precio	Notificación al alcanzar un precio objetivo.
RF-019	Gestión de listas	Altas, consultas, modificaciones y bajas de listas con límite de 20 activas.
RF-020	Gestión de ítems	Texto libre normalizado, cantidades y marcado de comprado.
RF-021	Compartir lista	Edición colaborativa con otros usuarios registrados.
RF-022	Plantillas	Listas reutilizables instanciables.
RF-023	Optimización de ruta	Asignación ítem→(producto, tienda) con resolución semántico-difusa y ruta recomendada ordenada según los pesos del usuario; ambigüedades resolubles con recálculo.
RF-024	Ruta en mapa	Recorrido, paradas y productos por parada sobre mapa interactivo.
RF-025	Desglose de ahorro	Precio total, distancia, tiempo y desglose por parada.
RF-026	Recálculo bajo demanda	Nueva optimización en menos de 5 s al cambiar parámetros.
RF-027	Captura OCR	Foto de lista o ticket procesada con reconocimiento óptico.
RF-028	Emparejamiento OCR	Candidatos del catálogo con nivel de confianza y corrección manual.
RF-029	Consulta en lenguaje natural	Chat con respuestas contextualizadas con datos reales.
RF-030	Sugerencias de ahorro	Estrategias de ahorro limitadas al dominio de compra.
RF-031	Registro de negocio	Alta PYME con datos fiscales sujeta a verificación.
RF-032	Precios del comercio	Actualización manual con fuente verificada sin caducidad.
RF-033	Promociones	Creación y gestión de promociones con vigencia.
RF-034	Notificaciones	Push y email por eventos según preferencias.

5.4— Criterios de aceptación del producto

Los criterios de aceptación se validan a nivel funcional y de experiencia:

- flujo de alta y acceso operativo en menos de 3 minutos,
- comparación de cesta completa en entorno de tiendas cercanas,
- optimización con ruta recomendada, desglose por parada y resolución de ambigüedades semánticas con recálculo,
- OCR con confirmación/edición previa a la inserción en lista,
- portal PYME con gestión de precios y promociones,
- notificaciones de eventos críticos (alertas, promociones, cambios compartidos).

5.5— Requisitos no funcionales

Se definieron ocho requisitos no funcionales (RNF), identificados como RNF-001 a RNF-008, que fijan las cualidades que el sistema debe satisfacer más allá de sus funciones concretas:

RNF-001 Rendimiento. La API debe responder con un percentil 95 inferior a 500 ms en las operaciones de crear, leer, actualizar y borrar (CRUD) e inferior a 5 segundos en la optimización de rutas.

RNF-002 Disponibilidad. Disponibilidad mínima del 99 % mensual en el entorno de *staging*, excluyendo ventanas de mantenimiento. El alcance real de su validación se detalla en el Capítulo 8.

RNF-003 Seguridad. HTTPS obligatorio, testigos de autenticación firmados (JWT) con rotación, limitación de peticiones (100/h anónimos y 1.000/h autenticados), *hashing* seguro de contraseñas y cifrado de los datos sensibles. Se trata de las *medidas técnicas* que protegen el sistema frente a accesos y usos indebidos.

RNF-004 Usabilidad. Las acciones principales deben completarse en un máximo de 3 interacciones, con diseño adaptable y conforme a las pautas de accesibilidad para el contenido web (WCAG) 2.1 nivel AA (World Wide Web Consortium (W3C), 2018).

RNF-005 Escalabilidad. La arquitectura debe poder crecer hasta unos 10.000 usuarios concurrentes apoyándose en procesos de tareas en segundo plano independientes y en una API sin estado.

RNF-006 Mantenibilidad. El sistema debe ser sencillo de evolucionar y corregir a lo largo del tiempo. Para garantizarlo se exige una cobertura de pruebas automatizadas de al menos el 80 %, verificada de forma *bloqueante* en la integración continua (un cambio que la reduzca por debajo de ese umbral no se integra), el cumplimiento de convenciones de estilo homogéneas comprobadas automáticamente (PEP 8 en el backend y ESLint/Prettier en el frontend) y una organización modular del código, con la lógica de negocio separada en una capa de servicios. El objetivo es que cualquier desarrollador pueda incorporarse al proyecto y modificarlo con bajo riesgo de regresión.

RNF-007 Compatibilidad. El sistema debe funcionar en iOS 15 o superior, Android 10 o superior y las dos últimas versiones de los navegadores principales.

RNF-008 Protección de datos. Cumplimiento del Reglamento General de Protección de Datos (RGPD) (Parlamento Europeo y Consejo de la Unión Europea, 2016): consentimiento explícito para el uso de la ubicación, derecho al olvido y minimización de los datos tratados. Conviene subrayar que este requisito **no debe confundirse con RNF-003**: aunque ambos conciernen al usuario, RNF-003 establece las medidas *técnicas* de seguridad del sistema, mientras que RNF-008 atiende a una obligación *jurídica* sobre la licitud del tratamiento de los datos personales. Son requisitos complementarios pero de naturaleza distinta —técnica frente a legal— y se verifican por separado.

5.6— Reglas de negocio

Las reglas de negocio (RN), numeradas RN-001 a RN-010, gobiernan los límites y las políticas operativas que el sistema debe hacer cumplir:

RN-001 Radio de búsqueda entre 1 y 25 km (10 km por defecto).

RN-002 Máximo 4 paradas por ruta (3 por defecto).

RN-003 Caducidad de precios: 48 h para los obtenidos por recogida automática; sin caducidad para los introducidos por comercio verificado.

RN-004 Prioridad de fuentes: API oficial > comercio verificado > recogida automática.

RN-005 Rechazo de precios no positivos y marcado de precios anómalos (>200 % de la media) como sospechosos.

RN-006 Máximo 20 listas activas por usuario.

RN-007 El asistente solo responde consultas del dominio de compra.

RN-008 Verificación administrativa previa de los perfiles de comercio.

RN-009 Respeto a las directivas `robots.txt` (Koster, Illyes, Zeller, y Sassman, 2022) y exclusión de las cadenas que prohíban el acceso automatizado.

RN-010 Frecuencia máxima de recogida automatizada de una vez cada 24 h por cadena, con un retardo mínimo de 2 segundos entre peticiones.

5.7— Diagramas de casos de uso

Los diagramas de casos de uso recogen las interacciones principales de cada actor con el sistema.

5.8— Trazabilidad

La trazabilidad entre requisitos, implementación y pruebas se documenta en tres niveles:

- el Cuadro 5.3 de esta sección, que vincula cada dominio funcional con el módulo que lo implementa y las suites de prueba que lo verifican,
- `TASKS.md` en el repositorio (estado por fase/tarea, con identificadores referenciados desde los commits),
- `tests/unit`, `tests/integration` y `frontend/web/e2e` (verificación ejecutable).

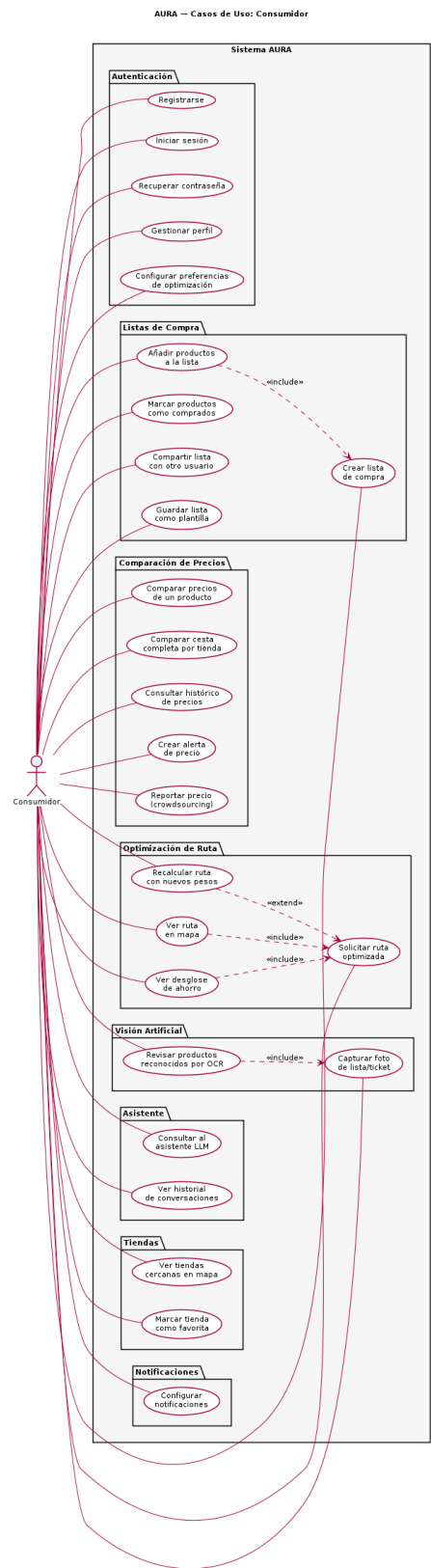


Figura 5.1: Casos de uso del Consumidor

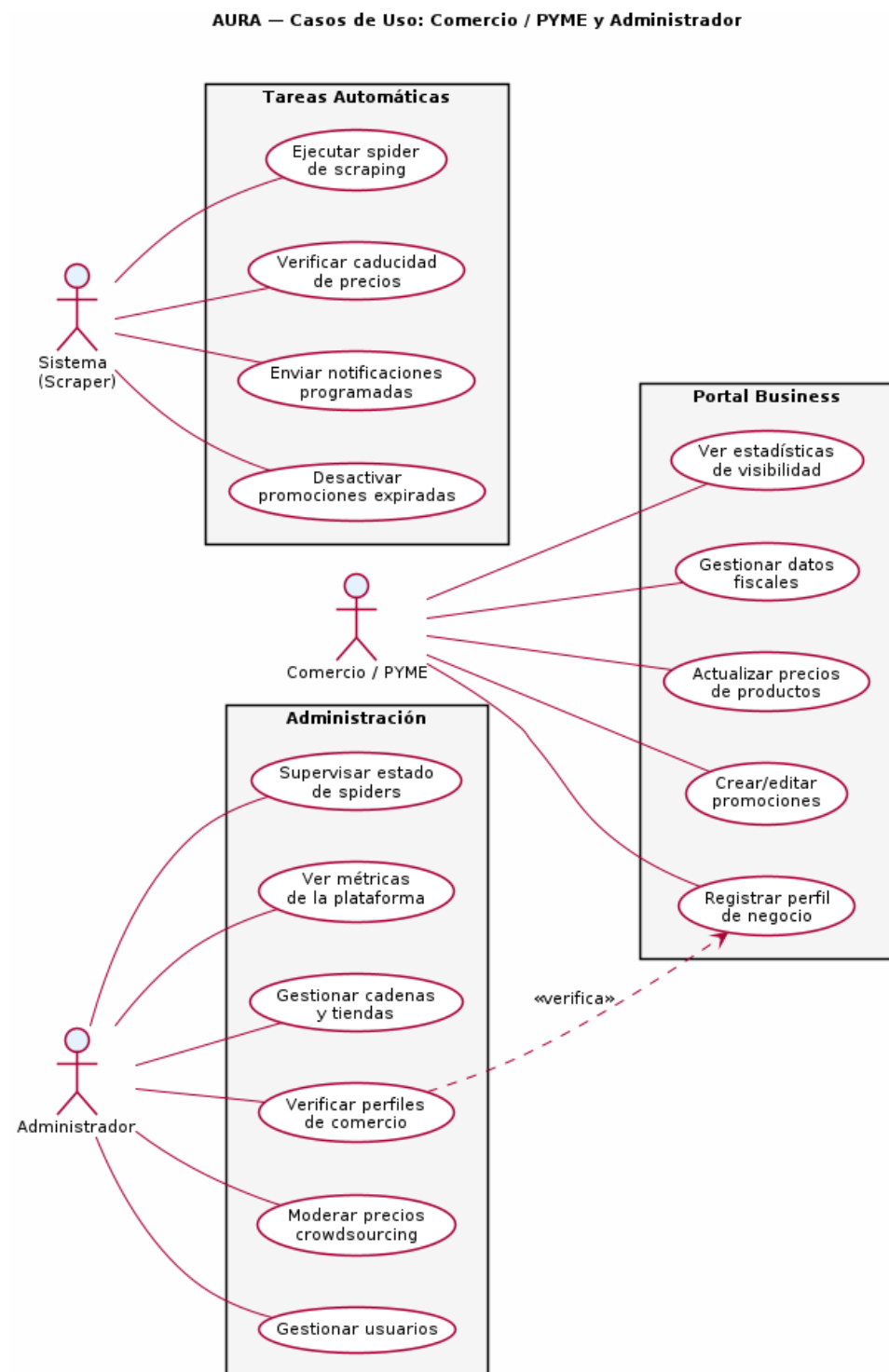


Figura 5.2: Casos de uso del Comercio y Administrador

Cuadro 5.3: Trazabilidad de dominios funcionales a módulos y suites de prueba

Dominio (RF)	Módulo	Suites de prueba
Autenticación (001–005)	apps/users	test_users, test_auth_endpoints, auth.spec
Catálogo (006–010)	apps/products	test_products, test_product_endpoints
Tiendas (011–014)	apps/stores	test_stores, test_store_endpoints
Precios (015–018)	apps/prices	test_prices, test_price_endpoints
Listas (019–022)	apps/shopping_lists	test_shopping_lists, test_list_endpoints
Optimizador (023–026)	apps/optimizer	test_optimizer*, test_distance_ors, optimizer.spec
OCR (027–028)	apps/ocr	test_ocr, test_ocr_api, ocr.spec
Asistente (029–030)	apps/assistant	test_assistant, test_assistant_api
Business (031–033)	apps/business	test_business_*, business.spec
Notificaciones (034)	apps/notifications	test_notification_*

Diseño e Implementación

6.1— Arquitectura del Sistema

6.1.1. Visión general

BarGAIN sigue una arquitectura de **tres capas** clásica (presentación, lógica de negocio y datos), desplegada mediante contenedores Docker y organizada internamente como una **arquitectura en capas con capa de servicios explícita** por aplicación Django (Django Software Foundation, 2026; Encode OSS Ltd., 2026): las vistas se limitan a la gestión HTTP y la lógica de dominio reside en módulos de servicio independientes. Esta separación desacopla la lógica de negocio de los detalles de infraestructura — bases de datos, APIs externas, brokers de mensajería — y facilita el testing independiente de cada componente.

Antes de describir la arquitectura conviene fijar la terminología que se emplea en todo el capítulo, pues BarGAIN da servicio a dos perfiles bien diferenciados. Por un lado está el **cliente** o **usuario** —ambos términos se usan indistintamente para referirse al consumidor final—: la persona que hace la compra, crea sus listas, compara precios y obtiene la ruta optimizada. Por otro lado está la **PYME** o comercio: el pequeño o mediano negocio que se da de alta para gestionar sus propios precios y promociones. A cada perfil le corresponde una superficie de acceso distinta:

- La **aplicación móvil para clientes**, construida con React Native (Expo) y orientada al consumidor final, es donde ocurre toda la experiencia de compra (listas, mapa, comparativa, optimización de ruta, OCR y asistente). La misma base de código se ejecuta también en navegador de escritorio para comodidad del usuario, sin cambiar su naturaleza de aplicación de consumo.
- El **portal web para PYMEs** (Portal Business), una aplicación web independiente en React, es donde el comercio gestiona su negocio. Es una herramienta de gestión, no de compra, y por tanto distinta en público, propósito e interfaz de la aplicación de cliente.

A lo largo del capítulo, cuando se hable de “la aplicación” sin más matiz se entenderá la aplicación de cliente; el portal de comercios se nombrará siempre de forma explícita.

La arquitectura global se divide en cinco bloques principales:

1. **Aplicación cliente:** React Native (Expo) para iOS, con una aplicación web React para el Portal Business de PYMEs.

2. **API Backend:** Django REST Framework —marco que sigue el estilo arquitectónico de transferencia de estado representacional (REST)— como núcleo de la lógica de negocio, expuesto bajo HTTPS mediante Gunicorn + Nginx.
3. **Capa de datos:** PostgreSQL 16 con extensión PostGIS 3.4 para consultas geoespaciales.
4. **Capa de tareas asíncronas:** Celery con Redis como broker, para scraping periódico, envío de notificaciones, procesamiento OCR y cálculos del optimizador.
5. **Integraciones y servicios auxiliares:** Google Gemini API (asistente basado en un modelo de lenguaje de gran tamaño, LLM), OpenRouteService (cálculo de distancias reales), Google Cloud Vision API (OCR) y Sentry (monitorización de errores).

6.1.2. Patrón de organización interna del backend

Cada área funcional del backend se organiza como un módulo independiente que sigue siempre la misma estructura interna: la definición de las entidades de datos, la transformación de la información que entra y sale, la capa que atiende las peticiones web, las rutas que la exponen, la lógica de negocio propiamente dicha, las tareas que se ejecutan en segundo plano, las reglas de autorización y su correspondiente batería de pruebas. Esta uniformidad facilita orientarse en cualquier módulo conociendo uno solo de ellos.

La separación entre la capa que atiende las peticiones web y la que contiene la lógica de negocio es deliberada: la primera se limita a los aspectos propios de la comunicación —autenticación, validación de datos y códigos de respuesta—, mientras que las reglas del negocio residen exclusivamente en la segunda. Así, esa lógica puede invocarse igualmente desde las tareas en segundo plano o desde las pruebas, sin necesidad de simular una petición real. En los módulos más complejos, como el optimizador, esa lógica se reparte además en varias piezas especializadas (correspondencia de productos, interpretación semántica, cálculo de distancias y resolución de la ruta).

La Figura 6.1 resume la distribución por capas de la solución.

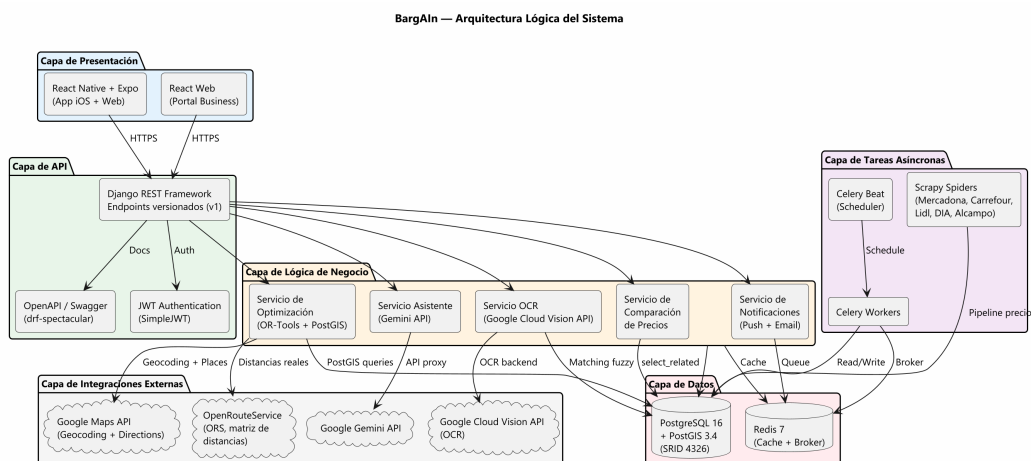


Figura 6.1: Arquitectura por capas de BarGAIN

6.1.3. Comunicación entre componentes

- **De la aplicación al backend:** a través de la API REST con sesión autenticada; el cliente renueva de forma transparente el token de acceso cuando este caduca, sin que el usuario perciba la operación.

- **Del backend a las tareas en segundo plano:** los trabajos pesados se encolan para que se procesen de forma asíncrona sin bloquear la respuesta al usuario.
- **Del backend a la base de datos:** mediante una capa de acceso que incorpora las consultas geoespaciales necesarias para las búsquedas por proximidad.
- **Del backend a los servicios externos:** a través de clientes aislados en la capa de servicios de cada módulo, que encapsulan la comunicación con los servicios de mapas, rutas y el modelo de lenguaje.

6.2— Modelo de Datos

6.2.1. Módulo de usuarios

El modelo de usuario se apoya en el sistema de autenticación de Django, conservando el nombre de usuario como identificador de inicio de sesión; el correo electrónico se valida como único en el registro y se emplea para las comunicaciones y la recuperación de contraseña. Se distinguen tres roles —consumidor, comercio y administrador— y se almacena la ubicación por defecto del usuario como una coordenada geográfica.

Las preferencias de optimización se guardan en el perfil como tres porcentajes —la importancia relativa que el usuario concede al precio, a la distancia y al tiempo— con un reparto inicial equilibrado de aproximadamente un tercio cada uno. La interfaz ajusta los tres deslizadores de forma conjunta y, en cada petición de optimización, el sistema reescala internamente esos valores para que sumen la unidad, de modo que el algoritmo siempre trabaja con pesos relativos con independencia de cómo se hayan introducido.

6.2.2. Módulo de productos

La búsqueda de productos por nombre tolera erratas y variantes de redacción mediante una técnica de *coincidencia aproximada*: cada producto guarda una versión normalizada de su nombre (en minúsculas, sin tildes ni caracteres especiales) y las búsquedas se comparan fragmentando las palabras en grupos de tres letras, de modo que “leche entra’encuentra “leche entera”. La base de datos resuelve esta comparación con un índice especializado que ordena los resultados de mayor a menor parecido, descartando los que no superan un umbral mínimo de similitud.

6.2.3. Módulo de tiendas

Cada tienda almacena su posición como una coordenada geográfica indexada espacialmente, lo que permite a la base de datos resolver con eficiencia preguntas del tipo “qué tiendas hay a menos de tantos kilómetros de este punto”. La consulta filtra los establecimientos activos situados dentro del radio indicado y los ordena por cercanía a la ubicación del usuario, apoyándose en las capacidades geoespaciales de PostGIS (PostGIS Project Steering Committee, 2026).

Para los volúmenes manejados en el TFG —del orden de decenas de tiendas por radio— esta consulta es más que suficiente. Para escenarios con cientos de miles de establecimientos se ha identificado una variante de la consulta que aprovecha de forma aún más directa el índice espacial (Obe y Hsu, 2021), sin necesidad de cambios en el resto del diseño.

6.2.4. Módulo de precios

El histórico de precios se conserva de forma automática: cada precio es inmutable una vez registrado, de modo que actualizar un precio equivale a añadir un nuevo registro y

mantener los anteriores como historia. Una tarea automática que se ejecuta cada hora marca como caducados los precios obtenidos por recogida automática que superan las 48 horas de antigüedad, conforme a la política de caducidad de precios del sistema; los precios introducidos directamente por los comercios verificados no caducan, al tratarse de información de primera mano.

6.2.5. Módulo del optimizador

El resultado de cada optimización se almacena de forma estructurada y trazable en tres niveles anidados:

- **El resultado global:** recoge los pesos empleados, el modo de optimización y las métricas totales de la ruta (precio, distancia y tiempo).
- **Cada parada de la ruta:** la tienda visitada, su orden en el recorrido y la distancia, el tiempo y el subtotal correspondientes.
- **Cada producto dentro de una parada:** el producto finalmente asignado, el precio elegido, el texto original que escribió el usuario y el grado de parecido con el que se resolvió la correspondencia.

6.2.6. Índices clave

El Cuadro 6.1 recoge los índices definidos explícitamente en los modelos, además de los índices automáticos de claves primarias y foráneas. Se emplean dos tipos de índice de PostgreSQL: el árbol de búsqueda generalizado (GiST), idóneo para las consultas geográficas, y el índice invertido generalizado (GIN), empleado en las búsquedas de texto aproximadas.

Cuadro 6.1: Índices principales de la base de datos

Tabla	Columna(s)	Tipo	Justificación
store	location	GiST	Búsquedas geoespaciales por radio
product	normalized_name	GIN (trigram)	Búsqueda fuzzy por nombre
price	(product, store, is_stale)	B-Tree	Precio vigente por par producto-tienda

6.2.7. Modelo entidad-relación

La Figura 6.2 muestra el diagrama entidad-relación completo de la base de datos.

6.2.8. Diagrama de clases

La Figura 6.3 detalla las clases principales del dominio y sus relaciones.

6.3— Diseño de la API REST

6.3.1. Estructura general

La API sigue las convenciones REST y agrupa todas sus rutas bajo una misma versión. Salvo el registro, el inicio de sesión, la comprobación de estado del servicio y la recuperación de contraseña, todas las peticiones requieren que el usuario esté autenticado. Para que el cliente trate siempre las respuestas de la misma forma, todas comparten un formato común: un sobre que indica si la operación tuvo éxito y, en ese caso, transporta los datos

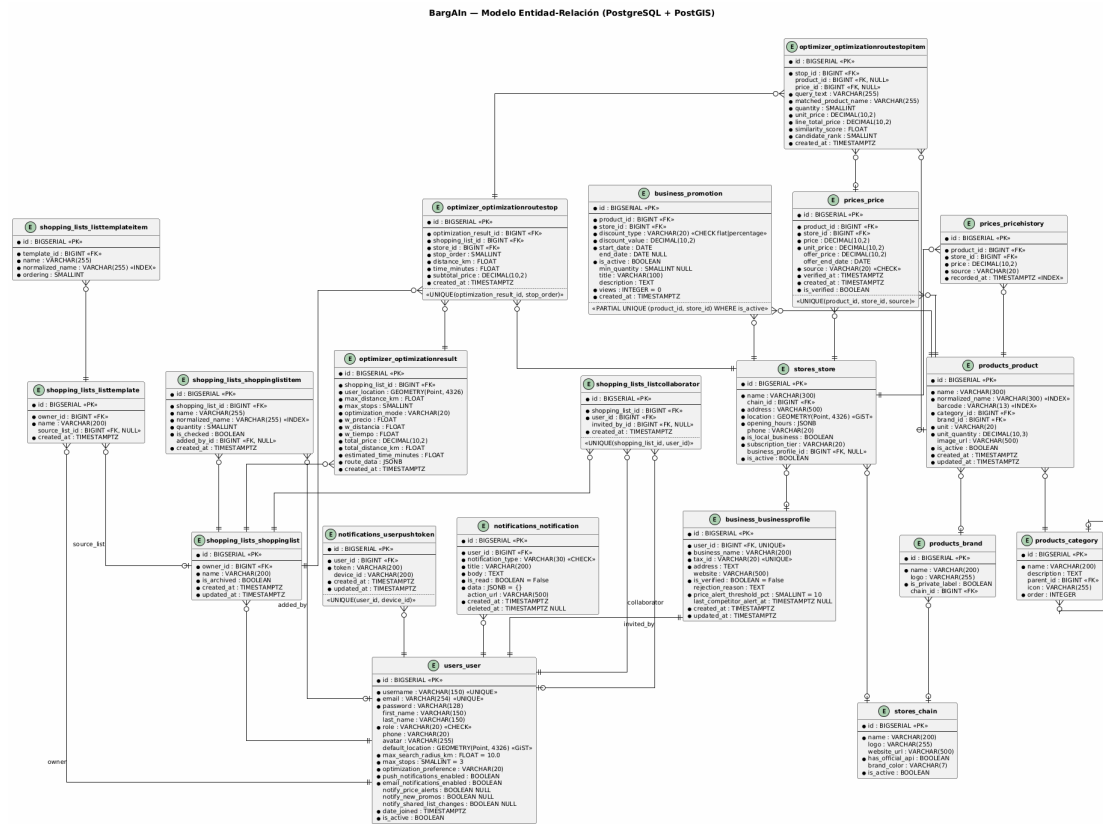


Figura 6.2: Diagrama entidad-relación de BarGAIN

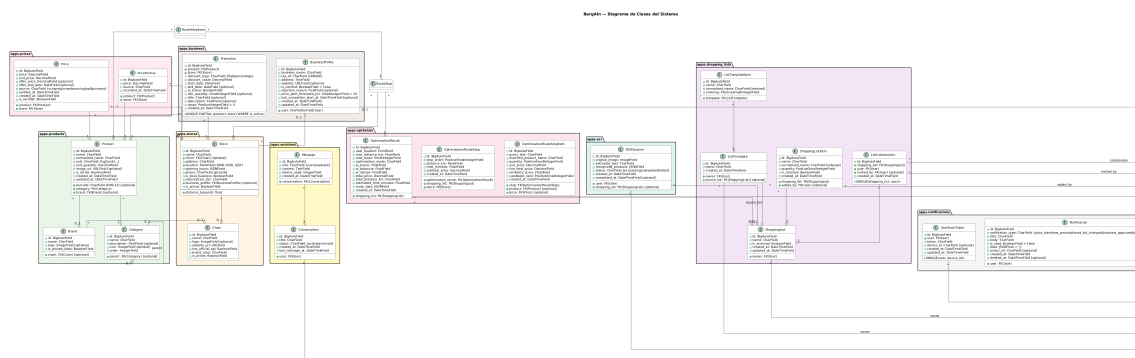


Figura 6.3: Diagrama de clases del dominio

solicitados; cuando algo falla, ese mismo sobre incorpora en su lugar un bloque de error con un código identificable, un mensaje legible y los detalles oportunos.

```
// Respuesta exitosa
{
  "success": true,
  "data": { ... }
}

// Respuesta de error
{
  "success": false,
  "error": {
    "code": "STORE_NOT_FOUND",
    "message": "No se encontraron tiendas en el radio especificado",
    "details": {}
  }
}
```

Los listados extensos se entregan paginados, incluyendo junto a los resultados el total de elementos y los enlaces a las páginas anterior y siguiente.

6.3.2. Endpoints principales

Los Cuadros 6.2 y 6.3 recogen los endpoints de los dominios de autenticación y de las capacidades diferenciales; la relación completa por módulo se incluye en el Apéndice A.

Cuadro 6.2: Endpoints de autenticación y usuarios

Método	Endpoint	Descripción
POST	/api/v1/auth/register/	Registro de nuevo usuario
POST	/api/v1/auth/token/	Login: access + refresh token
POST	/api/v1/auth/token/refresh/	Renovar access token
POST	/api/v1/auth/password-reset/	Solicitar restablecimiento
POST	/api/v1/auth/password-reset/confirm/	Confirmar restablecimiento
GET/PATCH	/api/v1/auth/profile/	Perfil y preferencias del usuario

Cuadro 6.3: Endpoints del optimizador, OCR y asistente

Método	Endpoint	Descripción
POST	/api/v1/optimize/	Calcular o recalcular la ruta óptima
GET	/api/v1/optimize/?shopping_list_id={id}	Cargar última ruta persistida
POST	/api/v1/optimize/choices/	Resolver ambigüedad semántica y recalcular
POST	/api/v1/ocr/scan/	Procesar imagen y devolver ítems
POST	/api/v1/assistant/chat/	Enviar mensaje al asistente

6.3.3. Autenticación y autorización

El control de acceso se articula en varios niveles que determinan quién puede hacer qué: la exigencia general de estar autenticado para las operaciones privadas; la restricción que limita a cada comerciante a gestionar únicamente su propio negocio; los permisos reservados a los administradores del sistema; y la regla que permite acceder a una lista

de la compra tanto a su propietario como a los colaboradores a los que este la haya compartido.

La sesión se sustenta en tokens de autenticación firmados (Jazzband, 2026): uno de acceso, de vida corta, y otro de refresco que permite renovarlo sin volver a introducir credenciales. El token de refresco se rota automáticamente y el anterior se invalida en cuanto se usa, de modo que un token robado deja de servir tras la primera renovación. Los tiempos de validez son configurables; la configuración de referencia del proyecto fija 60 minutos para el token de acceso y 7 días para el de refresco.

6.4— Implementación del Backend

6.4.1. Módulo de notificaciones

El módulo gestiona el buzón de notificaciones del usuario y el envío de avisos al dispositivo móvil, cuyo reparto se realiza en segundo plano para no penalizar la respuesta de la aplicación.

- **Notificaciones:** cada aviso se guarda de forma persistente con su tipo —bajada de precio, nueva promoción, cambio en una lista compartida, o aprobación y rechazo de un comercio—, su título y texto, el estado de leído o no leído, un enlace directo a la pantalla afectada y un borrado lógico que permite descartarlo sin perder la traza.
- **Dispositivos registrados:** se asocia a cada usuario el identificador de sus dispositivos para poder enviarles las notificaciones, evitando duplicados cuando se reinstala la aplicación.

6.4.2. Módulo de portal business

- **Perfil de negocio:** datos del comercio sujetos a verificación —con su identificación fiscal única— junto con el umbral a partir del cual se le avisa de que un competidor cercano ofrece un precio más bajo.
- **Promociones:** ofertas temporales de un producto en una tienda, con la garantía de que no pueden coexistir dos promociones activas para el mismo producto en el mismo establecimiento.

Una tarea automática diaria revisa los precios de la competencia y avisa al comerciante cuando detecta diferencias superiores al umbral que haya configurado.

6.5— El Algoritmo de Optimización de Rutas

6.5.1. Formulación del problema

El algoritmo de optimización es la aportación técnica central del TFG. El problema combina dos subproblemas acoplados: (1) la *asignación* de cada ítem de la lista a un producto concreto del catálogo y a la tienda donde comprarlo, y (2) la *ordenación* de las tiendas seleccionadas para construir el recorrido más conveniente.

Este segundo subproblema se enmarca de forma natural en la **teoría de grafos**. El entorno de compra puede representarse como un grafo ponderado en el que los *vértices* (o nodos) son la ubicación de partida del usuario y las tiendas candidatas dentro del radio de búsqueda, y las *aristas* que unen cada par de vértices llevan asociado un peso doble: la distancia y el tiempo reales de desplazamiento entre esos dos puntos. Sobre ese grafo, encontrar el orden de visita que minimiza el coste total de recorrer las tiendas elegidas

y regresar es precisamente el **problema del viajante de comercio** (TSP) (Applegate y cols., 2006), uno de los problemas clásicos de optimización combinatoria sobre grafos. BarGAIN trabaja con una variante enriquecida: cada vértice-tienda aporta además un *premio* proporcional al ahorro que se consigue comprando en él, de modo que el recorrido óptimo no es solo el más corto, sino el que mejor equilibra desplazamiento, tiempo y ahorro según las preferencias del usuario. Al limitarse el número de paradas a un valor pequeño (entre una y cuatro), el grafo resultante es de tamaño reducido, lo que hace abordable su resolución en tiempo real.

Entrada:

- Lista de la compra $L = \{i_1, i_2, \dots, i_m\}$ con m artículos
- Ubicación del usuario $u = (lat, lng)$
- Radio máximo de búsqueda r (km) y número máximo de paradas k (de 1 a 4)
- Pesos de preferencia w_{precio} , $w_{distancia}$, w_{tiempo} (normalizados de forma que sumen 1)

Salida: la asignación de cada artículo a un producto, un precio y una tienda; la ruta recomendada con las paradas ya ordenadas y su desglose (precio total, distancia, tiempo y productos por parada); y, en su caso, los artículos ambiguos detectados, que el usuario puede precisar para recalcular la ruta.

Formulación matemática del problema de minimización

Formalicemos el problema completo. Para cada artículo $i \in L$ se dispone de un conjunto de *opciones candidatas* O_i ; cada opción $o \in O_i$ representa una terna (producto, tienda, precio) que satisface ese artículo, con un precio efectivo $\pi_o \geq 0$ y una tienda asociada $s(o) \in S$, siendo S el conjunto de tiendas dentro del radio r . Se introducen dos familias de variables de decisión binarias:

$$y_{io} = \begin{cases} 1 & \text{si el artículo } i \text{ se satisface con la opción } o, \\ 0 & \text{en caso contrario,} \end{cases} \quad z_s = \begin{cases} 1 & \text{si la tienda } s \text{ se incluye en la ruta,} \\ 0 & \text{en caso contrario.} \end{cases}$$

El coste de una solución combina dos términos: el *coste económico* de la cesta y el *coste logístico* del recorrido. El segundo depende del conjunto de tiendas visitadas $V(z) = \{s \in S : z_s = 1\}$ a través de la longitud de la ruta óptima que parte de la ubicación del usuario u , recorre $V(z)$ y regresa a u ; denotamos su distancia y su tiempo por $D(V(z))$ y $T(V(z))$, magnitudes que se determinan resolviendo el subproblema de ordenación de la Sección 6.5.5. El problema se enuncia entonces como la minimización

$$\min_{y,z} \quad w_{precio} \sum_{i \in L} \sum_{o \in O_i} \pi_o y_{io} + w_{distancia} \hat{D}(V(z)) + w_{tiempo} \hat{T}(V(z)) \quad (6.1)$$

sujeta a las restricciones

$$\sum_{o \in O_i} y_{io} = 1, \quad \forall i \in L, \quad (6.2)$$

$$y_{io} \leq z_{s(o)}, \quad \forall i \in L, \forall o \in O_i, \quad (6.3)$$

$$\sum_{s \in S} z_s \leq k, \quad (6.4)$$

$$y_{io}, z_s \in \{0, 1\}, \quad \forall i, o, s. \quad (6.5)$$

La restricción (6.2) obliga a satisfacer cada artículo con exactamente una opción; la (6.3) garantiza que, si una opción se utiliza, su tienda forma parte de la ruta; la (6.4) impone el número máximo de paradas k ; y la (6.5) fija el carácter binario de las decisiones. Los símbolos $\hat{D}$ y $\hat{T}$ denotan la distancia y el tiempo *normalizados* a una escala comparable con el término de precio, y los pesos cumplen $w_{\text{precio}} + w_{\text{distancia}} + w_{\text{tiempo}} = 1$ con $w_{\bullet} \geq 0$.

El problema (6.1)–(6.5) es un programa de optimización combinatoria *acoplado*: la función objetivo contiene, a través de $\hat{D}$ y $\hat{T}$, el coste de un problema del viajante que a su vez depende de qué tiendas se seleccionan. Pertenece a la clase de los problemas NP-duros —generaliza tanto la selección de subconjuntos con cardinalidad acotada como el TSP—, por lo que no admite, en general, una resolución exacta eficiente. Dado que la aplicación debe responder en tiempo interactivo (menos de 5 segundos), BarGAIN no resuelve el modelo de forma monolítica, sino que lo **descompone** en una secuencia de fases tratables que aproximan su óptimo: una heurística de asignación artículo–opción (Fase 1), una consolidación voraz que impone la cota de paradas de la ecuación (6.4) (Fase 2), el cálculo de los pesos logísticos (Fase 3) y la *resolución exacta* del subproblema de ordenación (Fase 4). Las secciones siguientes detallan cada fase.

6.5.2. Fase 1 — Resolución de los artículos de la lista

El usuario escribe su lista en lenguaje libre (“leche”, “manzanas”, “pan de molde”), de modo que el primer reto es averiguar a qué producto concreto del catálogo se refiere cada anotación y en qué tiendas está disponible. Esta correspondencia se resuelve mediante una cascada de filtros que va acotando progresivamente los candidatos:

1. **Preselección por semejanza textual:** se localizan los veinte productos del catálogo cuyo nombre más se parece al texto escrito, comparando fragmentos de tres letras para tolerar erratas y variantes de redacción.
2. **Interpretación del significado:** un modelo de lenguaje (Gemini (Google, 2026)) recibe el texto del usuario y esos candidatos, y decide cuáles encajan mejor, distinguiendo los productos *preferentes* de los *alternativos*. Además señala los casos en que la anotación es genérica y conviene preguntar al usuario (“manzanas” frente a “manzanas Fuji”). Si el modelo no está disponible, el sistema recurre a una regla local de respaldo basada en términos que cambian la naturaleza del producto (“rallado”, “molido”...). Las elecciones que el usuario ya confirmó en ocasiones anteriores tienen prioridad sobre esta inferencia.
3. **Recuperación de precios vigentes:** para los productos candidatos se obtiene, en una sola consulta a la base de datos, el último precio no caducado de cada combinación de producto y tienda dentro del radio.
4. **Puntuación de cada opción:** cada pareja de artículo y precio recibe una puntuación de parecido que pondera sobre todo la coincidencia del nombre del producto (85 %) y, en menor medida, el contexto de la tienda (15 %), penaliza los matices no solicitados por el usuario y premia las opciones que el modelo marcó como preferentes. Se conservan los doce mejores candidatos por artículo.
5. **Selección final:** entre las opciones prácticamente empatadas en parecido se elige la de menor precio efectivo, teniendo en cuenta el precio de oferta cuando existe.

6.5.3. Fase 2 — Consolidación de paradas

Si la asignación inicial usa más tiendas que el máximo de paradas k , una reducción greedy recoloca ítems de tiendas con un solo producto hacia tiendas ya seleccionadas,

eligiendo en cada paso el cambio de menor sobrecoste. Adicionalmente, si varias tiendas de una misma cadena participan en la ruta, se consolidan en una única tienda de esa cadena siempre que exista una alternativa sin sobrecoste.

6.5.4. Fase 3 — Cálculo de distancias y tiempos

Para poder valorar cada recorrido es necesario conocer el coste de desplazarse entre la ubicación del usuario y cada una de las tiendas, y entre las tiendas entre sí; en términos del grafo descrito antes, se trata de asignar un peso a cada arista. Estas distancias y tiempos reales de conducción se obtienen del servicio de rutas OpenRouteService (Heidelberg Institute for Geoinformation Technology (HeiGIT), 2026), que sobre la cartografía de calles devuelve de una sola vez todos los trayectos entre los puntos implicados. Cuando ese servicio no está disponible, el sistema recurre a una estimación de respaldo que calcula la distancia en línea recta entre los puntos y la convierte en tiempo suponiendo una velocidad urbana media de 40 km/h, de modo que la optimización nunca queda bloqueada por un fallo externo.

6.5.5. Fase 4 — Ordenación de la ruta

Una vez ponderadas las aristas del grafo, ordenar las paradas equivale a recorrerlo partiendo de la ubicación del usuario, visitando las tiendas elegidas y regresando al punto de partida con el menor coste posible: el ya mencionado problema del viajante. Su resolución se delega en la biblioteca de optimización combinatoria OR-Tools (Google OR-Tools Team, 2026), ampliamente empleada en el sector logístico. El peso de cada arista (i, j) —el coste de ir de la tienda i a la j — combina los tres criterios según las preferencias del usuario:

$$c(i, j) = w_{distancia} \cdot \hat{d}_{ij} + w_{tiempo} \cdot \hat{t}_{ij} - w_{precio} \cdot \hat{s}_j \quad (6.6)$$

donde $\hat{d}_{ij}$ y $\hat{t}_{ij}$ son la distancia y el tiempo entre ambos puntos, expresados en una escala comparable, y $\hat{s}_j$ representa el *ahorro* que aporta la tienda de destino j : cuanto más barata resulta la compra asignada a esa tienda frente a la siguiente mejor alternativa, mayor es el premio que reduce el coste de dirigirse hacia ella. Así, quien da prioridad al precio acepta pequeños desvíos hacia tiendas con mayor ahorro, mientras que quien prioriza la cercanía o la rapidez obtiene recorridos más compactos.

Formalmente, sea $N = \{0\} \cup V$ el conjunto de nodos del recorrido, donde 0 es la ubicación de partida del usuario y V las tiendas seleccionadas en las fases anteriores. Se define, para cada par ordenado de nodos (u, v) con $u \neq v$, la variable de decisión binaria

$$x_{uv} = \begin{cases} 1 & \text{si el recorrido va directamente del nodo } u \text{ al nodo } v, \\ 0 & \text{en caso contrario.} \end{cases}$$

La ordenación óptima de la ruta es el problema del viajante que minimiza el coste total de las aristas efectivamente recorridas:

$$\text{mín} \quad \sum_{u \in N} \sum_{\substack{v \in N \\ v \neq u}} c(u, v) x_{uv} \quad (6.7)$$

sujeto a las restricciones de grado —cada nodo se abandona y se alcanza exactamente una vez— y a la eliminación de subtours (formulación de Miller–Tucker–Zemlin), que impide soluciones formadas por ciclos desconectados:

$$\sum_{\substack{v \in N \\ v \neq u}} x_{uv} = 1, \quad \forall u \in N, \quad (6.8)$$

$$\sum_{\substack{u \in N \\ u \neq v}} x_{uv} = 1, \quad \forall v \in N, \quad (6.9)$$

$$\mu_i - \mu_j + |N| x_{ij} \leq |N| - 1, \quad \forall i, j \in V, i \neq j, \quad (6.10)$$

$$x_{uv} \in \{0, 1\}, \quad \forall u, v \in N, u \neq v. \quad (6.11)$$

donde las variables auxiliares $\mu_i \in \mathbb{R}$ asignan una posición de orden a cada tienda. El coste de arista $c(u, v)$ es el definido en la ecuación (6.6), e incorpora la distancia, el tiempo y el ahorro ponderados por las preferencias del usuario; obsérvese que la suma doble de la ecuación (6.7) recorre todas las aristas dirigidas posibles y que, gracias al carácter binario de x_{uv} , solo contribuyen al coste las que forman parte del recorrido elegido. Las distancias y los tiempos $\hat{D}$ y $\hat{T}$ que alimentan el término logístico del problema global (6.1) se obtienen evaluando este recorrido óptimo.

La búsqueda parte de una solución inicial razonable —encadenar en cada paso la conexión más económica— y se detiene en un máximo de cinco segundos, lo que garantiza una respuesta ágil aun cuando exista un gran número de combinaciones posibles. Dado que el número de paradas está acotado por $k \leq 4$, el grafo es de tamaño muy reducido y, en la práctica, OR-Tools alcanza el óptimo de forma prácticamente inmediata.

6.5.6. Fase 5 — Persistencia y desambiguación interactiva

El resultado de la optimización —la ruta, sus paradas ordenadas y los productos asignados a cada una— se guarda de forma íntegra en la base de datos, de manera que el usuario puede recuperarlo más tarde sin necesidad de recalcularlo. Cuando durante la resolución algún artículo resulta ambiguo, la respuesta ofrece las distintas interpretaciones posibles; al elegir una, el sistema la recuerda como preferencia de esa lista y recalcula la ruta, de modo que las optimizaciones futuras de la misma lista ya no vuelven a plantear la pregunta.

6.5.7. Diagrama de secuencia del optimizador

La Figura 6.4 ilustra el flujo de mensajes entre componentes durante una petición de optimización.

6.5.8. Evolución respecto al diseño inicial

El planteamiento preliminar de la fase de análisis contemplaba enumerar combinaciones de tiendas y devolver las tres mejores rutas alternativas ordenadas por una función de score global. Durante la implementación (fase F5) ese diseño se sustituyó por el descrito en esta sección por dos razones contrastadas en las pruebas de aceptación: primera, la calidad percibida del resultado depende mucho más de resolver correctamente la ambigüedad ítem-producto (de ahí la capa semántica) que de explorar un gran número de combinaciones de tiendas; y segunda, una única ruta recomendada con desglose por parada y desambiguación interactiva resultó más accionable para el usuario que tres alternativas similares. La generación de rutas alternativas se mantiene como línea de trabajo futuro (Capítulo 9).

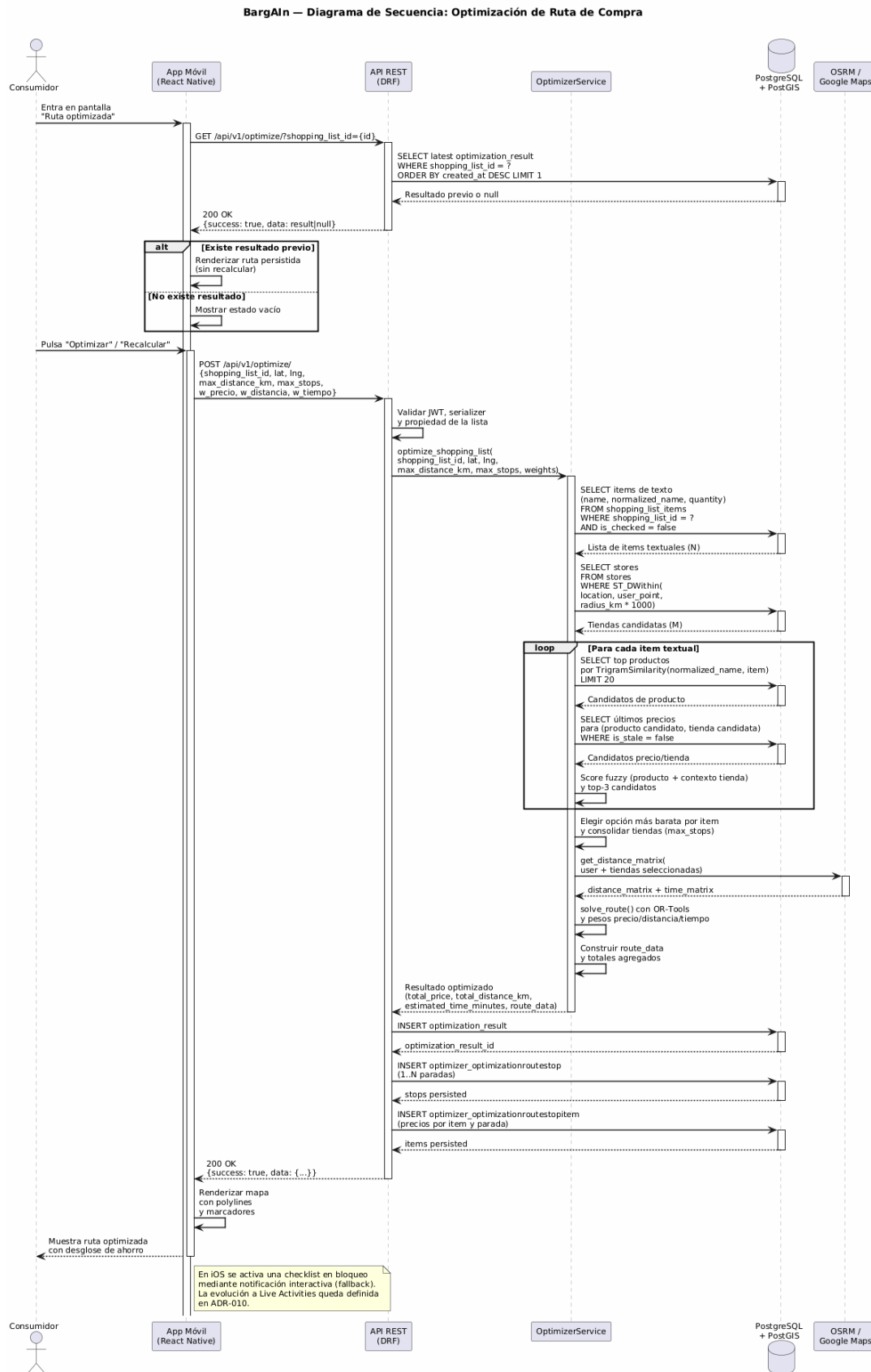


Figura 6.4: Diagrama de secuencia del proceso de optimización de ruta

6.5.9. Complejidad y salvaguardas de rendimiento

El objetivo de rendimiento de resolver la optimización en menos de 5 segundos se garantiza *por construcción* mediante cotas explícitas en cada fase, resumidas en el Cuadro 6.4. No se ha realizado aún una campaña formal de medición de latencias (p95) sobre el conjunto de datos de demostración; queda documentada como trabajo pendiente en el Capítulo 8.

Cuadro 6.4: Salvaguardas de rendimiento del optimizador

Fase	Mecanismo	Cota
Resolución de artículos	Búsqueda indexada + límite de candidatos	20 productos por artículo
Puntuación de candidatos	Recorte de la lista de opciones	12 candidatos por artículo
Cálculo de distancias	Una única consulta al servicio de rutas	1 llamada externa por optimización
Ordenación de ruta	Límite de tiempo de búsqueda	≤ 5 s

La optimización se resuelve de forma inmediata dentro de la propia petición del usuario: para los volúmenes del TFG —listas de decenas de artículos y decenas de tiendas por radio— las cotas anteriores mantienen el tiempo de respuesta dentro del objetivo. Procesar la optimización en segundo plano y reutilizar las distancias ya calculadas mediante una caché se identificaron como mejoras para escenarios de mayor carga, aunque no forman parte de la implementación actual.

6.6— Módulo de Web Scraping

La recogida automatizada de precios se construye como un proyecto independiente (Scrapy Developers, 2026), separado del código de la API. Cada recolector visita el catálogo en línea de una cadena, extrae los precios y los escribe ya normalizados en la base de datos común, registrando su origen como recogida automática.

El proyecto incluye **once recolectores**, uno por cadena (Mercadona, Carrefour, Lidl, DIA, Alcampo, Eroski, Hipercor, Consum, Covirán, Spar y Costco), todos cubiertos por pruebas. De ellos, **cuatro operan de forma programada** (Mercadona, Carrefour, Lidl y DIA, una vez al día y de madrugada, dentro de la frecuencia máxima de una recogida diaria por cadena); los siete restantes están implementados y verificados, pero no se han incorporado todavía a la ejecución periódica ni validado de forma continuada contra los sitios reales.

Cada cadena requiere una estrategia distinta según cómo publique sus precios. Algunas, como Mercadona, exponen los datos de forma estructurada y estable, accesible directamente. Otras, como Carrefour, Lidl o DIA, generan su catálogo dinámicamente en el navegador, por lo que se recurre a una herramienta capaz de simular un navegador real (Microsoft Playwright Team, 2026) que ejecute esa lógica y permita leer el contenido ya cargado.

Todos los recolectores respetan una espera mínima de dos segundos entre peticiones consecutivas para no sobrecargar los servidores de destino.

6.6.1. Consideraciones legales y éticas del scraping

La obtención automatizada de precios públicos plantea cuestiones legales y éticas que el proyecto aborda explícitamente:

- **Naturaleza de los datos.** Los datos extraídos son precios de venta al público, información factual y pública sin datos personales; el RGPD (Parlamento Europeo y Consejo de la Unión Europea, 2016) no aplica a esta extracción, aunque sí al

tratamiento de los datos de los usuarios de BarGAIN (en particular su ubicación), que se realiza con consentimiento explícito.

- **Protocolo de exclusión.** La política de scraping responsable del proyecto exige respetar las directivas `robots.txt` (Koster y cols., 2022) —el estándar mediante el que un sitio web declara qué puede recorrerse de forma automatizada— y excluir las cadenas que prohíban expresamente el acceso automático. Debe señalarse con honestidad que, en la configuración actual, la comprobación global de esas directivas está desactivada —la recogida se limita a rutas concretas, con un retardo de dos segundos y frecuencia diaria—; su activación, junto con la revisión individual de las condiciones de uso de cada cadena, está identificada como mejora necesaria para alinear plenamente la implementación con esa política de exclusión antes de cualquier operación a escala.
- **Minimización de impacto.** El retardo entre peticiones, la frecuencia máxima diaria por cadena y la preferencia por los datos estructurados que ya ofrecen los propios sitios reducen la carga impuesta a los servidores de destino, en línea con las obligaciones generales de diligencia de la Ley de Servicios de la Sociedad de la Información y de Comercio Electrónico (LSSI-CE) (Jefatura del Estado, 2002).
- **Estrategia de sustitución.** La prioridad de fuentes del sistema, que antepone la API oficial a los datos de primera mano del comercio y estos a la recogida automática, permite reemplazar la recogida automática por acuerdos o APIs oficiales sin cambios de arquitectura, que es el escenario objetivo de una eventual explotación comercial.

6.7— Módulo OCR

Cuando el usuario fotografía un ticket o una lista manuscrita, el backend envía la imagen al servicio de reconocimiento óptico de Google Cloud Vision (Google Cloud, 2026), que devuelve el texto contenido en ella. La elección de este servicio (ADR-007) se adoptó tras comprobar que la alternativa local previamente considerada (Tesseract) no reconocía con suficiente fiabilidad los tickets y listas fotografiados en condiciones reales. Antes de enviarla, la imagen se preprocesa para mejorar la calidad del reconocimiento, y del resultado se filtran las líneas que realmente parecen corresponder a productos, descartando el resto del ruido (totales, fechas, datos del establecimiento, etc.).

Cada línea reconocida se normaliza y se empareja con el producto más parecido del catálogo mediante la misma técnica de coincidencia aproximada empleada en la búsqueda, exigiendo un nivel mínimo de confianza. Al cliente se le devuelve, para cada línea, una lista de candidatos con su grado de confianza, de modo que el usuario revise y confirme el resultado antes de incorporarlo a la lista.

6.7.1. Diagrama de secuencia OCR

La Figura 6.5 muestra el flujo de procesamiento desde la captura de imagen hasta la inserción de ítems en la lista.

6.8— Integración del Asistente LLM

El asistente conversacional se apoya en un modelo de lenguaje de Google, Gemini (ADR-008). El backend actúa como intermediario: antes de remitir cada mensaje al modelo, le añade el contexto de la compra del usuario —sus listas, las tiendas cercanas y

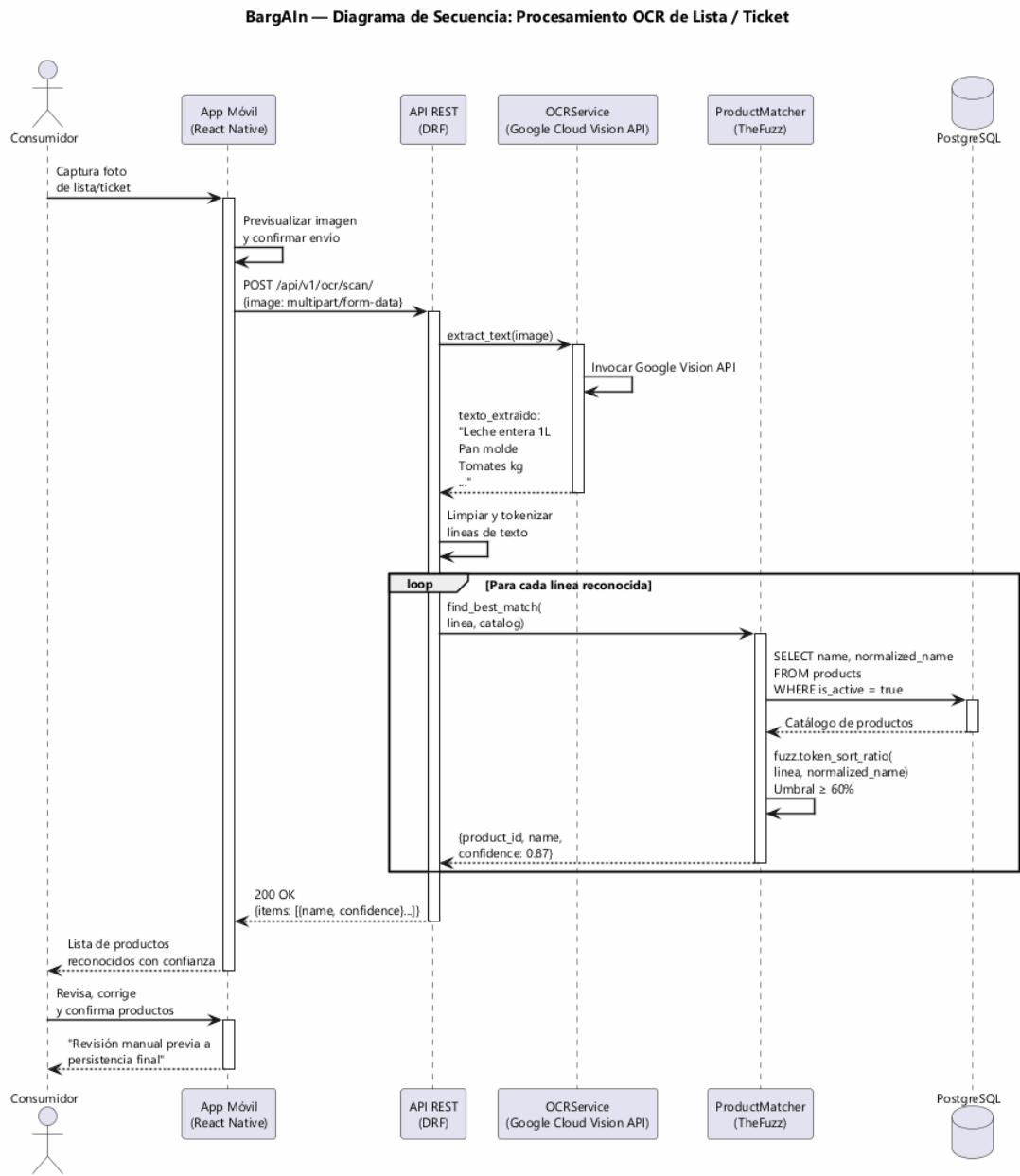


Figura 6.5: Diagrama de secuencia del flujo OCR

los precios pertinentes— de modo que las respuestas se basen en datos reales y no en información genérica.

Para contener el coste de cada consulta, solo se envían al modelo los mensajes más recientes de la conversación en lugar de todo el historial. Además, se le instruye explícitamente para que limite sus respuestas al ámbito de la compra doméstica, y el servicio impone un máximo de treinta consultas por hora y usuario para evitar un uso abusivo.

De forma independiente, la fase de interpretación de la lista descrita en la Sección 6.5.2 emplea un modelo configurable por separado, lo que permite reservar para la clasificación de productos un modelo distinto al del asistente conversacional.

6.9— Infraestructura y Despliegue

6.9.1. Contenedores Docker

Cada componente se ejecuta en un contenedor independiente, coordinados entre sí, lo que aísla sus dependencias y facilita reproducir el sistema en cualquier entorno:

- un servidor frontal que recibe las peticiones y sirve los contenidos estáticos;
- el servicio del backend que atiende la lógica de la API;
- los procesos que ejecutan las tareas en segundo plano y las que se lanzan de forma periódica;
- la base de datos con sus capacidades geoespaciales;
- el componente que actúa como cola de tareas y caché.

Para el entorno de desarrollo se adopta un modelo híbrido (ADR-002): el backend se ejecuta en contenedores, pero la aplicación de usuario se ejecuta directamente en el equipo de desarrollo, ya que contenerizarla en Windows interfería con la recarga automática de cambios durante la programación.

No obstante, esa decisión atañe únicamente al ciclo de desarrollo. Para facilitar la puesta en marcha del frontend sin preparar un entorno local, el proyecto incluye además un despliegue en contenedor de la versión web de la aplicación: una imagen genera su compilación estática y la sirve mediante un servidor ligero, conectada al backend desplegado, de modo que el frontend puede levantarse con un único comando (Capítulo 7). Al tratarse de una compilación ya construida, este contenedor no recurre a la recarga en caliente y, por tanto, no incurre en la limitación que motivó el modelo híbrido.

6.9.2. Pipeline CI/CD

El repositorio automatiza la integración y la entrega continuas (CI/CD) mediante cinco flujos de trabajo que se disparan automáticamente ante cada cambio:

- la verificación del backend, que comprueba el estilo del código y ejecuta su batería de pruebas, exigiendo de forma bloqueante una cobertura mínima del 80 %;
- la verificación de la aplicación de usuario, con sus propias comprobaciones de estilo y pruebas;
- la generación automática de la versión instalable para iPhone, que se compila en un entorno macOS y se instala después en el dispositivo de demostración;
- el despliegue del backend en el servidor público cuando se actualiza su código;

- la publicación del portal de comercios como sitio web estático.

El despliegue público se describe de forma declarativa, de modo que toda la infraestructura —servidor, procesos en segundo plano, cola de tareas y base de datos— se reconstruye automática y reproduciblemente a partir de esa descripción. Durante el TFG se empleó una variante adaptada al plan gratuito que consolida varios procesos en un único servicio para ajustarse a sus recursos.

6.9.3. Gestión de la configuración sensible

La información sensible —claves de los servicios externos, credenciales de la base de datos y las claves de seguridad de la aplicación— nunca se incrusta en el código, sino que se gestiona exclusivamente mediante variables de entorno. El proyecto incluye una plantilla con los nombres de todas las variables requeridas, sin sus valores reales, para facilitar la puesta en marcha en un nuevo entorno.

6.10— Diseño de la Interfaz de Usuario

6.10.1. Sistema de diseño

La aplicación cuenta con un sistema de diseño propio, articulado bajo el concepto “Mercado Mediterráneo Digital”é inspirado en la estética de los mercados sevillanos, que unifica color, tipografía y espaciado en toda la interfaz para transmitir cercanía y confianza. En lugar de fijar valores concretos dispersos por el código, el sistema centraliza todas las decisiones visuales en un conjunto de *tokens* de diseño —variables con nombre semántico— que las pantallas consumen de forma indirecta. Así, un cambio en el origen se propaga de manera coherente a toda la aplicación, y la apariencia queda desacoplada de cada pantalla concreta.

La paleta se organiza por *función* y no por tono: un color principal de marca para las acciones y los énfasis, un color de signo positivo reservado al ahorro y a las confirmaciones, un color de aviso para errores y situaciones críticas, y una gama neutra de fondos, superficies y textos que sostiene la legibilidad. Esta asignación semántica garantiza que un mismo significado —por ejemplo, “ahorro”ó “error”— se represente siempre del mismo modo, refuerza la jerarquía visual y facilita mantener el contraste exigido por las pautas de accesibilidad. La tipografía sigue el mismo criterio: una fuente con personalidad para los titulares y otra de alta legibilidad para el texto corrido, con el espaciado organizado sobre una escala regular que aporta ritmo y consistencia a todas las vistas.

6.10.2. Pantallas principales

La aplicación se estructura en cinco pestañas principales:

1. **Inicio / Dashboard:** Resumen de la lista activa, precio estimado en la tienda más barata cercana, acceso rápido al asistente y alertas activas.
2. **Mi Lista:** Gestión de la lista activa con buscador de autocompletado, grupos por categoría, gestos de deslizamiento para marcar o eliminar productos y botón de optimizar ruta.
3. **Explorar:** Mapa interactivo con marcadores de tiendas y panel inferior deslizable con detalles de la tienda seleccionada.
4. **Optimizador:** Deslizadores de pesos, resumen agregado de la ruta recomendada con su desglose por parada, resolución de ambigüedades en los productos y visualización de la ruta en el mapa.

5. **Perfil:** Datos personales, preferencias de optimización, configuración de notificaciones y acceso al historial del asistente.

6.10.3. Adaptación a uso web (responsive y conveniencias de escritorio)

La misma base de código de la aplicación móvil se reutiliza para su ejecución en navegadores de escritorio, sin duplicar pantallas ni alterar la navegación. La adaptación al escritorio se construye como una capa transversal sobre las pantallas existentes, organizada en cuatro ejes:

1. **Diseño adaptable.** La interfaz reconoce el ancho disponible y se reorganiza en consecuencia: en móvil presenta una sola columna; en pantallas grandes centra el contenido a un ancho cómodo de lectura y distribuye los listados de tarjetas en varias columnas, aprovechando disposiciones de dos paneles (lista y detalle) cuando el espacio lo permite.
2. **Interacción con ratón y teclado.** Se incorporan realces al pasar el cursor, atajos de teclado para las acciones frecuentes (enfocar el buscador, añadir un producto o cerrar un diálogo), enfoque automático de los campos y ayudas contextuales; la lista de la compra puede reordenarse arrastrando sus elementos.
3. **Conveniencias de escritorio.** El usuario puede exportar la lista de la compra, la comparativa de precios y la ruta a ficheros, así como copiar direcciones y enlaces al portapapeles. La exportación a hoja de cálculo incorpora una salvaguarda de seguridad que neutraliza el riesgo de inyección de fórmulas en las celdas.
4. **Enlaces directos.** Cada pantalla relevante dispone de una dirección web estable, de modo que el estado de la aplicación —una lista, una tienda o una búsqueda concreta— puede guardarse como marcador o compartirse mediante un enlace.

Todas estas mejoras se activan únicamente en la versión web o según el tamaño de pantalla, de forma que el comportamiento en móvil permanece inalterado. Se incorporaron en una fase específica de adaptación web posterior al cierre del plan inicial, y se validaron con un amplio conjunto de pruebas automatizadas y una batería de verificación manual en navegador.

Manual de Usuario

7.1— Introducción

Este capítulo describe, desde la perspectiva del usuario final, el funcionamiento completo de BarGAIN en sus tres contextos de uso: la aplicación móvil para consumidores (iOS), su versión de escritorio ejecutada en navegador, y el portal web orientado a pequeños y medianos comercios (PYMEs). La exposición sigue el recorrido natural de un usuario: desde el registro y el acceso, pasando por la gestión de listas de la compra y la comparación de precios, hasta las funcionalidades diferenciales del sistema —la optimización multicriterio de rutas de compra, el reconocimiento óptico de tickets y el asistente conversacional—, para concluir con las herramientas de gestión que ofrece el portal *business*.

Todas las imágenes que ilustran este capítulo son capturas reales de la aplicación desplegada en el entorno de desarrollo descrito en el Capítulo 6, obtenidas sobre un conjunto de datos de demostración representativo (catálogo normalizado, precios de varias cadenas de supermercados en Sevilla y un comercio local verificado). Las capturas de la aplicación móvil corresponden a un *viewport* de 390×844 puntos, equivalente al de un teléfono actual de gama media; las de escritorio, a una ventana de 1440×900.

7.2— Despliegue y acceso al sistema

BarGAIN se encuentra desplegado en infraestructura pública: el backend (API REST, base de datos PostgreSQL/PostGIS y cola Redis) se ejecuta en *Render* bajo el localizador uniforme de recursos (URL) <https://bargain-free-api.onrender.com>, y el portal para comercios se sirve como aplicación estática desde *GitHub Pages* en <https://qhx0329.github.io/bargain-tfg/>. La aplicación de usuario final, construida con Expo, puede ejecutarse en un navegador de escritorio desde el propio repositorio o instalarse en un iPhone a partir del artefacto generado por la integración continua. Todas las variantes del cliente consumen el mismo backend de Render, por lo que los datos son consistentes entre plataformas.

Conviene tener presente una particularidad del plan gratuito de Render: tras un periodo de inactividad el servicio se hiberna, de modo que la primera petición puede demorarse en torno a un minuto (*cold start*) mientras la instancia se reactiva. Las peticiones posteriores responden con normalidad.

7.2.1. Acceso al portal para empresas (GitHub Pages)

El portal *business* no requiere instalación alguna: basta un navegador moderno. El procedimiento de acceso es el siguiente:

1. Navegar a <https://qh0329.github.io/bargain-tfg/>. Se carga la página de presentación del portal.
2. Pulsar *Empezar onboarding* (o navegar a */login* si ya se dispone de cuenta).
3. Registrar la cuenta del comercio con sus datos de acceso (usuario, correo y contraseña) y los datos del negocio (razón social, CIF/NIF y dirección).
4. Esperar la verificación del perfil por parte de un administrador; hasta entonces la cuenta permanece en estado *pendiente*.
5. Una vez aprobado, iniciar sesión para acceder al centro de operaciones descrito en la Sección 7.15.

7.2.2. Ejecución de la aplicación de usuario en local (Expo web)

La aplicación de consumo puede ejecutarse en cualquier equipo de escritorio sin necesidad de dispositivo móvil, reutilizando el soporte web de Expo. Los únicos prerequisites son *Git* y *Node.js* 24 o superior (que incluye *npm*). El procedimiento es:

1. Clonar el repositorio y situarse en el directorio del frontend:

```
git clone https://github.com/QHX0329/bargain-tfg.git
cd bargain-tfg/frontend
```

2. Instalar las dependencias del proyecto:

```
npm install
```

3. Revisar la configuración de entorno del fichero `frontend/.env.local`. La variable `EXPO_PUBLIC_API_URL` apunta por defecto al backend desplegado en Render:

```
EXPO_PUBLIC_API_URL=https://bargain-free-api.onrender.com/api/v1
```

Para trabajar contra un backend local en Docker basta sustituirla por `http://localhost:8000/api/v1`.

4. Arrancar el servidor de desarrollo en modo web:

```
npx expo start --web
```

5. Abrir `http://localhost:8081` en el navegador. Se mostrará la pantalla de inicio de sesión; el registro de una cuenta nueva se realiza desde la propia interfaz.

7.2.3. Despliegue del frontend en Docker (la forma más simple)

El procedimiento anterior exige instalar *Node.js* y las dependencias del proyecto en la máquina. Como alternativa aún más sencilla, pensada para quien solo desea probar la aplicación sin preparar un entorno de desarrollo, el proyecto incorpora un despliegue del frontend en contenedor: una imagen compila la versión web de la aplicación y la sirve ya construida, conectada automáticamente al backend público de Render. El único requisito es disponer de Docker en el equipo.

A diferencia del modelo de desarrollo —en el que la aplicación se ejecuta de forma nativa para conservar la recarga en caliente (Capítulo 6)—, este despliegue genera una *compilación estática* de la aplicación web y la publica mediante un servidor ligero, de modo que el contenedor arranca de forma reproducible en cualquier sistema sin interferir con el flujo de programación. El procedimiento se reduce a dos pasos: clonar el repositorio y, situado en su raíz, construir y levantar el contenedor con un único comando:

```
docker compose -f docker-compose.web.yml up --build
```

Al finalizar, la aplicación queda disponible en `http://localhost:8081`, lista para registrarse o iniciar sesión contra el backend desplegado. Si se desea apuntar a otra instancia del backend, la dirección puede ajustarse mediante una variable de entorno documentada en el propio fichero de orquestación; por defecto utiliza el servicio público de Render, por lo que no requiere configuración adicional.

7.2.4. Instalación de la aplicación en iPhone

La distribución para iOS se apoya en el flujo de integración continua *iOS Build* (GitHub Actions), que compila la aplicación en un runner macOS y publica como artefacto una IPA sin firmar (**BarGAIN-unsigned**). Al no estar distribuida a través del App Store, la instalación requiere firmar la IPA con un identificador de Apple, operación que resuelven herramientas de *sideloading* como *Sideloadly* o *AltStore*. El procedimiento con Sideloadly es el siguiente:

1. Descargar el artefacto **BarGAIN-unsigned** de la última ejecución del workflow *iOS Build* en la pestaña *Actions* del repositorio, y descomprimirlo para obtener el fichero `.ipa`.
2. Conectar el iPhone al ordenador mediante USB y autorizar el equipo desde el teléfono.
3. Abrir Sideloadly, arrastrar la IPA a la ventana de la aplicación, seleccionar el dispositivo e introducir un Apple ID. Con una cuenta gratuita, la firma personal tiene una validez de siete días, transcurridos los cuales basta repetir la operación; con una cuenta del programa de desarrolladores la validez se extiende a un año.
4. Pulsar *Start* y esperar a que finalice la firma e instalación.
5. En el iPhone, confiar en el certificado del desarrollador desde *Ajustes* → *General* → *VPN y gestión de dispositivos*.
6. Abrir BarGAIN. La aplicación se conecta automáticamente al backend de Render, por lo que no requiere configuración adicional; conceder el permiso de ubicación cuando se solicite para habilitar el mapa y la optimización de rutas.

La versión actual de BarGAIN se distribuye exclusivamente para iPhone (iOS 14 o superior) y navegadores de escritorio; las funciones de mapa y optimización requieren además el permiso de ubicación concedido y conexión a Internet.

Para el entorno de desarrollo local completo (backend incluido), el sistema se levanta con el backend en Docker (`make dev`), la base de datos migrada (`make migrate`) y el frontend Expo ejecutado de forma nativa en el host (`make frontend`), conforme al modelo híbrido adoptado en el ADR-002.

7.3— Registro y acceso al sistema

El acceso a la aplicación se articula en torno a un esquema clásico de autenticación con nombre de usuario y contraseña sobre tokens JWT. La pantalla de inicio de sesión (Figura 7.1, izquierda) presenta un formulario deliberadamente austero —dos campos y una única acción principal— con el fin de minimizar la fricción de entrada. El formulario de registro (Figura 7.1, derecha) solicita únicamente los datos imprescindibles para crear la cuenta: nombre de usuario, nombre y apellidos, correo electrónico (único, empleado para

The image shows two side-by-side screenshots of the BarGAIN application interface. The left screenshot is the login screen, featuring the BarGAIN logo at the top, the tagline 'Tu compra inteligente', and input fields for 'Usuario' (demo@bargain.local) and 'Contraseña'. Below these is an 'Iniciar sesión' button, a link for '¿Olvidaste tu contraseña?', and a link for '¿No tienes cuenta? Regístrate'. The right screenshot is the registration screen, titled 'Crea tu cuenta' with the tagline 'Únete a BargAln y empieza a ahorrar'. It includes input fields for 'Nombre de usuario' (Nico), 'Nombre' (Parrilla), 'Apellidos' (nico@example.), and 'Email' (Demo1234!). There are also fields for 'Contraseña' and 'Repetir' (both masked with dots), followed by a 'Crear cuenta' button. At the bottom, there is a link for '¿Ya tienes cuenta? Inicia sesión'.

Figura 7.1: Acceso al sistema: inicio de sesión (izquierda) y registro de un nuevo usuario (derecha)

comunicaciones y recuperación de contraseña) y contraseña con confirmación, aplicando validación inmediata en el cliente antes de delegar en la validación definitiva del servidor.

Tras una autenticación satisfactoria, el sistema emite un par de tokens (acceso y refresco) y carga la navegación principal por pestañas. La sesión se mantiene de forma transparente: si el token de acceso expira, el interceptor HTTP lo renueva automáticamente con el token de refresco, de modo que el usuario solo vuelve a introducir credenciales si permanece inactivo más allá de la vida útil de este último. En el cliente móvil ambos tokens se custodian en el almacenamiento seguro del sistema operativo (*Keychain* de iOS).

7.4– Pantalla principal

La pantalla de inicio (Figura 7.2) actúa como cuadro de mando del usuario y condensa, en un único recorrido vertical, la información de mayor valor inmediato: un buscador global con acceso directo al catálogo; una botonera de acciones rápidas (plantillas, productos, escaneo OCR y favoritos); las notificaciones más recientes; las listas activas con su número de productos; las tiendas detectadas en el radio configurado; y las alertas de precio vigentes con su precio objetivo. Esta disposición responde a un principio de diseño que ha guiado toda la interfaz: las tareas frecuentes deben resolverse en un máximo de tres interacciones desde el arranque de la aplicación.

7.5– Gestión de listas de la compra

La lista de la compra es la entidad vertebradora de BarGAIN: sobre ella se calculan totales por tienda, se lanza la optimización de rutas y se articula la colaboración entre usuarios. La pestaña *Listas* (Figura 7.3, izquierda) muestra las listas del usuario con ac-

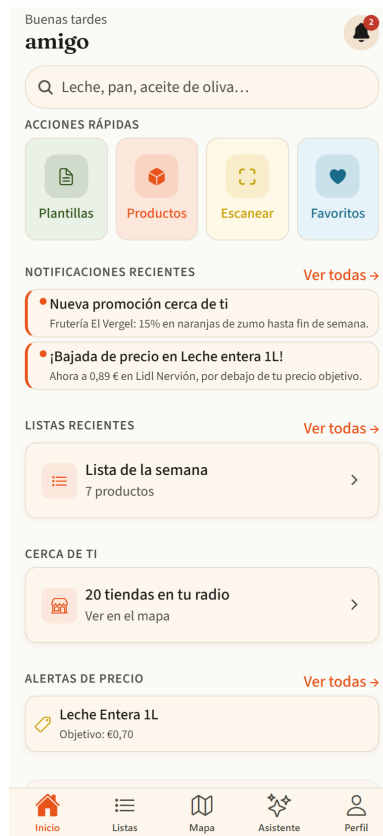


Figura 7.2: Pantalla principal de la aplicación móvil, con notificaciones, listas activas, tiendas cercanas y alertas de precio

ciones directas de duplicado, conversión en plantilla y archivado. El detalle de una lista (Figura 7.3, derecha) presenta los ítems con controles de cantidad “+/-”, casillas de verificación para marcar productos como comprados y acciones de exportación y compartición en la cabecera.

El flujo de trabajo habitual es el siguiente: el usuario crea una lista con nombre personalizado; añade productos desde el catálogo, desde el buscador con autocompletado o mediante el escáner OCR; ajusta cantidades; y, durante la compra, marca cada producto adquirido. Una lista puede compartirse con hasta diez colaboradores, cuyas modificaciones se notifican al resto de participantes, y puede convertirse en plantilla para institucionalizar compras recurrentes. Las plantillas (Figura 7.4) funcionan como generadores de listas: a partir de una plantilla —por ejemplo, la compra básica semanal— se instancian nuevas listas con un solo gesto, evitando reconstruir manualmente colecciones de productos que apenas varían de una semana a otra.

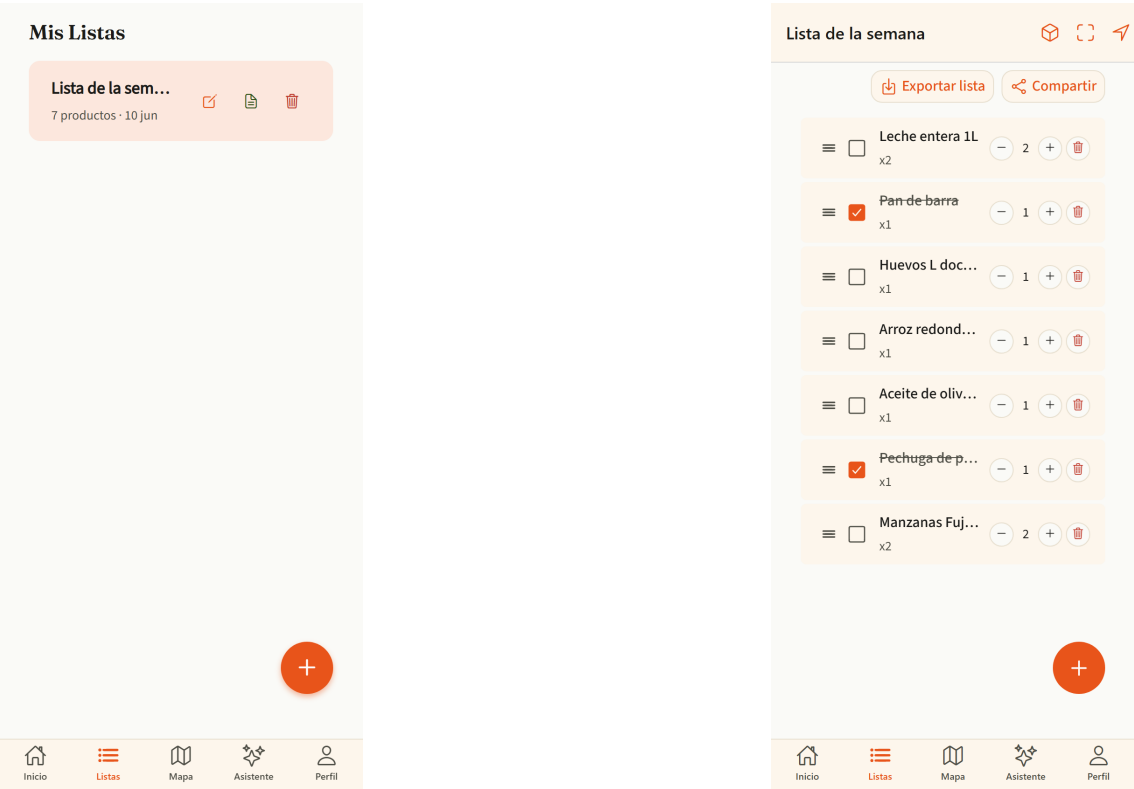


Figura 7.3: Gestión de listas: colección de listas del usuario (izquierda) y detalle de una lista con controles de cantidad y verificación (derecha)

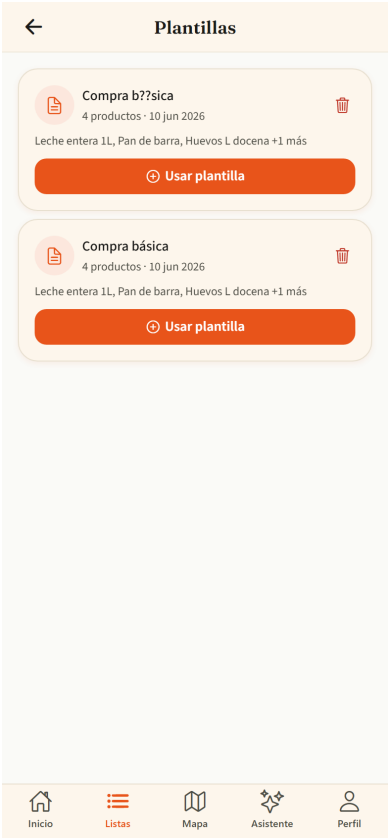


Figura 7.4: Plantillas de lista: instanciación de compras recurrentes a partir de una plantilla guardada

7.6— Catálogo y comparativa de precios

El catálogo (Figura 7.5, izquierda) ofrece una vista paginada del repositorio de productos normalizados, con filtros plegables por categoría, precio y distancia. Cada tarjeta sintetiza la información decisiva para el usuario: el precio mínimo detectado y la tienda que lo ofrece, junto con una acción de añadido rápido a la lista activa que se resuelve con un único toque.

Desde cualquier tarjeta se accede a la comparativa de precios del producto (Figura 7.5, derecha), que constituye una de las aportaciones centrales del sistema: una tabla ordenable con el precio del artículo en cada tienda del radio configurado, señalando las ofertas vigentes y la antigüedad de cada dato según su fuente (API oficial, datos del propio comercio o recogida automática, por este orden de fiabilidad). La misma pantalla incorpora el histórico de precios con un gráfico de evolución temporal, que permite valorar si el precio actual es coyuntural o estable, y la creación de alertas de precio sobre el producto consultado.

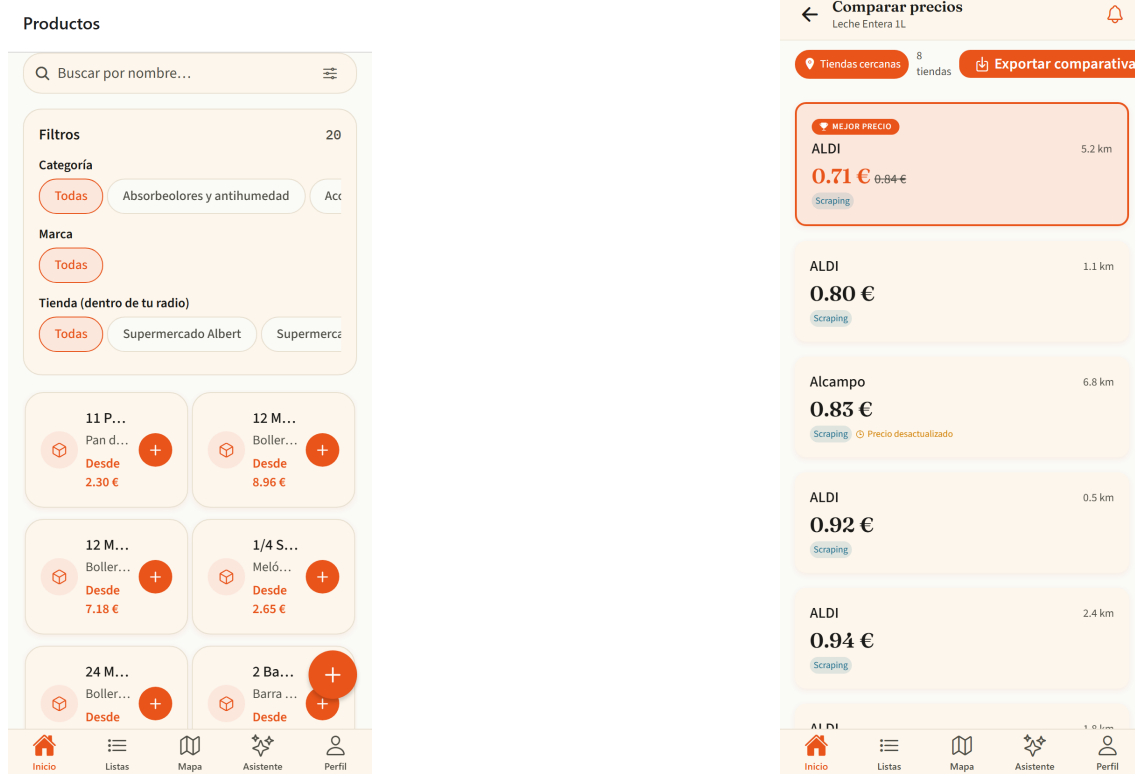


Figura 7.5: Catálogo de productos con filtros y añadido rápido (izquierda) y comparativa de precios por tienda de un producto (derecha)

El catálogo es deliberadamente abierto a la participación: cuando un usuario no encuentra un producto, puede proponer su alta mediante el formulario de la Figura 7.6, indicando nombre, marca, código de barras y categoría. La propuesta queda pendiente de revisión por un administrador, lo que preserva la calidad del repositorio común sin renunciar al crecimiento colaborativo del mismo.

The screenshot shows a mobile application interface for proposing a new product. The title bar at the top is orange with a back arrow and the text 'Proponer producto'. The form is divided into two main sections: 'INFORMACIÓN DEL PRODUCTO' and 'PRECIO'. The 'INFORMACIÓN DEL PRODUCTO' section includes fields for 'Nombre *' (with example 'Ej. Leche semidesnatada 1L'), 'Marca' (with example 'Ej. Hacendado'), and 'Código EAN-13' (with example 'Ej. 8412345678901'). The 'PRECIO' section includes fields for 'Precio (€)' (with example '0.00') and 'Precio unitario (€/kg o €/L)' (with example 'Ej. 1.20'). Below these is a 'Tienda' section with two buttons: 'Supermercado Albert' and 'Supermercados El Jamón'. At the bottom of the form is a 'Notas adicionales' section with a text area containing the example 'Ej. Encontré este producto en el supermercado X pero no aparece en la app'. The bottom of the screen features a navigation bar with five icons: a house for 'Inicio', a list for 'Listas', a map for 'Mapa', a star for 'Asistente', and a person for 'Perfil'.

< Proponer producto

INFORMACIÓN DEL PRODUCTO

Nombre *

Ej. Leche semidesnatada 1L

Marca

Ej. Hacendado

Código EAN-13

Ej. 8412345678901

PRECIO

Precio (€)

0.00

Precio unitario (€/kg o €/L)

Ej. 1.20

Tienda

Supermercado Albert Supermercados El Jamón

Notas adicionales

Ej. Encontré este producto en el supermercado X pero no aparece en la app

Inicio Listas Mapa Asistente Perfil

Figura 7.6: Propuesta de alta de un nuevo producto en el catálogo, sujeta a aprobación por el administrador

7.7– Mapa de tiendas

La pestaña *Mapa* (Figura 7.7, izquierda) sitúa al usuario en el centro de su entorno comercial: sobre cartografía interactiva se representan las tiendas del radio configurado con marcadores diferenciados por cadena, y un panel inferior plegable lista los establecimientos visibles con su distancia y tiempo estimado de desplazamiento. Al seleccionar un establecimiento se abre su perfil (Figura 7.7, derecha), que reúne la información de contacto y horario junto con el catálogo de productos detectados en esa tienda y sus precios, de modo que el usuario puede evaluar un comercio concreto sin abandonar el contexto del mapa.

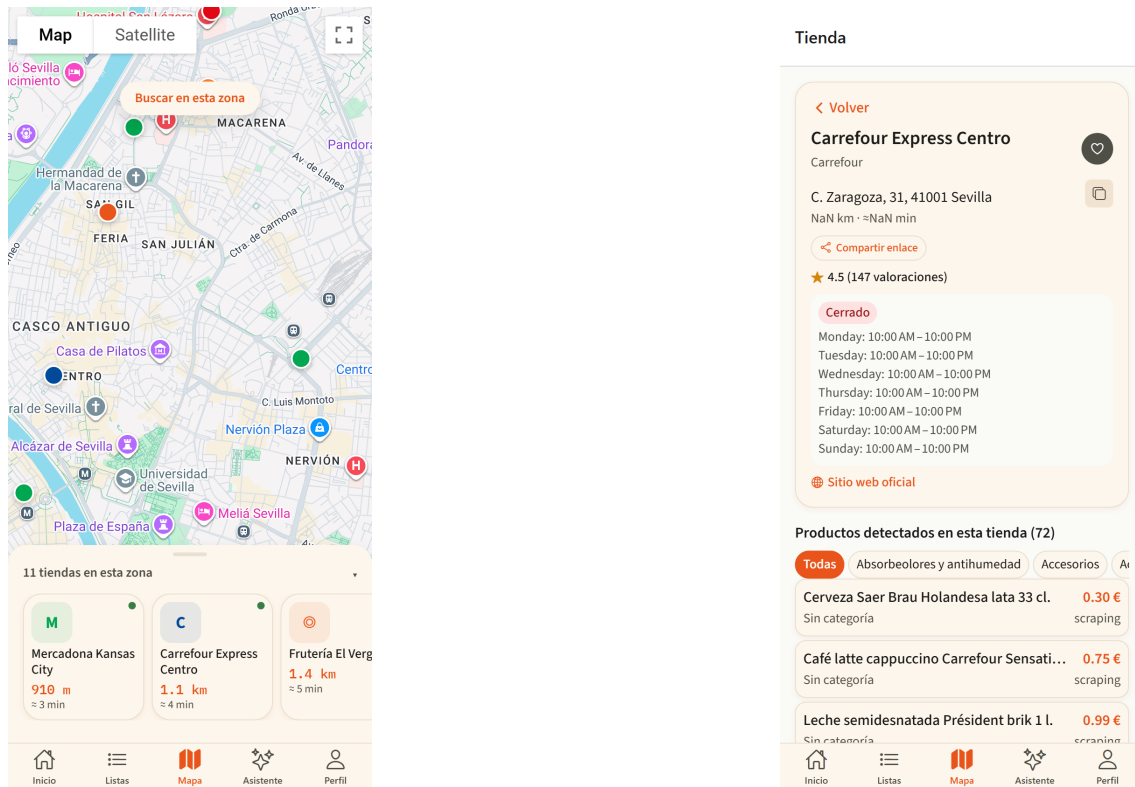


Figura 7.7: Mapa de tiendas con marcadores por cadena y panel de establecimientos (izquierda) y perfil de tienda con sus productos y precios (derecha)

El sistema permite marcar tiendas como favoritas, que se agrupan en una vista propia (Figura 7.8) accesible tanto desde el mapa como desde las acciones rápidas de la pantalla principal. Las tiendas favoritas reciben un tratamiento preferente en la generación de candidatos del optimizador de rutas, reflejando la fidelidad del usuario a determinados establecimientos.

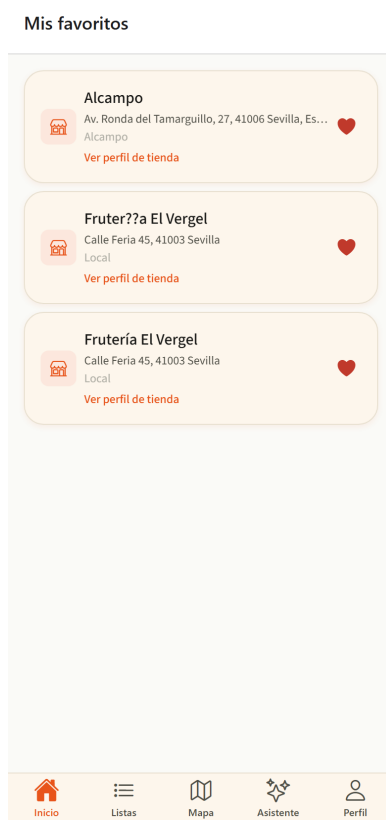


Figura 7.8: Tiendas favoritas del usuario, con acceso directo a su perfil

7.8— Alertas de precio y notificaciones

BarGAIN invierte la carga de la vigilancia de precios: en lugar de obligar al usuario a consultar periódicamente la comparativa, es el sistema quien le avisa cuando se cumple una condición de interés. La bandeja de notificaciones (Figura 7.9, izquierda) centraliza tres familias de avisos —bajadas de precio que alcanzan el objetivo fijado, nuevas promociones de comercios cercanos y cambios en listas compartidas— con marcado de leídas individual o masivo, borrado lógico y enlaces profundos que conducen directamente a la pantalla afectada. Estos mismos avisos se entregan como notificaciones *push* en el dispositivo.

La gestión de alertas (Figura 7.9, derecha) muestra las alertas activas con su precio objetivo y permite editarlas o eliminarlas. Una alerta se crea desde la comparativa de un producto indicando el umbral deseado; cuando algún establecimiento publica un precio igual o inferior, el backend dispara la notificación correspondiente.

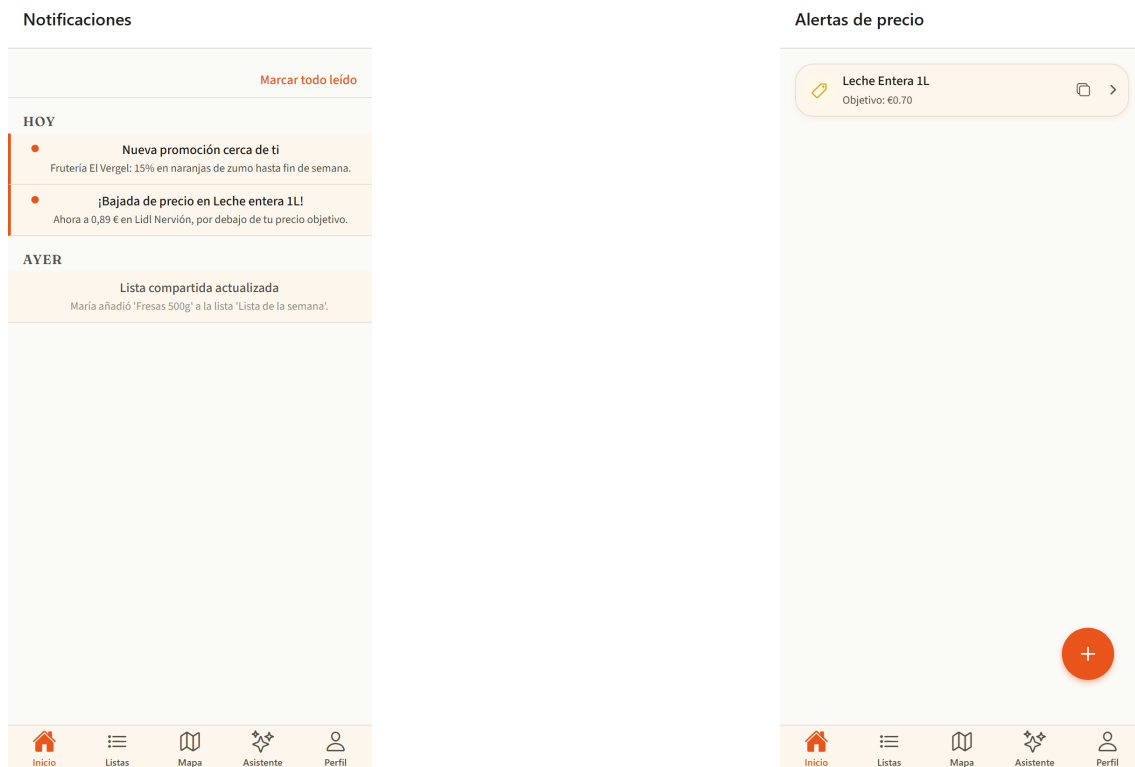


Figura 7.9: Bandeja de notificaciones con avisos de precio, promociones y listas compartidas (izquierda) y gestión de alertas de precio con su objetivo (derecha)

7.9— Perfil y preferencias del usuario

La pestaña *Perfil* (Figura 7.10, izquierda) agrupa la identidad del usuario y la configuración global de la aplicación: datos personales, preferencias de notificación y cierre de sesión, que elimina ambos tokens del almacenamiento seguro del dispositivo (el token de refresco rotado queda además en lista negra en el servidor en la siguiente renovación). Su apartado más relevante para el comportamiento del sistema es la configuración del optimizador (Figura 7.10, derecha), donde el usuario gobierna la función de *scoring* multicriterio descrita en el Capítulo 6: el radio máximo de búsqueda (1–20 km), el número máximo de paradas por ruta y, sobre todo, los pesos relativos w_{precio} , $w_{distancia}$ y w_{tiempo} , ajustables mediante deslizadores cuya suma se normaliza automáticamente a la unidad. De este modo, una misma lista produce rutas distintas para un usuario que prioriza el ahorro

frente a otro que prioriza la inmediatez, sin que ninguno de los dos necesite comprender el algoritmo subyacente.

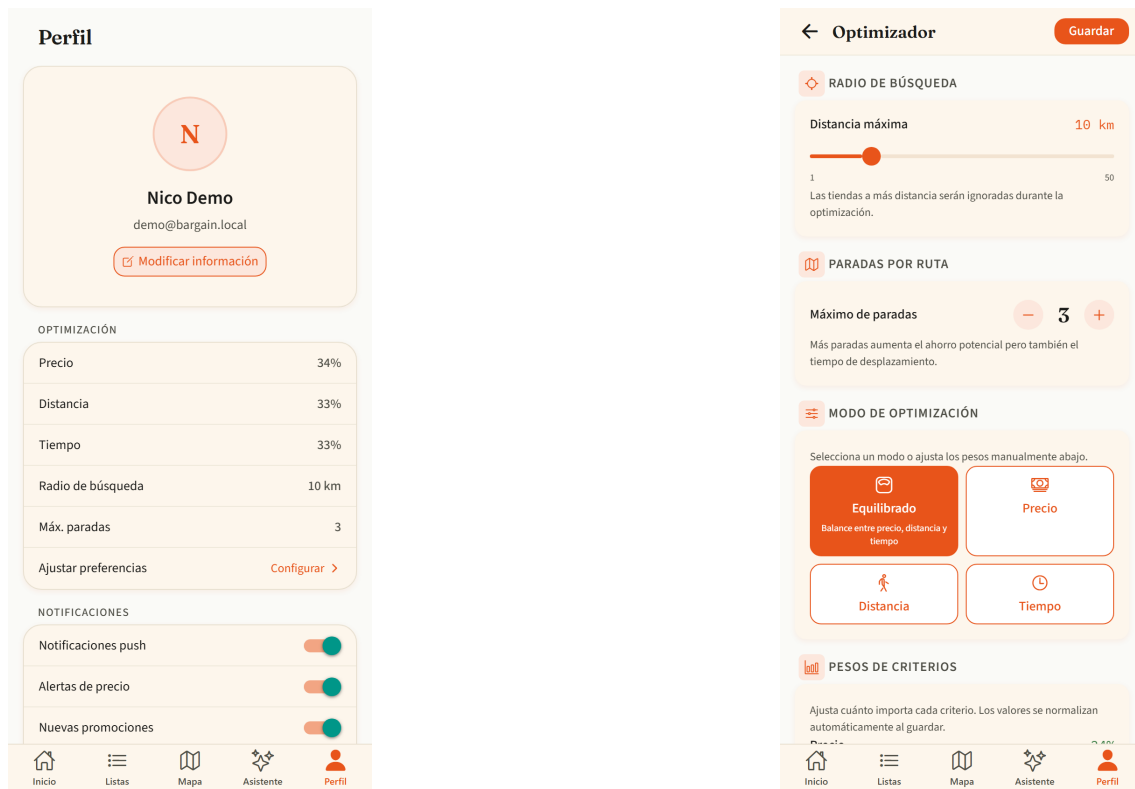


Figura 7.10: Perfil del usuario (izquierda) y configuración del optimizador con radio, paradas y pesos del *scoring* multicriterio (derecha)

Las operaciones sobre la cuenta se completan con la edición de los datos básicos y el cambio de contraseña (Figura 7.11), este último con verificación de la contraseña vigente y validación de robustez de la nueva.

Modificar información

Modificar información

Actualiza tus datos personales para mantener tu perfil al día.



Toca para cambiar la foto

Nombre

Apellidos

Email

Guardar cambios

Cambiar contraseña

Cambiar contraseña

Introduce tu contraseña actual y elige una nueva de al menos 8 caracteres.

Contraseña actual

Nueva contraseña

Confirmar contraseña

Guardar contraseña

Figura 7.11: Operaciones sobre la cuenta: edición del perfil (izquierda) y cambio de contraseña (derecha)

7.10— Ruta de compra optimizada

La optimización de rutas es la funcionalidad nuclear del proyecto y el punto donde convergen precios, geolocalización y preferencias del usuario. Desde el detalle de una lista, la acción *Ruta optimizada* conduce a la pantalla de la Figura 7.12, que resume el resultado de la última optimización persistida y permite recalcularla con la ubicación y los pesos actuales.

La cabecera recoge los parámetros empleados (radio y pesos), seguida del selector de paradas máximas (2–5) y del botón de recálculo. El bloque central presenta el resultado agregado —precio total estimado, distancia del recorrido, duración aproximada y número de paradas— y, a continuación, el desglose por parada: cada establecimiento de la ruta, con su distancia y tiempo parciales, enumera los productos que conviene adquirir en él junto con su precio. En el ejemplo de la figura, el optimizador ha asignado las manzanas a un comercio local porque su precio compensa el desvío conforme a los pesos configurados.

Cuando un ítem de la lista admite varias interpretaciones en el catálogo, la capa semántica respaldada por el modelo Gemini señala la ambigüedad y ofrece las alternativas (“Manzanas Fuji”, “Manzanas Golden”...), cada una con la acción *usar y recalcular*; la elección se persiste como preferencia de esa lista, de modo que las optimizaciones futuras ya no repiten la pregunta. El comportamiento de la pantalla es persistente: al salir y volver, se recupera la última ruta calculada desde la base de datos, y *Ver en mapa* abre el recorrido completo en la aplicación de mapas del dispositivo.



Figura 7.12: Ruta de compra optimizada: parámetros, resultado agregado, desglose por parada y resolución de ambigüedades semánticas con recálculo

7.11– Reconocimiento de tickets y listas (OCR)

El módulo OCR reduce drásticamente el coste de entrada de datos: en lugar de teclear una lista, el usuario fotografía un ticket de compra o una lista manuscrita y el sistema la convierte en ítems estructurados. El flujo sigue un asistente de tres pasos. En el primero (Figura 7.13, izquierda), la pantalla de captura ofrece la cámara o la galería como origen de la imagen. El segundo paso (Figura 7.13, derecha) presenta el resultado del reconocimiento —delegado en Google Cloud Vision y refinado con un emparejamiento difuso contra el catálogo—: cada línea detectada aparece con el texto original, el producto del catálogo con el que se ha emparejado y una etiqueta de confianza codificada por color (verde a partir del 80 %, amarillo entre el 50 y el 79 %, rojo por debajo). El usuario actúa como verificador final: corrige nombres en línea, ajusta cantidades con los selectores y descarta las líneas irrelevantes. En el tercer paso confirma la operación, eligiendo entre volcar los ítems a una lista existente o crear una nueva.

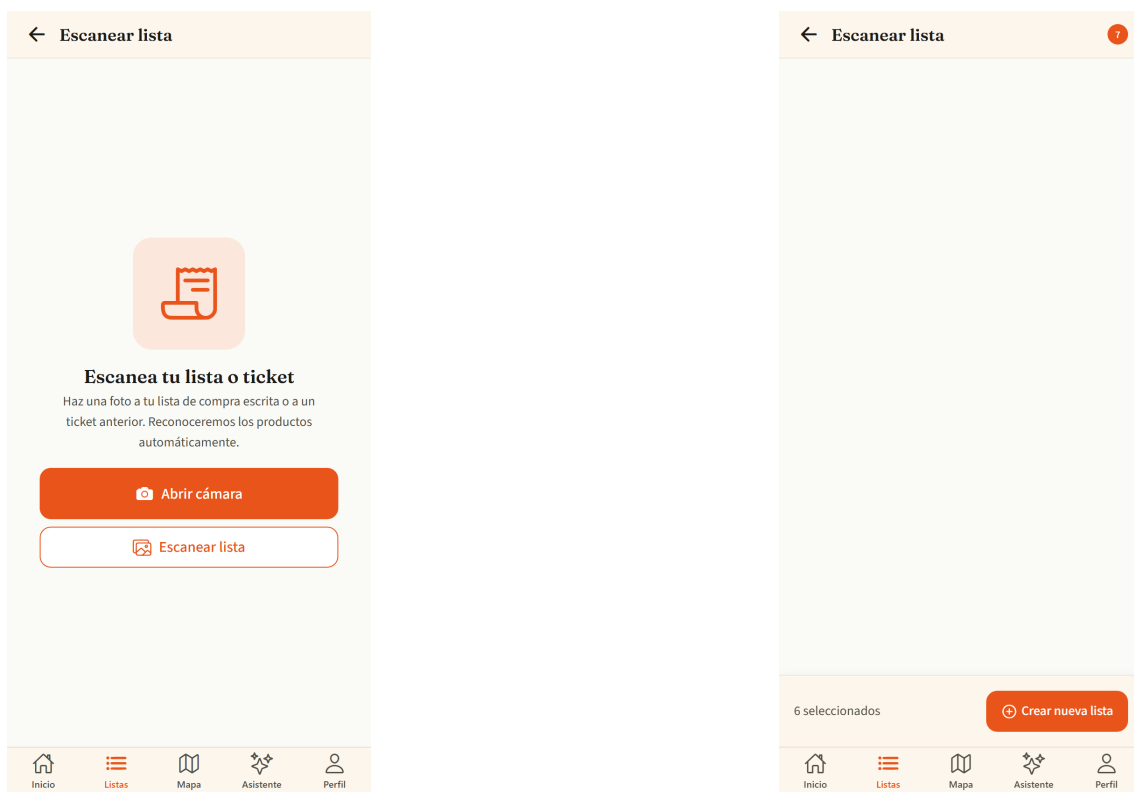


Figura 7.13: Asistente OCR: captura del ticket o lista (izquierda) y revisión de los ítems reconocidos con su nivel de confianza (derecha)

7.12– Asistente conversacional

El asistente (Figura 7.14) incorpora un modelo de lenguaje (Gemini, mediado por el backend conforme al ADR-008) especializado en la compra doméstica. La pantalla sigue el patrón conversacional estándar —mensajes del usuario a la derecha, respuestas del asistente a la izquierda— e incluye sugerencias de arranque que comunican el alcance del sistema: comparar precios entre cadenas, localizar la tienda más barata para un producto o pedir la optimización de la lista activa. Cuando una respuesta menciona productos, precios o tiendas, incorpora fichas interactivas que permiten añadir el producto a la lista sin abandonar la conversación. El asistente está acotado deliberadamente a su dominio:

ante preguntas ajenas a la compra responde declinando la consulta, como parte de las salvaguardas de uso descritas en el Capítulo 6.

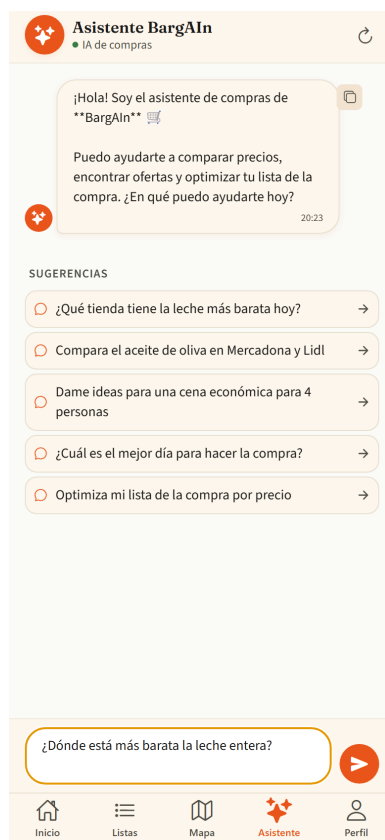


Figura 7.14: Asistente conversacional con sugerencias de inicio y campo de consulta; las respuestas con productos incluyen fichas de añadido directo a la lista

7.13— Lista de comprobación en pantalla de bloqueo (iOS)

Como complemento del flujo de compra presencial, desde la pantalla de ruta optimizada el botón *Checklist en bloqueo* publica una notificación interactiva con los productos de la parada actual. El usuario puede marcar cada artículo como comprado directamente desde la pantalla bloqueada del teléfono, sin desbloquear el dispositivo ni reabrir la aplicación; la notificación se actualiza tras cada acción. La versión basada en *Live Activities* de ActivityKit queda fuera del flujo gestionado de Expo y se documenta como trabajo futuro en el ADR-010.

7.14— La aplicación en el navegador de escritorio

La misma base de código Expo de la aplicación móvil se ejecuta en navegadores de escritorio, donde la interfaz se reorganiza para aprovechar la superficie adicional y los dispositivos de entrada propios del ordenador. Esta sección documenta dicha adaptación, que no es una mera ampliación del lienzo: cambia la disposición, la densidad de información y las *affordances* de interacción.

La pantalla principal (Figura 7.15, izquierda) centra el contenido a un ancho legible y convierte las acciones rápidas en una botonera horizontal. El catálogo (Figura 7.15, derecha) pasa de la lista vertical móvil a una rejilla de dos a cuatro columnas según el ancho de la ventana, con realces *hover* sobre las tarjetas; el atajo **Cmd/Ctrl+K** enfoca el buscador desde cualquier punto, **Enter** añade el producto resaltado a la lista y **Esc** cierra los diálogos abiertos.

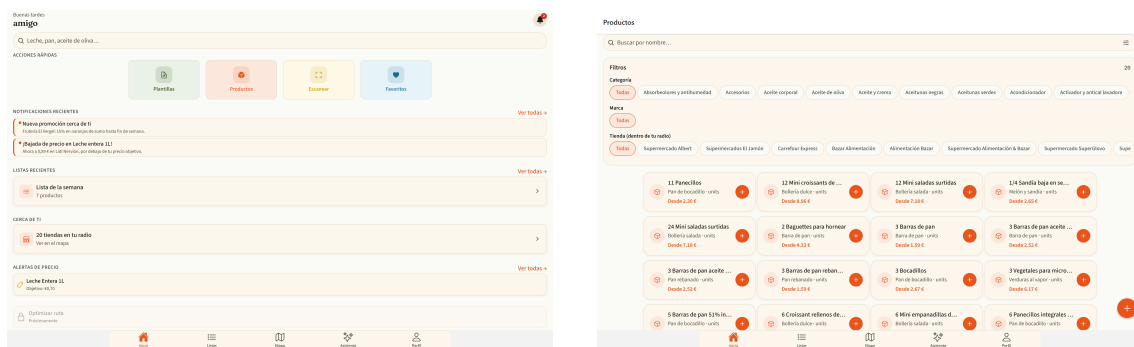


Figura 7.15: Versión de escritorio: pantalla principal con accesos horizontales (izquierda) y catálogo en rejilla multicolumna con soporte de teclado (derecha)

El resto de pantallas sigue el mismo patrón de adaptación; sus capturas se recogen en el Apéndice B para no sobrecargar este capítulo. En síntesis: la comparativa de precios (Figura B.1) gana una cabecera fija y exportación de la tabla a un fichero de valores separados por comas (CSV); el detalle de lista añade el reordenado de ítems por arrastre y la exportación en texto plano; las colecciones de listas y plantillas (Figura B.2) se presentan como rejillas de tarjetas; el mapa ancla el panel de tiendas como columna lateral y el perfil de tienda adopta dos columnas (Figura B.3); las tiendas favoritas y el asistente conversacional (Figura B.4) centran su contenido a un ancho de lectura cómodo, con copia individual de cada respuesta del asistente; la ruta optimizada incorpora la exportación del itinerario y el módulo OCR delega en el selector de ficheros del sistema cuando no hay cámara (Figura B.5); y la bandeja de notificaciones y las alertas (Figura B.6) se centran a un ancho fijo.

Cada pantalla relevante de la versión web dispone además de URL propia, de modo que cualquier estado —una búsqueda concreta del catálogo mediante **?q=**, una lista, una tienda— puede guardarse como marcador o compartirse como enlace.

7.15– Portal Business para PYMEs

El portal *business* es la contrapartida del sistema para el pequeño comercio: una aplicación web independiente desde la que un establecimiento local publica sus precios y promociones para competir en igualdad de condiciones con las grandes cadenas dentro de BarGAIN. Su página de presentación (Figura 7.16) expone la propuesta de valor para el comercio —visibilidad ante consumidores cercanos, gestión autónoma de precios y promociones, y estimaciones orientativas de retorno— y resume el proceso de alta en tres pasos.



Figura 7.16: Página de presentación del portal Business, con la propuesta de valor para el comercio local y el proceso de alta en tres pasos

El acceso al portal (Figura 7.17) reutiliza el mismo sistema de autenticación JWT de la aplicación de consumo, con cuentas de rol *business*. Tras el registro, el comerciante completa el *onboarding* (Figura 7.18): un formulario con los datos legales y comerciales del negocio —razón social, CIF/NIF, dirección y sitio web— que crea el perfil de empresa en estado *pendiente*. Ningún comercio publica precios sin que un administrador haya verificado previamente esos datos, una salvaguarda imprescindible para sostener la confianza del consumidor en la información que muestra la aplicación.

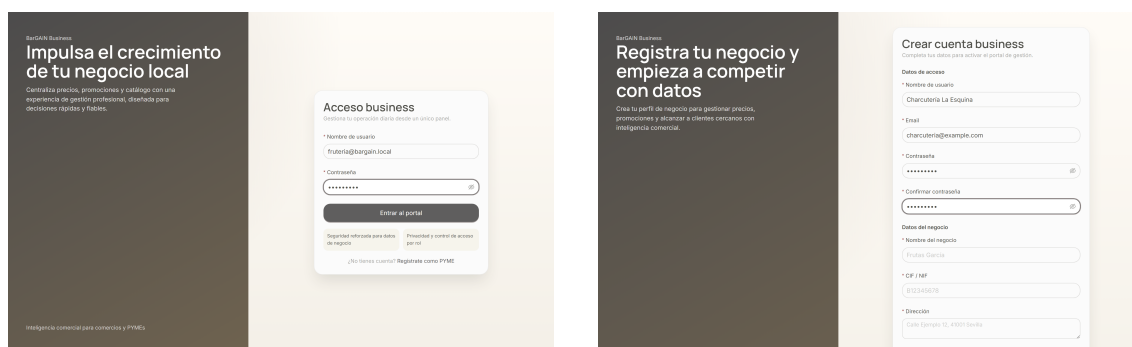


Figura 7.17: Acceso al portal Business: inicio de sesión (izquierda) y registro de una cuenta de comercio (derecha)



Figura 7.18: *Onboarding* del comercio: alta de los datos legales y comerciales que un administrador verificará antes de activar la cuenta

Una vez aprobado, el comerciante accede a su centro de operaciones (Figura 7.19): un panel ejecutivo con los indicadores esenciales del negocio en la plataforma —tiendas activas, precios publicados, promociones vigentes y visualizaciones de perfil acumuladas— , la salud de sus campañas y los últimos ajustes de precio aplicados. El propósito del panel es que un comerciante sin formación analítica pueda responder de un vistazo a las dos preguntas que le importan: qué estoy ofreciendo ahora mismo y cuánta atención está recibiendo.

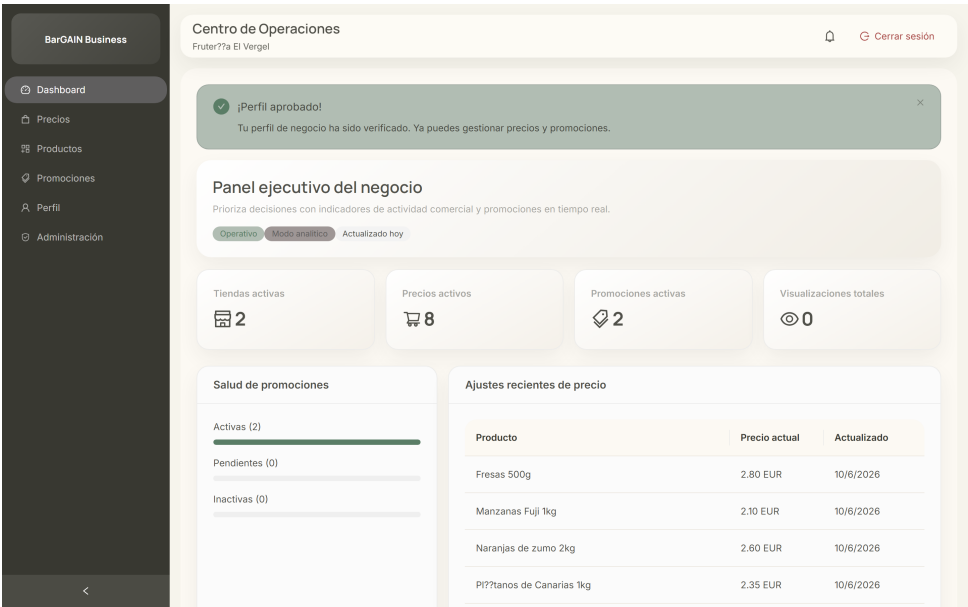


Figura 7.19: Centro de operaciones del comercio: indicadores de actividad, salud de promociones y ajustes recientes de precio

La gestión del surtido se apoya en dos vistas complementarias. La pantalla de productos (Figura 7.20, izquierda) resume el catálogo del comercio con el número de tiendas que ofrecen cada artículo, su precio base mínimo y la mejor oferta vigente. Para incorporaciones masivas, la carga por CSV (Figura 7.20, derecha) permite importar el catálogo completo en una sola operación, con validación fila a fila e informe de incidencias, lo que rebaja considerablemente la barrera de entrada para comercios con decenas de referencias.

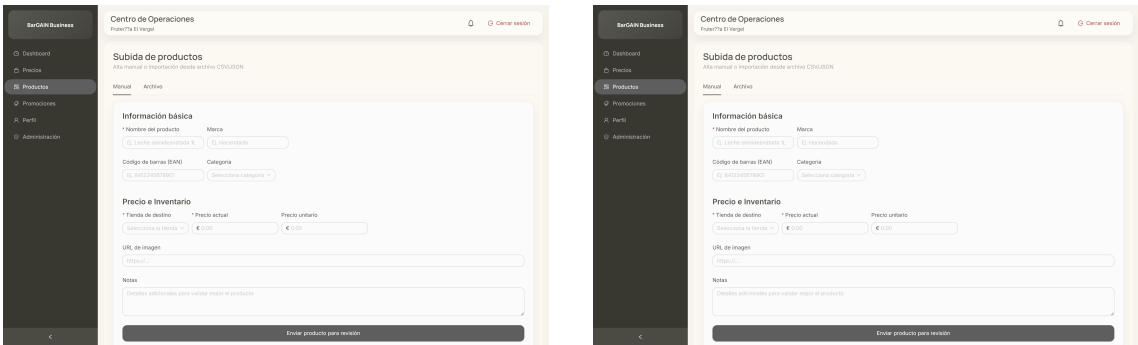


Figura 7.20: Gestión del surtido: resumen de productos del comercio (izquierda) y carga masiva de catálogo mediante CSV con validación (derecha)

El mantenimiento diario de precios se realiza en la tabla editable de la Figura 7.21: cada fila representa un precio publicado (producto–tienda) y el panel lateral permite modificar el precio base, el precio unitario y, opcionalmente, un precio de oferta con fecha de caducidad. Los cambios se publican de forma inmediata en la aplicación de consumo con la fuente *business*, que el sistema de prioridades de BarGAIN sitúa por delante de los datos de *scraping* por tratarse de información de primera mano.

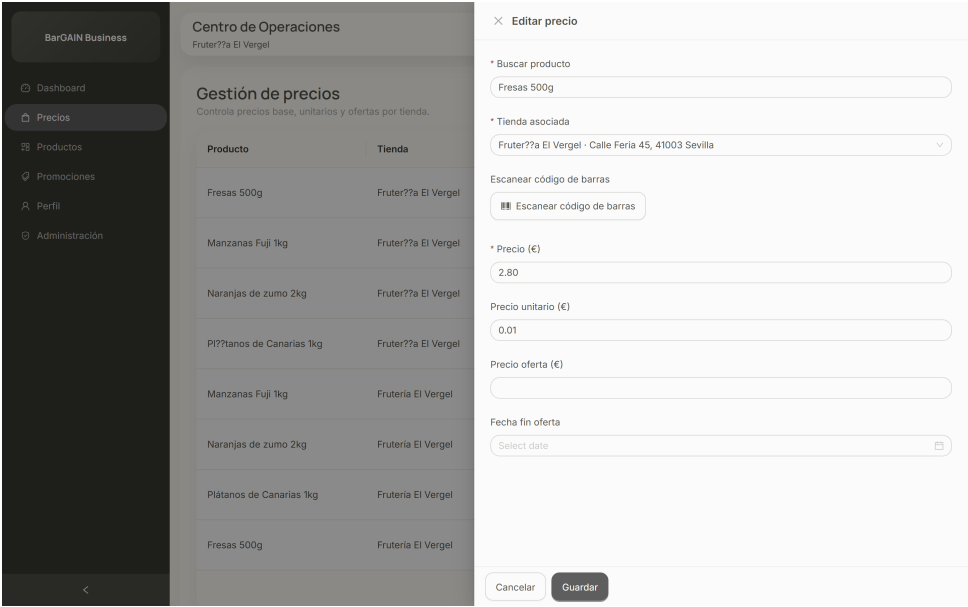


Figura 7.21: Gestión de precios: tabla editable con panel lateral de edición de precio base, unitario y oferta con caducidad

Sobre los precios se construyen las promociones (Figura 7.22, izquierda): descuentos fijos o porcentuales con fecha de inicio y fin, cantidad mínima opcional y título comercial, cuyo rendimiento (visualizaciones, estado) se supervisa desde el propio listado. Las promociones activas se difunden a los consumidores del entorno mediante notificaciones, cerrando el círculo entre la oferta del comercio y la demanda cercana. El perfil del negocio (Figura 7.22, derecha) completa la gestión con los datos públicos del establecimiento y el umbral de alerta competitiva: si un competidor del entorno publica un precio inferior en más del porcentaje configurado, el comerciante recibe un aviso automático.

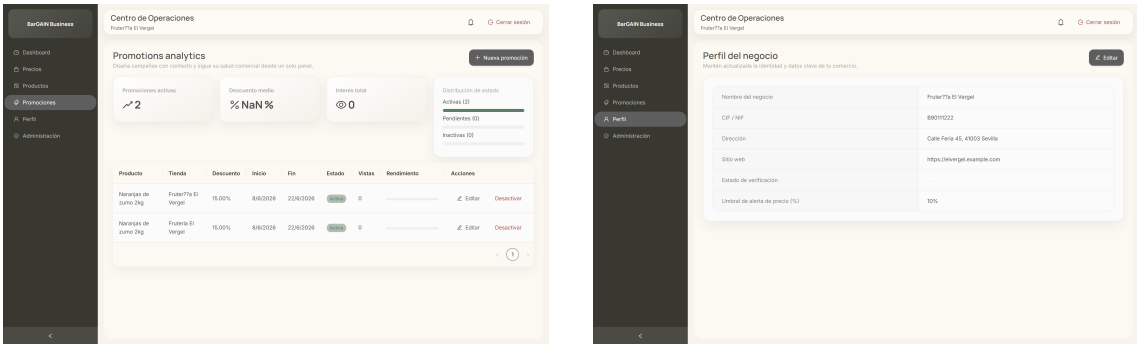


Figura 7.22: Promociones del comercio con analítica básica (izquierda) y perfil del negocio con umbral de alerta competitiva (derecha)

El ciclo administrativo se cierra en el panel de administración (Figura 7.23), reservado a usuarios con privilegios de gestión. En él se revisan los perfiles PYME pendientes —con sus datos legales a la vista— y las propuestas de producto remitidas por los consumidores, con acciones de aprobación y rechazo motivado. Este punto único de moderación garantiza que tanto los comercios como el catálogo crecen de forma controlada.

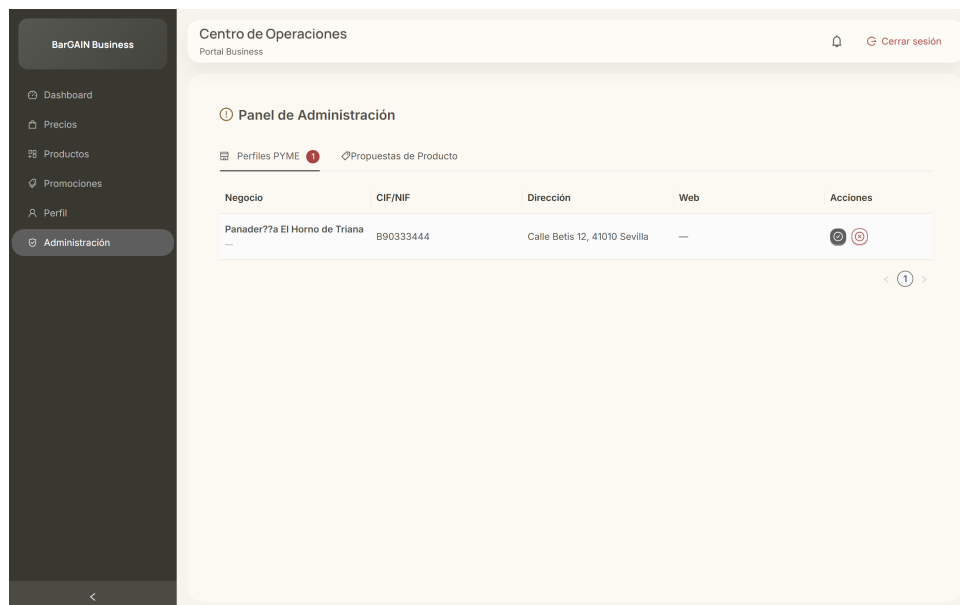


Figura 7.23: Panel de administración: verificación y aprobación de perfiles PYME y de propuestas de producto

7.16— Resolución de incidencias comunes

Se recogen, por último, las incidencias más habituales en el uso del sistema y su resolución:

- **El frontend no conecta con el backend:** verificar que el backend está levantado (`make dev`) y expuesto en `http://localhost:8000`.
- **Errores de autenticación al navegar:** cerrar sesión e iniciarla de nuevo para regenerar el par de tokens.
- **No aparecen tiendas en el mapa:** comprobar los permisos de ubicación del dispositivo o fijar unas coordenadas válidas desde las preferencias del perfil.
- **El OCR no reconoce texto:** procurar buena iluminación y texto legible; la opción de galería con una fotografía de mayor calidad suele resolver el problema.
- **La primera optimización tarda:** tras arrancar el backend, la primera optimización puede demorarse hasta 30 segundos (arranque en frío del servicio de rutas); las siguientes son prácticamente inmediatas.

Pruebas y Resultados

8.1— Objetivo

Validar que BarGAIN cumple los requisitos funcionales y no funcionales definidos en la Fase de Análisis, con cobertura automática de los flujos críticos y verificación documentada de los requisitos no funcionales clave (RNF-002, RNF-004, RNF-005).

8.2— Estrategia de testing

Se aplica una estrategia por capas, siguiendo el modelo de pirámide de tests:

- **Tests unitarios backend:** por módulo (`users`, `products`, `stores`, `prices`, `shopping_lists`, `business`, `notifications`, `optimizer`).
- **Tests de integración backend:** flujos cross-domain que ejercitan varios módulos en conjunto.
- **Tests de componentes frontend:** Jest + React Native Testing Library (111 tests en 15 suites), incluyendo los componentes y utilidades de la adaptación web.
- **Tests de extremo a extremo (E2E) con Playwright:** 4 flujos críticos del sistema integrado (backend + frontend web).
- **Pruebas de aceptación de usuario (UAT) en dispositivo móvil:** flujos nativos con GPS y cámara que no son automatizables mediante Playwright (verificados en iPhone con la app compilada).

8.3— Entorno de ejecución

- **Backend:** ejecución dentro del contenedor Docker (modelo híbrido ADR-002). Pytest con configuración en `backend/pytest.ini`, `DJANGO_SETTINGS_MODULE = config.settings.test`.
- **E2E Playwright:** ejecución nativa en host (no Docker) contra el backend Docker local o el staging de Render. Instalado en `frontend/web/` como dependencia de desarrollo.

Los comandos de ejecución principales son:

```
# Backend (desde raiz del repo)
make test-backend          # Tests con -v --tb=short
make test-backend-cov      # Con cobertura HTML

# E2E (desde host)
cd frontend/web && npx playwright test
cd frontend/web && npx playwright test --reporter=html
```

8.4– Suite de tests backend

8.4.1. Tests de integración

Los tests de integración verifican flujos API completos usando una base de datos de test real (sin mocks de BD), validando el comportamiento end-to-end del backend. La suite backend completa suma **333 tests**: 162 unitarios (17 archivos) y 171 de integración (17 archivos). El Cuadro 8.1 resume las suites de integración.

Cuadro 8.1: Suite de tests de integración backend

Archivo de test	Módulo	Cobertura funcional
test_auth_endpoints.py	users	Registro, login, refresh, perfil
test_optimizer_api.py	optimizer	POST /optimize/ con matriz mock
test_ocr_api.py	ocr	POST /ocr/scan/ con Vision API mock
test_business_verification.py	business	Aprobar/rechazar BusinessProfile
test_business_registration.py	business	Onboarding PYME
test_business_prices.py	business	Gestión de precios PYME
test_bulk_prices.py	business	Actualización masiva CSV
test_proposal_admin.py	business	Aprobar/rechazar propuestas
test_promotions.py	business	Gestión de promociones
test_list_endpoints.py	shopping_lists	CRUD de listas e ítems
test_cross_domain.py	core	Flujos cross-domain
test_notification_dispatch.py	notifications	Despacho push/email
test_notification_events.py	notifications	Eventos de notificación
test_price_endpoints.py	prices	API de precios
test_product_endpoints.py	products	API de productos
test_store_endpoints.py	stores	API de tiendas
test_assistant_api.py	assistant	Endpoint LLM con mock de Gemini

8.4.2. Cobertura

La suite backend mantiene una cobertura mínima del 80 % sobre los módulos de aplicación (apps/), medida con `pytest-cov`. El umbral no es declarativo sino **bloqueante**: el workflow `ci-backend.yml` ejecuta `pytest -cov=apps -cov-fail-under=80`, de modo que ninguna integración por debajo de ese umbral supera la CI; el informe se publica además en Codecov en cada build. La distribución de tests unitarios por módulo refleja dónde se concentra la lógica de negocio: `shopping_lists` (19), `products` (17), `prices` (15), `scraping` (22 entre spiders y pipeline), `stores` (11), `business` (11), `notifications` (11), `ocr` (9), `users` (8), `assistant` (8) y `optimizer` (15 entre solver, semántica y cliente de OpenRouteService, ORS), además de tests de configuración (despliegue Render y comandos de seed).

8.5— Tests E2E con Playwright

Se desarrollaron 4 tests E2E automatizados con Playwright para verificar los flujos críticos del sistema integrado (backend + web companion Vite+React):

Cuadro 8.2: Suite de tests E2E con Playwright

Flujo	Archivo	Tipo	Descripción
Autenticación completa	auth.spec.ts	UI + API	Registro → login → refresh → logout
Lista → Optimizar → Ruta	optimizer.spec.ts	API-level	Crear lista → optimizar → verificar
OCR ticket → lista	ocr.spec.ts	API-level	Subir imagen → OCR → respuesta
Business portal	business.spec.ts	UI	Registro PYME → aprobación → panel

Los flujos de optimizador y OCR se implementan como tests a nivel de API (usando el *fixture request* de Playwright, sin interfaz de usuario) porque estas funcionalidades son exclusivas de la app móvil — el portal web es exclusivamente de empresas y no dispone de pantallas de consumidor. Este enfoque verifica la integración backend end-to-end de los features principales del TFG sin requerir una interfaz web que no existe.

8.5.1. Configuración Playwright

- **Ubicación:** frontend/web/playwright.config.ts
- **Target:** web companion en http://localhost:5173 (o PLAYWRIGHT_BASE_URL)
- **Backend:** Docker local o staging Render (PLAYWRIGHT_API_URL)
- **Navegador:** Chromium (un solo proyecto para evitar duplicación de estado de BD)
- **Workers:** 1 (secuencial — los tests comparten la misma base de datos)

8.6— UAT manual en dispositivo móvil

Los flujos que requieren APIs nativas del dispositivo (GPS, cámara) se verifican manualmente en un iPhone con la app compilada mediante el workflow ios-build.yml:

Cuadro 8.3: Resultados UAT en dispositivo móvil

Flujo UAT	Resultado
Solicitar ubicación y ver tiendas en mapa	Verificado
Crear lista y añadir ítems manualmente	Verificado
Capturar ticket con cámara y reconocer productos	Verificado
Lanzar optimización y navegar la ruta sugerida	Verificado
Chat con asistente LLM y verificar guardrail	Verificado

8.7— Validación de Requisitos No Funcionales

8.7.1. RNF-002: Disponibilidad del entorno de staging

El backend de staging se desplegó en Render.com (plan gratuito, blueprint render.free.yaml) con la siguiente arquitectura:

- Web Service único: Django con Gunicorn (2 workers) y un proceso Celery worker + beat embebido (`-concurrency 1`) para las tareas asíncronas.
- PostgreSQL 16 + PostGIS gestionado por la plataforma.
- Redis: broker Celery + caché Google Places.

El health check se configura en `GET /api/v1/health/` respondiendo HTTP 200. Los reinicios automáticos ante fallos se gestionan por la plataforma Render. Como optimización del arranque en frío del free tier, los ficheros estáticos se precompilan durante el build de la imagen Docker en lugar de generarse en el arranque del servicio.

Debe precisarse el alcance de la validación: el enunciado original de RNF-002 (disponibilidad mensual $\geq 99\%$) **no es medible tal cual en el free tier**, porque la plataforma hiberna el servicio tras unos minutos de inactividad (Sección 7.2) y no se ha desplegado monitorización continua de uptime. Lo que se ha verificado es la *disponibilidad funcional durante las ventanas de evaluación*: el health check responde correctamente tras cada despliegue, y los cuatro flujos E2E se ejecutan con éxito contra el staging público. La instrumentación de monitorización continua (y, con ella, la verificación del 99 % en sentido estricto) queda registrada como trabajo pendiente en el Capítulo 9.

8.7.2. RNF-004: Usabilidad y flujo máximo de 3 interacciones

Los flujos principales se completan en ≤ 3 interacciones en la app móvil:

- **Ver precios cercanos:** Home $\rightarrow$ Buscar $\rightarrow$ Resultados (3 taps)
- **Optimizar ruta:** Lista $\rightarrow$ Optimizar $\rightarrow$ Ver ruta (3 taps)
- **Añadir ítem OCR:** Lista $\rightarrow$ Cámara $\rightarrow$ Confirmar (3 taps)

Respecto al criterio de accesibilidad de RNF-004, la verificación realizada ha sido *heurística* (flujos cortos, etiquetas de formularios, estados de foco en la versión web y *tooltips* accesibles): no se ha ejecutado una auditoría formal WCAG 2.1 AA (World Wide Web Consortium (W3C), 2018) con herramientas específicas (contraste, navegación por teclado completa, lector de pantalla), que queda identificada como tarea previa a cualquier publicación en tiendas de aplicaciones.

8.7.3. RNF-005: Escalabilidad para 10.000 usuarios

La arquitectura de BarGAIN soporta el crecimiento a 10.000 usuarios concurrentes mediante cuatro decisiones de diseño complementarias:

1. **Workers Celery independientes:** el scraping, el OCR y la optimización se ejecutan en workers separados del API principal, evitando que tareas pesadas bloqueen la gestión de peticiones web.
2. **Redis como broker desacoplado:** las peticiones de usuario se encolan en Redis y se procesan de forma asíncrona, permitiendo absorber picos de carga sin timeout de petición.
3. **PostGIS con índices GiST:** las consultas de tiendas cercanas utilizan índices espaciales con complejidad $O(\log n)$, independiente del volumen total de tiendas registradas.
4. **API stateless con JWT:** el backend no mantiene estado de sesión en servidor. Escalar horizontalmente (añadir réplicas del Web Service) no requiere sincronización de sesiones.

8.8— Resumen de resultados

Cuadro 8.4: Resumen de resultados de la estrategia de testing

Tipo de test	Cantidad	Estado
Tests unitarios backend	162 (17 archivos)	Todos pasan
Tests de integración backend	171 (17 archivos)	Todos pasan
Tests frontend (Jest)	111 (15 suites)	Todos pasan
Tests E2E Playwright	4 flujos	Todos pasan
UAT manual móvil	5 flujos	Verificados
RNF-002 · Disponibilidad	Health check + E2E en staging	Verificación funcional
RNF-004 · Usabilidad	3 flujos	Verificación heurística (3 taps máx.)
RNF-005 · Escalabilidad	Justificación arquitectural	Documentado

8.9— Conclusión

La estrategia de testing multicapa garantiza la calidad del sistema a diferentes niveles de granularidad. La cobertura de tests backend se mantiene por encima del 80 % con umbral bloqueante en CI (333 tests entre unitarios y de integración), la suite frontend añade 111 tests de componentes y utilidades con Jest, y los 4 flujos E2E automatizados con Playwright verifican que el sistema integrado funciona correctamente de extremo a extremo contra el entorno de staging. Quedan explícitamente acotadas las validaciones pendientes: medición formal de latencias p95 (RNF-001), monitorización continua de disponibilidad (RNF-002), auditoría WCAG completa (RNF-004) y prueba de carga (RNF-005), todas recogidas como trabajo futuro en el Capítulo 9.

Conclusiones

9.1— Grado de cumplimiento

El proyecto BarGAIN ha completado todos los bloques de desarrollo planificados:

- **F1:** análisis, requisitos y diseño de arquitectura.
- **F2:** infraestructura base de desarrollo y CI/CD.
- **F3:** backend core completo con todos los módulos de dominio y API documentada.
- **F4:** aplicación de usuario, básica y avanzada — autenticación, listas, catálogo, mapa, notificaciones, perfil e información enriquecida de los establecimientos.
- **F5:** sistema de optimización multicriterio (OR-Tools + OpenRouteService), scraping de precios (Scrapy + Playwright, 11 spiders implementados y 4 en operación programada), reconocimiento de tickets (Google Cloud Vision con emparejamiento aproximado), asistente conversacional (Google Gemini) y conexión de las pantallas móviles con los servicios reales del backend.
- **F6:** portal business completo para PYMEs — servicio de aprobación compartido, gestión de precios y promociones, tests de integración, UAT verificada, notificaciones push y email, sincronización documental.
- **F7:** pruebas integrales (unitarias, de integración y E2E), despliegue en staging y cierre documental de la memoria.

Adicionalmente, tras el cierre del plan inicial se incorporó una fase de adaptación de la app de usuario al uso web de escritorio (diseño adaptable, atajos de teclado, exportaciones y enlaces directos), documentada en los Capítulos 6 y 7.

El sistema está desplegado en staging (Render.com) y los cuatro flujos E2E críticos han sido verificados de forma automatizada con Playwright.

9.2— Aportaciones principales del trabajo

- **Optimizador multicriterio propio con interpretación de la lista:** resolución de cada artículo combinando semejanza textual, un modelo de lenguaje y emparejamiento aproximado con desambiguación interactiva; consolidación de paradas; y ordenación de la ruta como un problema del viajante sobre un grafo cuyas aristas ponderan precio, distancia y tiempo, con distancias reales de conducción. Es el núcleo diferencial del TFG.

- **Recogida de precios automatizada:** once recolectores implementados y verificados con pruebas, de los que cuatro (Mercadona, Carrefour, Lidl y DIA) operan de forma programada y normalizan cada producto contra el catálogo interno.
- **Reconocimiento de tickets:** extracción del texto, emparejamiento aproximado contra el catálogo y conversión automática en artículos de la lista, eliminando la entrada manual.
- **Portal business para PYMEs:** flujo completo de registro, aprobación por administrador, gestión de precios y promociones, con visibilidad para el consumidor final.
- **Asistente LLM con restricción de dominio:** modelo Gemini ceñido temáticamente a consultas de compra, con historial de conversación y limitación de peticiones.
- **Arquitectura full-stack coherente:** separación clara entre apps Django, workers Celery independientes, PostGIS para geoespacial y React Native + Expo para iOS y web desde una única base de código.
- **Verificación automatizada:** suite de tests backend (unitarios + integración, >80 % cobertura), 111 tests de frontend con Jest, tests E2E con Playwright (4 flujos críticos) y UAT manual en iPhone.

9.3— Limitaciones

- **Cobertura de scraping:** el sistema opera de forma programada contra cuatro cadenas (Mercadona, Carrefour, Lidl, DIA). Los otros siete spiders implementados (Alcampo, Eroski, Hipercor, Consum, Covirán, Spar, Costco) están verificados a nivel de parser pero no validados de forma continuada contra los sitios reales, y las cadenas con protección agresiva contra scrapers requieren mantenimiento adicional. Asimismo, la opción global de respeto a `robots.txt` del proyecto Scrapy debe activarse antes de cualquier operación a escala (Capítulo 6).
- **Caducidad de precios:** los precios obtenidos de forma automática caducan a las 48 horas; pueden no reflejar cambios de precio intradiarios.
- **Despliegue en plan gratuito (Render):** la infraestructura pública del proyecto se sostiene sobre el *free tier* de Render, lo que impone tres restricciones operativas. Primera, la *hibernación*: tras unos minutos sin tráfico la instancia se suspende y la siguiente petición sufre un *cold start* de entre 30 y 60 segundos mientras el contenedor se reactiva; el manual de usuario advierte de ello explícitamente. Segunda, los recursos computacionales (CPU y memoria compartidas) limitan el rendimiento del optimizador en escenarios de carga. Tercera, la base de datos del plan gratuito presenta restricciones de capacidad y vigencia que la hacen apta para demostración, pero no para producción. La migración a un plan de pago o a la infraestructura AWS prevista en el Capítulo 6 eliminaría estas tres limitaciones sin cambios de código.
- **Certificado iOS gratuito:** la distribución con Sideloadly y Apple ID gratuito genera certificados con caducidad de 7 días, que requieren re-sideload periódico para el dispositivo de demo.
- **RNF-005 sin prueba de carga real:** la justificación de escalabilidad a 10.000 usuarios concurrentes es arquitectónica (procesos Celery independientes, índice espacial de PostGIS y API sin estado). Una prueba de carga completa requeriría infraestructura de pago fuera del alcance del TFG.

- **Validaciones no funcionales pendientes:** no se ha realizado una campaña formal de medición de latencias p95 (RNF-001) ni se ha instrumentado monitorización continua de disponibilidad (RNF-002); la conformidad WCAG 2.1 AA (RNF-004) se ha verificado solo de forma heurística. Las tres validaciones quedan acotadas en el Capítulo 8 y recogidas como trabajo futuro.
- **Una única ruta recomendada:** el diseño preliminar contemplaba devolver tres rutas alternativas; la implementación final prioriza una única ruta con desambiguación semántica interactiva (Capítulo 6). La generación de alternativas se pospone como trabajo futuro.

9.4— Lecciones aprendidas

- **Modelo híbrido backend Docker / frontend nativo en host:** determinante para la productividad en Windows. Docker volumes en Windows rompen la recarga de módulos en caliente (HMR) de Metro/Expo; ejecutar el frontend nativo elimina este problema manteniendo el backend aislado.
- **Contratos API explícitos y documentación viva:** mantener TASKS.md, ADRs y la memoria actualizados durante el desarrollo (no al final) reduce la fricción entre backend y frontend y facilita la incorporación de agentes IA al flujo.
- **OR-Tools frente a algoritmo propio:** usar la librería de optimización combinatoria de Google (producción en empresas logísticas) es preferible a reinventar el VRP desde cero. El valor del TFG está en la integración y la función de scoring, no en reimplementar un solucionador genérico.
- **OCR cloud vs local:** Google Vision API supera a Tesseract en precisión sobre tickets de supermercado reales (texto impreso variable), justificando el coste de la llamada API frente a la alternativa local.
- **Pruebas a nivel de API para flujos exclusivos del móvil:** las funciones de lista y optimizador solo existen en la aplicación móvil. Verificarlas directamente contra la API, sin pasar por una interfaz, permite automatizar por completo su comprobación sin necesidad de una versión web equivalente.

9.5— Trabajo futuro

1. **Publicación en App Store y Google Play:** el proyecto está técnicamente listo para iniciar el proceso de publicación. Requiere cuenta de desarrollador Apple/-Google y revisión de las guías de contenido.
2. **Activación y ampliación de scrapers:** poner en operación programada y validar contra los sitios reales los siete spiders ya implementados (Alcampo, Eroski, Hipercor, Consum, Covirán, Spar, Costco), activar el respeto global a `robots.txt`, e incorporar nuevas cadenas (Aldi, El Corte Inglés alimentación) y PYMEs locales vía integración API propia.
3. **Precios en tiempo real:** varios supermercados están desplegando interfaces semipúblicas o canales de difusión de precios; la arquitectura está preparada para incorporarlos como una nueva fuente oficial de datos sin cambios de diseño.

4. **Sistema de reseñas de productos:** los usuarios podrían valorar la calidad/frescura de productos por tienda, añadiendo una dimensión cualitativa al comparador.
5. **Suscripción premium para PYMEs:** el modelo de negocio contempla tres niveles de servicio —gratis, básico y premium— con funcionalidades diferenciadas; la implementación actual corresponde al nivel gratis.
6. **Internacionalización:** la arquitectura soporta múltiples países; la expansión requiere scrapers por mercado y ajuste de la lógica de normalización de cadenas.
7. **Rutas alternativas y validación no funcional completa:** generar las top-3 rutas alternativas del diseño preliminar sobre el pipeline actual, instrumentar monitorización continua de disponibilidad, medir latencias p95 contra RNF-001 y ejecutar una auditoría WCAG 2.1 AA (World Wide Web Consortium (W3C), 2018) con prueba de carga para RNF-005.

9.6— Cierre

BarGAIN demuestra que es posible construir, en el marco de un TFG, un sistema full-stack de complejidad real que integra optimización combinatoria, geoespacial, procesamiento de imágenes, generación de lenguaje natural y scraping automatizado bajo una arquitectura coherente y verificada. El proyecto cumple los objetivos planteados en la fase de análisis —con las desviaciones de diseño y validaciones pendientes documentadas explícitamente en los Capítulos 6 y 8— y deja una base técnica sólida para su evolución como producto real.

APÉNDICE A

Relación de endpoints de la API

Todos los endpoints cuelgan del prefijo `/api/v1/` y, salvo indicación contraria, requieren autenticación JWT. La especificación OpenAPI completa se publica en `/api/v1/schema/` (con interfaces Swagger UI y ReDoc).

Cuadro A.1: Endpoints de la API por módulo

Método	Ruta	Descripción
GET	<code>health/</code>	Health check público (exento de throttling)
POST	<code>auth/register/</code>	Registro de usuario (público)
POST	<code>auth/token/</code>	Obtención de tokens JWT (público)
POST	<code>auth/token/refresh/</code>	Renovación del token de acceso
POST	<code>auth/password-reset/</code>	Solicitud de restablecimiento (público)
POST	<code>auth/password-reset/confirm/</code>	Confirmación de restablecimiento
GET/PATCH	<code>auth/profile/</code>	Perfil y preferencias del usuario
GET	<code>products/</code>	Listado y búsqueda fuzzy de productos
GET	<code>products/{id}/</code>	Detalle de producto
GET	<code>products/categories/</code>	Jerarquía de categorías
POST	<code>products/proposals/</code>	Propuesta de alta de un nuevo producto
*	<code>products/proposals/admin/</code>	Moderación de propuestas (administrador)
GET	<code>stores/</code>	Búsqueda geoespacial (<code>lat</code> , <code>lng</code> , <code>radius_km</code>) o favoritas
GET	<code>stores/{id}/</code>	Detalle de tienda
POST	<code>stores/{id}/favorite/</code>	Alternar tienda favorita
POST	<code>prices/compare/</code>	Comparativa de un producto entre tiendas
POST	<code>prices/list-total/</code>	Coste total de una lista por tienda
GET	<code>prices/{product_id}/history/</code>	Histórico de precios
*	<code>prices/alerts/</code>	CRUD de alertas de precio
*	<code>lists/</code>	CRUD de listas de la compra e ítems
*	<code>lists/templates/</code>	CRUD de plantillas de lista
POST	<code>optimize/</code>	Calcular o recalcular la ruta óptima
GET	<code>optimize/?shopping_list_id={id}</code>	Última ruta persistida de una lista
POST	<code>optimize/choices/</code>	Resolución de ambigüedad semántica y recálculo
POST	<code>ocr/scan/</code>	Procesado OCR de imagen (límite 60/h)

Método	Ruta	Descripción
POST	assistant/chat/	Mensaje al asistente LLM (límite 30/h)
*	business/profiles/	Alta y gestión de perfiles PYME, con acciones de aprobación y rechazo para administradores
*	business/prices/	Gestión de precios del comercio (incl. carga CSV)
*	business/promotions/	Gestión de promociones
*	notifications/	Buzón de notificaciones (lectura, borrado lógico)
POST	notifications/push-token/	Registro del token push del dispositivo

Las filas marcadas con * corresponden a *routers* DRF que exponen el conjunto estándar de operaciones (GET de colección y detalle, POST, PATCH, DELETE) más las acciones específicas indicadas.

APÉNDICE B

Capturas de la versión de escritorio

Este apéndice complementa la Sección 7.14 con las capturas restantes de la aplicación de usuario ejecutada en navegador de escritorio (ventana de 1440×900).

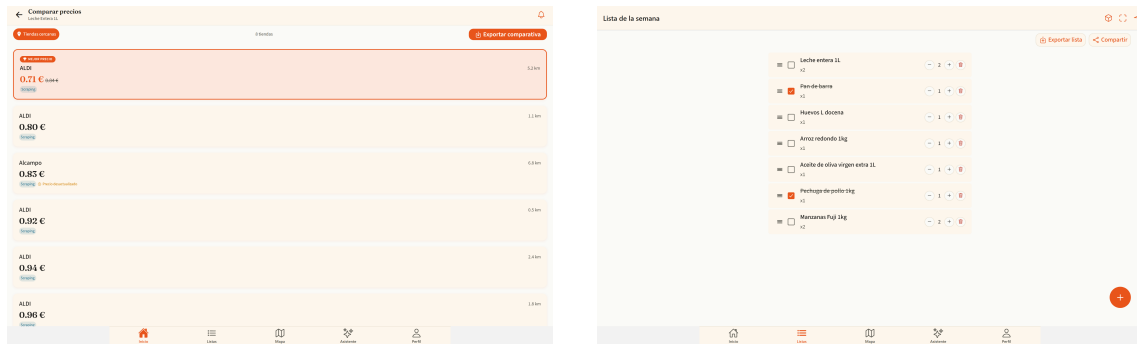


Figura B.1: Comparativa de precios con cabecera fija y exportación CSV (izquierda) y detalle de lista con reordenado por arrastre y exportación (derecha)

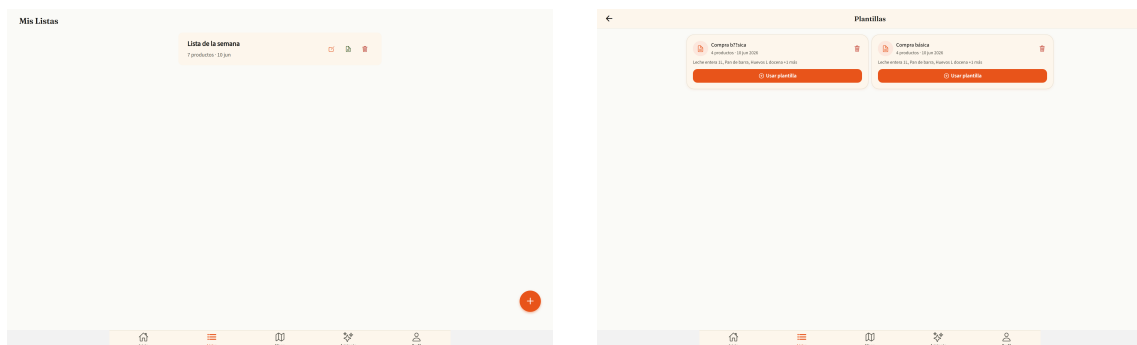


Figura B.2: Colecciones en rejilla responsiva: listas de la compra (izquierda) y plantillas (derecha)

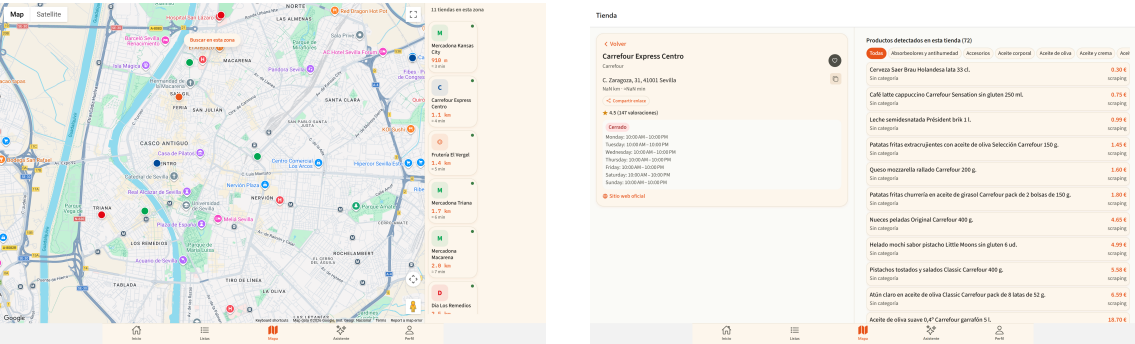


Figura B.3: Mapa con panel lateral de tiendas (izquierda) y perfil de tienda a dos columnas (derecha)

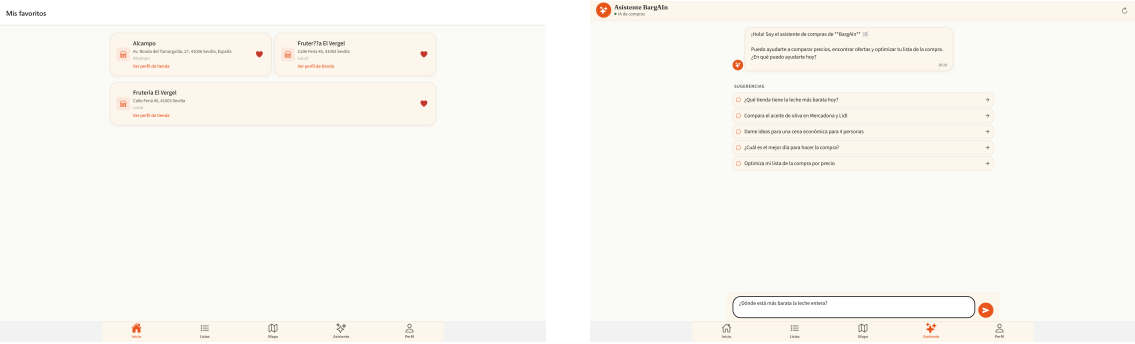


Figura B.4: Tiendas favoritas en rejilla (izquierda) y asistente conversacional centrado con copia por mensaje (derecha)

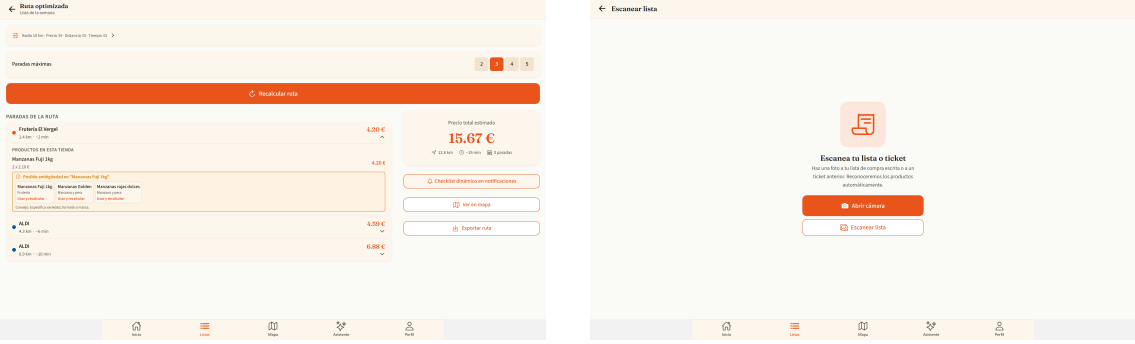


Figura B.5: Ruta optimizada con exportación del itinerario (izquierda) y módulo OCR adaptado al escritorio (derecha)

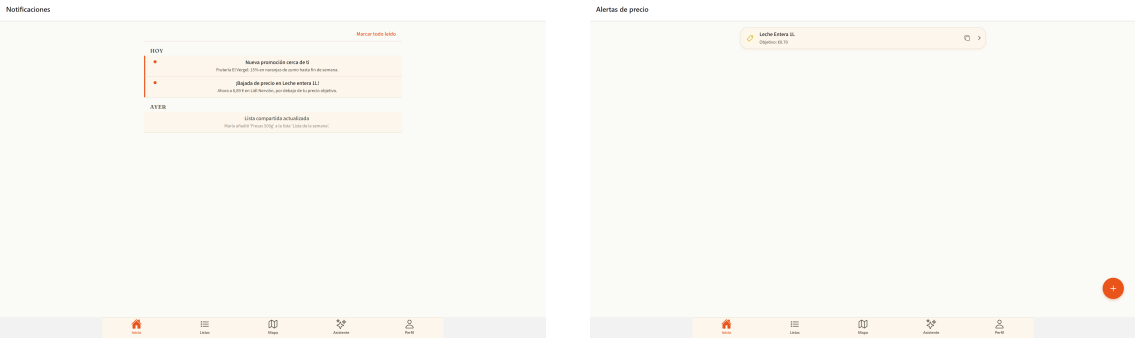


Figura B.6: Bandeja de notificaciones (izquierda) y alertas de precio (derecha) en la versión de escritorio

Referencias

- Applegate, D. L., Bixby, R. E., Chvatal, V., y Cook, W. J. (2006). *The traveling salesman problem: A computational study*. Princeton University Press.
- Brown, T. B., y cols. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
- Christopherson, L. (2026). *GSD (Get Stuff Done): A spec-driven development system for AI coding agents*. <https://github.com/gsd-build/get-shit-done>. (Accessed: 2026-06-20)
- Django Software Foundation. (2026). *Django documentation*. <https://docs.djangoproject.com/>. (Accessed: 2026-04-09)
- Encode OSS Ltd. (2026). *Django rest framework documentation*. <https://www.django-rest-framework.org/>. (Accessed: 2026-04-09)
- Google. (2026). *Gemini api documentation*. <https://ai.google.dev/gemini-api/docs>. (Accessed: 2026-06-12)
- Google Cloud. (2026). *Cloud vision documentation*. <https://cloud.google.com/vision/docs>. (Accessed: 2026-04-09)
- Google OR-Tools Team. (2026). *Or-tools documentation*. <https://developers.google.com/optimization>. (Accessed: 2026-04-09)
- Heidelberg Institute for Geoinformation Technology (HeiGIT). (2026). *Openrouteservice documentation*. <https://openrouteservice.org/>. (Accessed: 2026-06-12)
- Instituto Nacional de Estadística. (2026). *Indice de precios de consumo (ipc). base 2021*. https://www.ine.es/dyngs/INEbase/es/operacion.htm?c=Estadistica_C&cid=1254736176802. (Accessed: 2026-06-12)
- Jazzband. (2026). *Simple jwt: A json web token authentication plugin for django rest framework*. <https://django-rest-framework-simplejwt.readthedocs.io/>. (Accessed: 2026-06-12)
- Jefatura del Estado. (2002). *Ley 34/2002, de 11 de julio, de servicios de la sociedad de la información y de comercio electrónico*. Boletín Oficial del Estado, BOE-A-2002-13758.
- Junta de Andalucía. (2018). *Informe final de la consulta preliminar del mercado de perfiles profesionales en el ámbito informático*.
- Koster, M., Illyes, G., Zeller, H., y Sassman, L. (2022). *Rfc 9309: Robots exclusion protocol*. <https://www.rfc-editor.org/rfc/rfc9309>. (Internet Engineering Task Force (IETF))
- Lewis, P., y cols. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
- Luxen, D., y Vetter, C. (2011). Real-time routing with openstreetmap data. En *Proceedings of the 19th acm sigspatial international conference on advances in geographic information systems*. doi: 10.1145/2093973.2094062
- Microsoft Playwright Team. (2026). *Playwright documentation*. <https://playwright.dev/docs/intro>. (Accessed: 2026-04-09)
- Obe, R. O., y Hsu, L. S. (2021). *Postgis in action* (3rd ed.). Manning Publications.

- Organizacion de Consumidores y Usuarios (OCU). (2026). *Ocu market*. <https://www.ocu.org/>. (Accessed: 2026-03-11)
- Parlamento Europeo y Consejo de la Union Europea. (2016). *Reglamento (ue) 2016/679 relativo a la proteccion de las personas fisicas en lo que respecta al tratamiento de datos personales (rgpd)*. Diario Oficial de la Union Europea, L 119.
- PostGIS Project Steering Committee. (2026). *Postgis documentation*. <https://postgis.net/documentation/>. (Accessed: 2026-04-09)
- Scrapy Developers. (2026). *Scrapy documentation*. <https://docs.scrapy.org/>. (Accessed: 2026-04-09)
- Soysuper S.L. (2026). *Soysuper: tu supermercado de supermercados*. <https://soysuper.com/>. (Accessed: 2026-03-11)
- World Wide Web Consortium (W3C). (2018). *Web content accessibility guidelines (wcag) 2.1. w3c recommendation*. <https://www.w3.org/TR/WCAG21/>. (Accessed: 2026-06-12)